数学（上册）

Morning Flower and Evening Oath

目录

第一章 高精度计算 **1**

1.1 高精度整数：有符号加减乘与小整数除法 1

第二章 快速幂 **7**

2.1 无符号 64 位模乘与模幂 7

第三章 数论 **8**

3.1 模算术简介 8

3.1.1 动态模整数与合数模逆元 8

3.1.2 静态模整数 12

3.1.3 模运算的适用条件 15

3.1.3.1 除法与解同余式不是一回事 15

3.1.3.2 知道整数可整除时，先升模数再整除 15

3.1.3.3 组合数的整数性不等于分母可逆 15

3.1.3.4 大指数要同时保留余数和阈值信息 15

3.2 素数 16

3.2.1 确定性 64 位 Miller–Rabin 判素 16

3.2.2 64 位判素与质因数分解 17

3.3 欧拉函数 18

3.3.1 单个数的欧拉函数 18

3.3.2 单个数的欧拉函数 18

3.4 筛法 19

3.4.1 区间筛素数 19

3.4.2 线性筛、欧拉函数与 Möbius 函数 20

3.4.3 线性筛：三张表的约定 20

3.4.4 区间筛：只保存目标区间 21

3.5 分解质因数 21

3.5.1 Pollard–Rho 因数分解 21

3.6 裴蜀定理 & 一次不定方程 23

3.6.1 扩展欧几里得与 Bézout 系数 23

3.6.2 一次不定方程：特解与正整数解范围 24

3.6.2.1 有符号矩形范围：把两维约束化成一个参数区间 24

3.7 费马小定理 & 欧拉定理 25

3.7.1 十进制大指数：扩展欧拉降幂 25

3.7.2 Carmichael：最小公共指数 26

3.8 模逆元 26

3.8.1 任意正模数的单个逆元 26

3.8.2 任意数组的批量逆元 27

3.8.3 连续整数的线性逆元表 29

3.8.4 任意数组的批量逆元 29

3.8.5 连续整数的线性逆元表 29

3.9 线性同余方程 30

3.9.1 线性同余方程 30

3.9.2 线性同余：最小解与最小周期 . . . 30
3.10 中国剩余定理 . . . 31
3.10.1 广义 CRT：逐项合并同余 . . . 31
3.10.2 Garner：大周期 CRT 与混合进制 . . . 33
3.10.3 扩展 CRT：逐式合并 . . . 35
3.10.4 Garner：总周期可以超过整数范围 . . . 35
3.11 升幂引理 . . . 36
3.11.1 素因子估值与升幂：先约去公共因子 . . . 36
3.11.1.1 基本运算与公共因子 . . . 36
3.11.1.2 奇素数：差式与和式的条件不同 . . . 36
3.11.1.3 素数 2：先分指数奇偶 . . . 36
3.11.1.4 把大整数整除转成指数条件 . . . 36
3.12 阶乘取模 . . . 37
3.12.1 Legendre 赋值与 Kummer 进位判据 . . . 37
3.12.1.1 组合数：逐位相加时进了几次位 . . . 37
3.12.1.2 素数幂为零与 Lucas 的边界 . . . 37
3.12.2 阶乘表：组合数与排列数 . . . 37
3.13 卢卡斯定理 . . . 38
3.13.1 Lucas：小素数模的大下标组合数 . . . 38
3.13.2 合数模数组合数（扩展 Lucas） . . . 39
3.13.3 Lucas：按素数进制逐位计算 . . . 42
3.13.4 合数模数组合数与插值 . . . 42
3.14 二次剩余 . . . 43
3.14.1 素数模平方根（Cipolla） . . . 43
3.14.2 素数模平方根 . . . 45
3.15 阶 & 原根 . . . 45
3.15.1 乘法阶与原根 . . . 45
3.15.2 乘法阶与原根 . . . 48
3.15.3 自动准备模数分解与原根 . . . 48
3.16 离散对数 . . . 49
3.16.1 扩展 BSGS . . . 49
3.17 高次剩余 & 单位根 . . . 51
3.17.1 素数模高次剩余：全部根的紧凑表示 . . . 51
3.17.2 素数模高次剩余与单位根 . . . 51
3.17.3 素数幂模高次剩余与根的提升 . . . 52
3.17.4 一般合数模高次剩余：CRT 组合 . . . 52
3.18 数论分块 . . . 52
3.18.1 整除分块：前缀和来源决定算法 . . . 52
3.18.1.1 按数据与资源选前缀和 . . . 52
3.19 狄利克雷卷积 . . . 53
3.19.1 幂的约数和：不除以 $p-1$ . . . 53
3.20 莫比乌斯反演 . . . 53
3.20.1 整除与反演：先辨认求和方向 . . . 53
3.20.1.1 约数和与倍数和不能混用 . . . 53
3.20.1.2 GCD 权值先反演，再交换求和 . . . 53
3.20.2 GCD 计数：缩放、边界与对称性 . . . 53
3.20.2.1 恰好等于 k：先缩放整除对象，再做矩形容斥 . . . 53

3.20.2.2 去掉顺序之前，先检查交换是否仍在定义域内 . . . 53
3.20.3 矩形 GCD 计数：缩放后容斥 . . . 54
3.21 狄利克雷双曲线法 & 杜教筛 . . . 54
3.21.1 杜教筛 . . . 54
3.21.2 杜教筛：预筛与缓存都不可省略 . . . 55
3.22 Min_25 筛 . . . 56
3.22.1 Min_25 积性函数求和 . . . 56
3.23 类欧几里德算法 . . . 60
3.23.1 有符号整除和：类欧几里德 . . . 60
3.23.2 整除和、带权和与平方和 . . . 62
3.23.3 类欧几里德：有符号整除求和 . . . 64
3.23.4 类欧几里德的三个求和量 . . . 65

第四章 多项式与生成函数 66
4.1 快速傅里叶变换 . . . 66
4.1.1 复数 FFT（可复用单位根） . . . 66
4.1.2 拆系数 FFT 任意模卷积 . . . 68
4.1.3 CDQ 分治卷积递推 . . . 71
4.1.4 浮点 FFT：舍入前先约束最坏误差 . . . 72
4.2 快速数论变换 . . . 73
4.2.1 可指定素数与原根的 NTT 卷积 . . . 73
4.2.2 精确有符号整数卷积 . . . 76
4.2.3 参数化 NTT 与模数适配 . . . 77
4.3 快速沃尔什变换 . . . 78
4.3.1 按位 OR、AND、XOR 卷积 . . . 78
4.3.2 不相交子集卷积 . . . 80
4.3.3 集合变换与不相交子集卷积 . . . 81
4.3.4 集合变换：逆变换对模数的要求 . . . 82
4.4 Chirp Z 变换 . . . 82
4.4.1 Chirp Z：等比点求值 . . . 82
4.5 多项式牛顿迭代 . . . 84
4.5.1 形式幂级数求逆 . . . 84
4.5.2 多项式带余除法 . . . 86
4.5.3 多项式带余除法：先反转再求逆 . . . 87
4.6 多项式多点求值 | 快速插值 . . . 88
4.6.1 任意点多项式多点求值 . . . 88
4.6.2 任意点多项式快速插值 . . . 91
4.7 多项式初等函数 . . . 93
4.7.1 形式幂级数开根（含前导零与无解） . . . 93
4.7.2 形式幂级数微积分、对数与指数 . . . 96
4.7.3 形式幂级数快速幂（十进制大指数） . . . 98
4.7.4 级数开根：前导零和自由尾项 . . . 100
4.7.5 ln 与 exp：只处理所需前缀 . . . 100
4.7.6 幂函数的大指数：保留三份指数信息 . . . 100
4.8 常系数齐次线性递推 . . . 101
4.8.1 特征多项式求线性递推第 k 项 . . . 101
4.8.2 快速递推与有理生成函数单项 . . . 102
4.8.3 线性递推：系数方向与外推条件 . . . 104

4.8.4 Bostan–Mori 与快速递推单项 . . . 105
4.9 多项式平移 | 连续点值平移 . . . 106
4.9.1 多项式系数平移 . . . 106
4.9.2 Taylor 平移与等比点求值 . . . 107
4.10 符号化方法 . . . 108
4.10.1 生成函数建模到模板：三个可核验例子 . . . 108
4.10.1.1 例一：不同盒子、相同球、每盒有上下界（OGF 乘积） . . . 108
4.10.1.2 例二：步长 1、2 的有序序列（OGF 求逆） . . . 108
4.10.1.3 例三：不同元素分成单点与无序对（EGF 指数） . . . 108
4.11 Lagrange 反演 . . . 109
4.11.1 隐式生成函数与 Lagrange 反演 . . . 109
4.11.1.1 适用条件与取系数公式 . . . 109
4.11.1.2 模型一按内部结点计数的满有序多叉树 . . . 109
4.11.1.3 模型二带标号有根树的 EGF . . . 109
4.11.1.4 落到现有接口及模数边界 . . . 109
4.12 普通生成函数 . . . 110
4.12.1 生成函数与递推 . . . 110

第五章 组合数学 111
5.1 排列组合 . . . 111
5.1.1 阶乘组合数 . . . 111
5.1.2 计数建模：先确认哪些对象被区分 . . . 112
5.1.2.1 OGF、EGF 的选择来自乘法，而非题目用词 . . . 112
5.1.3 球盒问题：八种情形 . . . 112
5.1.4 组合数选型：先看模数，再看下标与预算 . . . 113
5.2 容斥原理 . . . 114
5.2.1 容斥与二项式反演：恰好与至少 . . . 114
5.2.1.1 从交集和恢复恰好数量 . . . 114
5.2.1.2 至少数量与变换方向 . . . 114
5.2.1.3 例：有标号球放入有标号盒 . . . 114
5.2.1.4 直接使用现有组合数与模整数 . . . 114
5.2.2 组合计数 . . . 115
5.3 错位排列 . . . 115
5.3.1 错位排列 . . . 115
5.4 卡特兰数 . . . 115
5.4.1 Catalan 与投票问题：把前缀限制变成反射双射 . . . 115
5.4.1.1 弱前缀条件：减去首次走到负一的坏路径 . . . 115
5.4.1.2 严格领先：先取走必需的第一步 . . . 115
5.4.1.3 初始有余额：反射起点，而非只改终点 . . . 116
5.4.1.4 取模时优先使用差式，除法条件不能省 . . . 116
5.5 斯特林数与斯特林反演 . . . 117
5.5.1 定义与性质 . . . 117
5.5.1.1 行列快速求解：推导与选择 . . . 117
5.5.2 无符号第一类斯特林数：同一行 . . . 118
5.5.3 无符号第一类斯特林数：同一列 . . . 120
5.5.4 第二类斯特林数：同一行前缀 . . . 121
5.5.5 第二类斯特林数：同一列 . . . 123
5.5.6 斯特林反演与应用 . . . 125

5.6 贝尔数 . . . 126
5.6.1 贝尔数 . . . 126
5.7 伯努利数 . . . 126
5.7.1 伯努利数批量计算 . . . 126
5.7.2 伯努利数与自然数幂和 . . . 126
5.8 Eulerian Number . . . 127
5.8.1 欧拉数（Eulerian number） . . . 127
5.9 分拆数 . . . 128
5.9.1 整数分拆与重数上限 . . . 128
5.9.2 至多 m 个部分的分拆表 . . . 129
5.9.3 至多 m 个部分的分拆表（生成函数） . . . 130
5.9.4 整数分拆：重数限制的差一问题 . . . 131
5.9.4.1 三个容易混淆的限制 . . . 131
5.9.4.2 重数限制与禁止倍数部分 . . . 131
5.10 范德蒙德卷积 . . . 131
5.10.1 范德蒙德卷积及常用变形 . . . 131
5.11 Pólya 计数 . . . 132
5.11.1 Burnside 平均：任意模数下的精确除法 . . . 132
5.11.2 置换的循环长度分解 . . . 133
5.11.3 Pólya 环染色：旋转与翻转 . . . 134
5.11.4 群作用计数：先确定什么算同一个方案 . . . 135
5.11.5 项链与手链：从置换循环推公式 . . . 135
5.11.6 指定颜色库存：给每个循环分配整份颜色 . . . 135
5.11.7 群阶不可逆：先提升模数，再作整数除法 . . . 136

第六章 线性代数 137
6.1 矩阵 . . . 137
6.1.1 模矩阵乘法与幂 . . . 137
6.1.2 素数模矩阵求逆 . . . 139
6.1.3 Max-plus 矩阵幂 . . . 141
6.1.4 线性代数与生成树 . . . 141
6.2 行列式 . . . 142
6.2.1 任意正模数行列式：欧几里得消元 . . . 142
6.2.2 任意正模数行列式 . . . 143
6.3 线性基 . . . 144
6.3.1 64 位异或线性基 . . . 144
6.3.2 带位置限制的异或线性基 . . . 146
6.3.3 异或线性空间求交 . . . 149
6.3.4 Zassenhaus：同时求和空间与交空间 . . . 151
6.4 特征多项式 . . . 153
6.4.1 有限状态计数到线性递推：先证明阶数，再用 BM . . . 153
6.4.1.1 先把状态、方向和输出写清楚 . . . 153
6.4.1.2 Cayley–Hamilton 给出阶数至多 d . . . 153
6.4.1.3 为什么前 2d 项足以恢复这个标量序列 . . . 153
6.4.1.4 模型：不含连续子串 101 的二进制串 . . . 153
6.4.1.5 按目标选算法，并把采样成本算进去 . . . 153

第七章 线性规划 **155**
7.1 单纯形法 . . . 155
7.1.1 线性规划：两阶段单纯形 . . . 155

第八章 概率论 **161**
8.1 随机变量的数字特征 . . . 161
8.1.1 期望选型：计数拆指标，等待拆尾和 . . . 161
8.1.2 条件期望：先定义状态，再解自环 . . . 161
8.1.3 模概率：可逆的不只是输入分母 . . . 161

第九章 博弈论 **162**
9.1 公平组合游戏 . . . 162
9.1.1 SG 的前提、递推与异或策略 . . . 162
9.1.2 减法游戏小例与三类失效边界 . . . 162
9.1.3 巴什博弈 . . . 162
9.1.4 Nim 游戏 . . . 162
9.1.5 威佐夫博弈 . . . 162
9.1.6 斐波那契博弈 . . . 163
9.1.7 阶梯博弈 . . . 163
9.1.8 反 Nim（取走最后一颗者负） . . . 163
9.1.9 Nim-k（每次至多动 k 堆） . . . 163
9.1.10 树上删边（Green Hackenbush） . . . 163
9.1.11 二分图博弈（不重复走点） . . . 164
9.1.12 对称策略：环形取石子 . . . 164
9.1.13 策略窃取：Chomp . . . 164

第十章 数值算法 **165**
10.1 插值 . . . 165
10.1.1 拉格朗日插值与连续点优化 . . . 165
10.1.2 横坐标连续的 O(n) 拉格朗日插值 . . . 168
10.2 数值积分 . . . 169
10.2.1 有工作上限的自适应 Simpson 积分 . . . 169
10.2.2 数值积分：局部估计、误差预算与采样盲区 . . . 172
10.2.3 数值算法与几何 . . . 172
10.3 高斯消元 . . . 172
10.3.1 压位模 2 高斯消元与全部解 . . . 172
10.3.2 素数模高斯消元与全部解 . . . 174
10.3.3 实数高斯消元与数值解空间 . . . 177
10.3.4 模线性方程组：工厂工期 . . . 179
10.3.5 模 2 压位消元与完整解空间 . . . 180
10.3.6 模 2 消元后的最少开关用法 . . . 180

第十一章 Berlekamp–Massey 算法 **181**
11.1 Berlekamp–Massey：最短线性递推 . . . 181

第一部分 跨类依赖 **183**

附录 A 树状数组 **184**
A.1 树状数组第 k 小与组合依赖 . . . 184

附录 B 最短路问题 **186**
B.1 最短路 186
B.1.1 Floyd 全源最短路与原边方案 186

附录 C 图的匹配 **189**
C.1 二分图最大匹配 189
C.1.1 Hopcroft–Karp 最大匹配与最小点覆盖 189
C.2 一般图最大匹配 192
C.2.1 一般图最大基数匹配：带花树 192

附录 D 矩阵树定理 **196**
D.1 任意正模数带权矩阵树 196

第一章　高精度计算

1.1　高精度整数：有符号加减乘与小整数除法

功能：有符号任意精度整数：可由 `long long` 或十进制字符串构造并转回字符串，支持比较、取负、加、减、乘，除以一个 `int` 得商和余数，以及非负整数次幂。

核心思路：绝对值按 10^9 进制、低位在前存于 `d`，符号单独存于 `sign`；零为空数组且符号为正，运算后用 `trim` 去掉高位零保持规范。同号加法逐节进位；异号加法化为绝对值大者减小者，结果取绝对值较大者的符号；减法即加相反数。乘法是逐节竖式，用 `long long` 携带进位。除以小整数从高节往低节做长除法，商向零截断，余数随被除数的符号。幂用二进制快速幂。

思路中的情况	代码中的处理	行
由字符串构造	从末尾起每 9 位数字一组转成一节	30
转为字符串	最高节原样输出，其余节补足 9 位前导零	51
异号相加	若本数绝对值较小则交换两数，再逐节借位相减	81
同号相加	逐节相加并进位	100
乘法	第 i 节乘以 b 的每一节，累加到第 $i+j$ 节并进位	126
除以小整数	高节到低节长除法，`rem` 为当前余数	145

接口契约：BigInteger(x) 接收 long long，BigInteger(s) 接收可带正负号及前导零的非空十进制串；str() 返回规范化十进制，cmp(b) 返回 $-1/0/1$。支持一元负号、加减乘，divmod(int b) 返回商和余数，b≠0；商向零截断，余数与被除数同号或为 0。pow(unsigned e) 计算非负整数幂，约定 0 的 0 次方为 1。

d 是低位在前的 10^9 进制绝对值，sign 为 ±1；零为空 d 且 sign=1，勿直接修改。n,m 为分节数：加减比较 O(n+m)，乘法 O(nm+n+m)，小整数除法 O(n)，结果空间 O(n+m)；幂执行 O(log(e+1)) 次乘法，位数增长另计。字符串长度、所有中间分节数及两操作数长度之和须小于 INT_MAX 并留进位空间。不含大整数除大整数；不用浮点或 Boost。

```
struct BigInteger
{
    static constexpr int base = 1000000000;
    vector<int> d;
    int sign = 1;
    void trim()
    {
        while (!d.empty() && d.back() == 0)
            d.pop_back();
        if (d.empty())
            sign = 1;
    }
    BigInteger(long long x = 0)
    {
        sign = x < 0 ? -1 : 1;
        auto v = x < 0 ? 0ULL - (unsigned long long)x : (unsigned long long)x;
        while (v)
        {
            d.push_back(v % base);
            v /= base;
        }
        trim();
    }
```

十进制构造、输出与比较（接上页结构体）：

```
    explicit BigInteger(const string &s)
    {
        assert(!s.empty());
        int first = s[0] == '-' || s[0] == '+';
        sign = s[0] == '-' ? -1 : 1;
        assert(first < (int)s.size());
        for (int r = int(s.size()); r > first; r -= 9)
        {
            int v = 0;
            for (int i = max(first, r - 9); i < r; i++)
            {
                assert('0' <= s[i] && s[i] <= '9');
                v = v * 10 + s[i] - '0';
            }
            d.push_back(v);
        }
        trim();
    }
    string str() const
    {
        if (d.empty())
            return "0";
        string s = sign < 0 ? "-" : "";
        s += to_string(d.back());
        for (int i = int(d.size()) - 2; i >= 0; i--)
        {
            string t = to_string(d[i]);
            s += string(9 - t.size(), '0') + t;
        }
        return s;
    }
    int abs_cmp(const BigInteger &b) const
    {
        if (d.size() != b.d.size())
            return d.size() < b.d.size() ? -1 : 1;
        for (int i = int(d.size()) - 1; i >= 0; i--)
            if (d[i] != b.d[i])
                return d[i] < b.d[i] ? -1 : 1;
        return 0;
    }
    int cmp(const BigInteger &b) const
    {
        if (sign != b.sign)
            return sign < b.sign ? -1 : 1;
        return sign * abs_cmp(b);
    }
```

取负与加减法（接上页结构体）：

```
    BigInteger operator-() const
    {
        BigInteger c = *this;
        if (!c.d.empty())
            c.sign = -c.sign;
        return c;
    }
    BigInteger operator+(const BigInteger &b) const
    {
        if (sign != b.sign)
        {
            if (abs_cmp(b) < 0)
                return b + *this;
            BigInteger c = *this;
            int borrow = 0;
            for (int i = 0; i < (int)d.size(); i++)
            {
                int v = d[i] - borrow;
                if (i < (int)b.d.size())
                    v -= b.d[i];
                borrow = v < 0;
                c.d[i] = v + borrow * base;
            }
            c.trim();
            return c;
        }
        BigInteger c;
        c.sign = sign;
        int carry = 0;
        int n = max(d.size(), b.d.size());
        for (int i = 0; i < n || carry; i++)
        {
            int v = carry;
            if (i < (int)d.size())
                v += d[i];
            if (i < (int)b.d.size())
                v += b.d[i];
            c.d.push_back(v % base);
            carry = v / base;
        }
        return c;
    }
    BigInteger operator-(const BigInteger &b) const
    {
        return *this + (-b);
    }
```

乘法、小整数除法与整数幂（接上页结构体）：

```
    BigInteger operator*(const BigInteger &b) const
    {
        BigInteger c;
        c.sign = sign * b.sign;
        c.d.resize(d.size() + b.d.size());
        for (int i = 0; i < (int)d.size(); i++)
        {
            long long carry = 0;
            for (int j = 0; j < (int)b.d.size(); j++)
            {
                long long v = 1LL * d[i] * b.d[j] + c.d[i + j] + carry;
                c.d[i + j] = v % base;
                carry = v / base;
            }
            c.d[i + b.d.size()] = carry;
        }
        c.trim();
        return c;
    }
    pair<BigInteger, int> divmod(int b) const
    {
        assert(b != 0);
        long long v = b < 0 ? -1LL * b : b;
        long long rem = 0;
        BigInteger q;
        q.sign = sign * (b < 0 ? -1 : 1);
        q.d.resize(d.size());
        for (int i = int(d.size()) - 1; i >= 0; i--)
        {
            rem = rem * base + d[i];
            q.d[i] = rem / v;
            rem %= v;
        }
        q.trim();
        return {q, int(sign * rem)};
    }
    BigInteger pow(unsigned int e) const
    {
        BigInteger a = *this;
        BigInteger r = 1;
        while (e)
        {
            if (e & 1)
                r = r * a;
            e >>= 1;
            if (e)
                a = a * a;
        }
        return r;
    }
};
```

模板题使用：P1480 高精度除法 I

题意：0≤a≤10^5000，1≤b≤10^9；输出整数商。divmod 同时返回余数，这里只打印 first。完整协议本地通过，在线状态见验证记录。

所需模板：BigInteger（第 1 页，1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P1480

```
int main()
{
    string s;
    int b;
    cin >> s >> b;
    BigInteger a(s);
    cout << a.divmod(b).first.str() << '\n';
}
```

第二章　快速幂

2.1　无符号 64 位模乘与模幂

功能： 64 位无符号整数的模乘 `mul` 与快速幂 `power`，供判素、分解等数论模板使用。

核心思路： `mul` 把乘数提升到 `__uint128_t` 再取模，乘积不会溢出且结果精确；`power` 是普通二进制快速幂，从低位到高位扫描指数，底数逐次平方。

接口契约： Mod64::mul(a,b,m) 与 power(a,e,m)，a、b、e 支持完整 unsigned long long，m 为非零无符号 64 位模数。乘积用 uint128 后取模，不通过浮点近似；m=1 返回 0，e=0 返回 1 mod m。mul 为 O(1)，power 为 O(log(e+1)) 次模乘、O(1) 空间。它也是 64 位判素的底层组件。

```
struct Mod64
{
    using ull = unsigned long long;
    using u128 = __uint128_t;

    static ull mul(ull a, ull b, ull m) { return u128(a) * b % m; }

    static ull power(ull a, ull b, ull m)
    {
        assert(m);
        ull r = 1 % m;
        a %= m;
        for (; b; b >>= 1, a = mul(a, a, m))
            if (b & 1) r = mul(r, a, m);
        return r;
    }
};
```

第三章　数论

3.1　模算术简介

3.1.1　动态模整数与合数模逆元

功能：运行时设定模数的取模整数，模数不要求是素数。支持加、减、乘、快速幂；与模数互素时可求逆、做除法，不可逆时 `try_inv` 返回空。不同 `tag` 各自持有独立的模数。

核心思路：成员 `v` 始终是 $[0,p)$ 内的代表，加减只做一次条件校正。乘法用 Barrett 约减代替 64 位除法：`set_mod` 预存 $\texttt{im}\approx\lceil 2^{64}/p\rceil$，对乘积 z 用 $\lfloor z\cdot \texttt{im}/2^{64}\rfloor$ 估计商 q，q 至多偏大 1，因此 $z-qp$ 为负时补一个 p。求逆对 (v,p) 做迭代扩展欧几里得并维护 v 的系数，最后余数为 1 才可逆，系数即逆元。

接口契约：mint<tag> 按 tag 独立保存运行时模数，默认 998244353。先 set_mod(p)，1≤p≤INT_MAX，再构造该模数下的所有对象；更换同 tag 的模数后，旧对象、容器元素和缓存全部失效，必须重建，不能跨模混算。构造支持 long long 全范围，v 为规范余数；加减乘及复合赋值、相等比较为 O(1)，加减只作一次区间校正，乘法使用 Barrett32，不依赖素数。pow(k) 的 k 为 uint64_t 非负指数，0 次幂为 1 mod p。try_inv() 以扩展欧几里得求逆，gcd(v,p)≠1 返回 nullopt；inv() 与除法要求可逆，否则违反断言前提。p=1 时唯一余数为 0，乘法单位元也为 0，try_inv 返回 0。幂和求逆为 O(log p) 或 O(log k)，除法包含求逆。可与 Fenwick 区间和组合，不能用于模数和的 kth。mod、reducer 只读，必须通过 set_mod 同步切换；Barrett32 本身支持完整 uint32，但此包装仍受 int 上界保护；不支持超过 INT_MAX 的模数，也不提供线程间安全切换模数。

```
template <int tag = 0> struct mint
{
    static inline int mod = 998244353;
    static inline unsigned long long im = -1ULL / 998244353 + 1; // Barrett reciprocal
    int v;

    static void set_mod(int p)
    {
        assert(p >= 1);
        mod = p;
        im = -1ULL / p + 1;
    }

    mint(long long x = 0) : v(x % mod)
    {
        if (v < 0)
            v += mod;
    }

    mint operator+(mint b) const
    {
        mint r;
        long long sum = (long long)v + b.v;
        r.v = sum >= mod ? sum - mod : sum;
        return r;
    }
```

模整数（接上页同一结构体）：

```
    mint operator-(mint b) const
    {
        mint r;
        r.v = v - b.v;
        if (r.v < 0)
            r.v += mod;
        return r;
    }

    mint operator*(mint b) const
    {
        mint r;
        unsigned long long z = 1ULL * v * b.v, y = (unsigned long long)((__uint128_t(z) * im) >> 64) * mod;
        r.v = int(z - y + (z < y ? mod : 0));
        return r;
    }

    mint &operator+=(mint b)
    {
        return *this = *this + b;
    }

    mint &operator-=(mint b)
    {
        return *this = *this - b;
    }

    mint &operator*=(mint b)
    {
        return *this = *this * b;
    }

    mint pow(uint64_t k) const
    {
        mint a = *this, r = 1;
        while (k)
        {
            if (k & 1)
                r *= a;
            a *= a;
            k >>= 1;
        }
        return r;
    }
```

模整数（接上页同一结构体）：

```
    optional<mint> try_inv() const
    {
        long long a = v, b = mod, x = 1, y = 0;
        while (b)
        {
            long long q = a / b;
            tie(a, b) = pair{b, a - q * b};
            tie(x, y) = pair{y, x - q * y};
        }
        if (a != 1)
            return nullopt;
        return mint(x);
    }

    mint inv() const
    {
        auto x = try_inv();
        assert(x.has_value());
        return *x;
    }

    mint operator/(mint b) const
    {
        return *this * b.inv();
    }

    mint &operator/=(mint b)
    {
        return *this = *this / b;
    }

    bool operator==(const mint &) const = default;
};
```

模板题使用：Luogu P5431 【模板】模意义下的乘法逆元 2

题意：输入 1≤n≤5000000、2≤k<p≤10^9、1≤a_i<p 且 p 为素数，求 ∑k^i/a_i 模 p。先设置 mint 的模数 p，再构造数组；求前缀积后只对总积求一次逆（try_inv），倒序乘出各项逆元。权重从 1 开始，每项先乘 k 再累加。read 与 main 一起抄。更换同 tag 模数后旧对象失效，计算期间不能切换。核心支持合数单位，本题只覆盖素数；动态模数、构造、乘法及累加、v 输出在此直接使用，合数求逆接口另由核心测试核验。

所需模板：mint（第 8 页，3.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5431

```
int read()
{
    int c = getchar_unlocked();
    while (c < '0' || c > '9')
    {
        if (c == EOF) return 0;
        c = getchar_unlocked();
    }
    int value = 0;
    while ('0' <= c && c <= '9')
    {
        value = value * 10 + c - '0';
        c = getchar_unlocked();
    }
    return value;
}

int main()
{
    int n = read(), p = read(), k = read();
    mint<>::set_mod(p);
    vector<mint<>> a(n);
    for (auto &x : a) x = read();
    // Batch inverse: one inversion of the prefix product.
    vector<mint<>> prefix(n + 1, 1);
    for (int i = 0; i < n; i++) prefix[i + 1] = prefix[i] * a[i];
    auto total = prefix[n].try_inv();
    if (!total) return 1;
    mint<> suffix = *total, answer = 0;
    vector<mint<>> inverse(n);
    for (int i = n - 1; i >= 0; i--)
    {
        inverse[i] = suffix * prefix[i];
        suffix *= a[i];
    }
    mint<> weight = 1;
    for (auto x : inverse)
    {
        weight *= mint<>(k);
        answer += weight * x;
    }
    printf("%d\n", answer.v);
    return 0;
}
```

3.1.2 静态模整数

功能：编译期固定模数的取模整数（模数可为合数），支持加、减、乘、快速幂；与模数互素时可求逆、做除法，不可逆时 `try_inv` 返回空。

核心思路：成员 `v` 始终保存 $[0, \mathrm{mod})$ 内的规范代表，构造时把负余数调正。加减各做一次条件校正，乘法用 `long long` 乘积取模，快速幂按二进制拆分指数。求逆不依赖费马小定理：对 (v, mod) 做迭代扩展欧几里得并维护 v 的系数 `x`，结束时 gcd 为 1 则该系数就是逆元。

接口契约：固定 int 模数 1≤mod≤INT_MAX，可为合数。构造接受 signed64 并正规化，v 保持 0..mod-1；不要手动破坏正规化。加减乘 O(1)，pow 非负 signed64 指数、时间 O(log(e+1))。try_inv 用欧几里得求逆：gcd(v,mod)=1 时返回 optional 值，否则 nullopt；mod=1 返回代表 0。inv、/、/= 要求分母可逆；断言开启时诊断违反前提，关闭断言不承诺拒绝非法除法。逆元 O(log(mod+1))、额外空间 O(1)，只用 long long，不新增辅助模板依赖。+=、-=、*=、/= 返回自身引用，支持自赋值与链式运算，可作为 Fenwick 区间和值类型。类型的扩展不放宽 Binomial、NTT、高斯或 FPS 自身的素数等前提。

```
template <int mod> struct ModInt
{
    static_assert(mod >= 1);
    int v;

    ModInt(long long x = 0)
    {
        v = x % mod;
        if (v < 0)
            v += mod;
    }

    ModInt operator+(ModInt b) const
    {
        long long x = (long long)v + b.v;
        if (x >= mod)
            x -= mod;
        ModInt r;
        r.v = x;
        return r;
    }

    ModInt operator-(ModInt b) const
    {
        int x = v - b.v;
        if (x < 0)
            x += mod;
        ModInt r;
        r.v = x;
        return r;
    }

    ModInt operator*(ModInt b) const
    {
        ModInt r;
        r.v = 1LL * v * b.v % mod;
        return r;
    }
```

模整数（接上页同一结构体）：

```
    ModInt pow(long long e) const
    {
        assert(e >= 0);
        ModInt a = *this, r = 1;
        while (e)
        {
            if (e & 1)
                r = r * a;
            a = a * a;
            e >>= 1;
        }
        return r;
    }

    optional<ModInt> try_inv() const
    {
        long long a = v, b = mod, x = 1, y = 0;
        while (b)
        {
            long long q = a / b;
            a -= q * b;
            swap(a, b);
            x -= q * y;
            swap(x, y);
        }
        if (a != 1)
            return nullopt;
        return ModInt(x);
    }

    ModInt inv() const
    {
        auto r = try_inv();
        assert(r);
        return *r;
    } // requires an invertible residue
```

模整数（接上页同一结构体）：

```
    ModInt operator/(ModInt b) const
    {
        return *this * b.inv();
    }

    ModInt &operator+=(ModInt b)
    {
        return *this = *this + b;
    }

    ModInt &operator-=(ModInt b)
    {
        return *this = *this - b;
    }

    ModInt &operator*=(ModInt b)
    {
        return *this = *this * b;
    }

    ModInt &operator/=(ModInt b)
    {
        return *this = *this / b;
    }
};
```

3.1.3 模运算的适用条件

模逆元存在当且仅当 $\gcd(a,m)=1$；费马逆元 a^{p-2} 仅适用于素数模数且 $a\not\equiv 0$。$x\equiv a\pmod m$、$x\equiv b\pmod n$ 可合并当且仅当 $\gcd(m,n)\mid(b-a)$，新模数 $\operatorname{lcm}(m,n)$。Lucas 仅限素数模；素数模阶乘表上限须小于模数。

3.1.3.1 除法与解同余式不是一回事

$g=\gcd(a,m)$：$ax+b\equiv 0\pmod m$ 在 $g\nmid b$ 时无解，否则

$$\frac{a}{g}x+\frac{b}{g}\equiv 0\pmod{m/g}$$

有唯一解 x_0，原模下全部解为 $x_0+t(m/g)$，$0\le t<g$。约去 g 时模数也要除以 g。`linear_congruence(a,b,m)` 解 $ax+b\equiv 0$，返回（最小非负解，最小正周期），无解 $(-1,-1)$；$ax\equiv c$ 须先把 c 规范到 $[0,m)$ 再取负传入。见第 30 页。`mod_inverse` 不可逆返回 -1，`ModInt::try_inv` 返回 `nullopt`；`inv` 与除法以可逆为前提。见第 26 页、第 12 页。$m=1$ 时代表与逆元均为 0、周期 1；域上算法（高斯、NTT、FPS、组合数）不推广到模 1。

3.1.3.2 知道整数可整除时，先升模数再整除

已知 $d\mid X$，求 $(X/d)\bmod m$（$\gcd(d,m)$ 任意）：取规范剩余 $r=X\bmod(md)\in[0,md)$，则

$$\frac{X}{d}\equiv\frac{r}{d}\pmod m.$$

须能计算 md 及其下的乘模（必要时多精度）；新模数可能是合数，不能套素数模模板；生成 X 的过程含除法时需重验。应用见第 5.11.7 节（第 136 页）。

3.1.3.3 组合数的整数性不等于分母可逆

`Binomial` 要求素数 p 且 $N<p$；合数模下 $N<m$ 不够。大下标素数模用 Lucas；合数模用素数幂分解、单位部分与 CRT（exLucas）。见第 111 页。插值节点差、级数积分分母、模概率分母都须各自检查可逆。

3.1.3.4 大指数要同时保留余数和阈值信息

$a\ge 0$，$b\ge 0$，$m\ge 1$，$\phi=\varphi(m)$：

$$a^b\equiv a^e\pmod m,\qquad e=\begin{cases}b, & b<\phi,\\ b\bmod\phi+\phi, & b\ge\phi.\end{cases}$$

零次幂为 1，$m=1$ 返回 0；$\gcd(a,m)=1$ 时可直接对 ϕ 取模。十进制大指数逐位维护 $b\bmod\phi$ 与 $\min(b,\phi)$。`euler_power(a,b,m,phi)` 要求传入真实 $\varphi(m)$，基数与模数为 unsigned64（负数先规范化），降幂指数可达 65 位，内部用 128 位。见第 25 页。

3.2 素数

3.2.1 确定性 64 位 Miller–Rabin 判素

问题描述：判断一个 64 位无符号整数 n 是否为素数，结果是确定的而不是概率性的。

核心思路：先用 $2 \sim 37$ 的小素数试除，整除时只有 n 恰为该素数才是素数。否则写 $n-1 = d \cdot 2^s$（d 为奇数），对 7 个固定底数做 Miller–Rabin：$x = a^d$ 为 1 或 $n-1$ 则该底数通过；否则再平方至多 $s-1$ 次，途中出现 $n-1$ 才通过，否则 n 是合数。这组底数在 64 位范围内没有反例；底数是 n 的倍数时跳过。

代码依赖：Mod64（第 7 页，2.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：Prime64::prime(n) 接收完整 unsigned long long，n<2 返回 false。先试除小素数，再使用 2、325、9375、28178、450775、9780504、1795265022 七个底数，结果在 64 位输入范围内确定。继承 Mod64，模乘用 uint128；O(log n) 次模乘，O(1) 空间，不包含因数分解。

```
struct Prime64 : Mod64
{
    static bool prime(ull n)
    {
        if (n < 2) return false;
        for (ull p : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37})
            if (n % p == 0) return n == p;
        ull d = n - 1;
        int s = 0;
        while (!(d & 1))
        {
            d >>= 1;
            ++s;
        }
        for (ull a :
             {2ULL, 325ULL, 9375ULL, 28178ULL, 450775ULL, 9780504ULL, 1795265022ULL})
        {
            if (a % n == 0) continue;
            ull x = power(a, d, n);
            if (x == 1 || x == n - 1) continue;
            bool ok = false;
            for (int j = 1; j < s; j++)
            {
                x = mul(x, x, n);
                if (x == n - 1)
                {
                    ok = true;
                    break;
                }
            }
            if (!ok) return false;
        }
        return true;
    }
};
```

模板题使用：Library Checker primality_test

题意：逐个判断给定正整数是否为素数，输出 Yes 或 No。此题输入不超过 10^{18}；模板支持的完整范围另见接口约定。

所需模板：Mod64（第 7 页，2.1 节）、Prime64（第 16 页，3.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/primality_test

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int q;
    cin >> q;
    while (q--)
    {
        unsigned long long n;
        cin >> n;
        cout << (Prime64::prime(n) ? "Yes" : "No") << '\n';
    }
}
```

3.2.2 64 位判素与质因数分解

Miller–Rabin 用七个固定底数，覆盖无符号 64 位整数：

$$2,\ 325,\ 9375,\ 28178,\ 450775,\ 9780504,\ 1795265022.$$

$n-1=d2^s$，检查 a^d 及平方链；$n<2$ 非素数；底数模 n 为 0 时跳过。模乘用无符号 128 位。

Pollard–Rho 迭代 $f(x)=x^2+c \pmod n$，取 $\gcd(|x-y|,n)$；只接受 $1<d<n$，否则换参数重试，无确定最坏上界。`factor` 返回排序且保留重复的质因子，`factor(1)` 为空，取首尾前先判空。

3.3 欧拉函数

3.3.1 单个数的欧拉函数

问题描述：求单个正整数 n 的欧拉函数 $\varphi(n)$。提供两个入口：试除法，以及借助 PollardRho 分解后计算（适合含大质因子的 n）。

核心思路：公式 $\varphi(n)=n\prod_{p\mid n}(1-1/p)$。试除版从 $p=2$ 枚举到 $\sqrt{n}$，遇到质因子就把答案改为 answer/p·($p-1$) 并把 n 中的 p 除尽，最后剩下的 $n>1$ 是唯一的大质因子。分解版对升序质因子列表去重后做同样的乘法。先除后乘，中间值不超过原数。

代码依赖：PollardRho（第 21 页，3.5.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：euler_phi(n) 使用试除，O($\sqrt{n}$) 时间、O(1) 额外空间；euler_phi(n,solver) 复用 PollardRho，时间由随机分解主导，分解结果与递归占 O(log n) 空间。1≤n≤ULLONG_MAX，phi(1)=1。大素数或平衡大半素数应选分解器入口，不要用试除穷扫平方根；批量求 1..N 用 LinearSieve。先除质因子再乘 p-1，结果及中间乘积均不超过原 n。只抄试除函数时不需要 Prime64/PollardRho。

```
unsigned long long euler_phi(unsigned long long n)
{
    assert(n >= 1);
    unsigned long long answer = n;
    for (unsigned long long p = 2; p <= n / p; p++)
        if (n % p == 0)
        {
            answer = answer / p * (p - 1);
            while (n % p == 0) n /= p;
        }
    if (n > 1) answer = answer / n * (n - 1);
    return answer;
}

unsigned long long euler_phi(unsigned long long n, PollardRho &solver)
{
    assert(n >= 1);
    auto factors = solver.factor(n);
    unsigned long long answer = n;
    unsigned long long previous = 0;
    for (auto p : factors)
        if (p != previous)
        {
            answer = answer / p * (p - 1);
            previous = p;
        }
    return answer;
}
```

联合使用示例见第 26 页（Luogu P5091），其中直接调用本模板。

3.3.2 单个数的欧拉函数

实现见第 18 页。对 $n\ge 1$，

$$\varphi(n)=n\prod_{p\mid n}\left(1-\frac{1}{p}\right),$$

按 ans ← (ans/p)($p-1$) 先除后乘，不会溢出。试除入口用 $p\le n/p$，$O(\sqrt{n})$，不适合 64 位大素数；传入 PollardRho 对象时先分解再去重更新。$\varphi(1)=1$，0 不在定义域内。

3.4 筛法

3.4.1 区间筛素数

问题描述：给定区间 $[l, r]$，列出其中全部素数；r 可以很大，只要区间长度可承受。

核心思路：区间内的合数必有不超过 $\sqrt{r}$ 的质因子。先用埃氏筛（`composite`）求出 $[2, \lfloor\sqrt{r}\rfloor]$ 的素数，同时对每个素数 p，从 $\max(p^2, \lceil l/p \rceil p)$ 开始，在区间数组 `removed` 上按步长 p 标记倍数（下标偏移 $j - l$）。最后从 $\max(l, 2)$ 起收集未被标记的数。

接口契约：segmented_primes(l,r) 返回闭区间内递增的全部素数，0≤l≤r≤INT_MAX；区间长度须能分配内存。独立函数，每次重建底筛；不依赖预存素数表。令 W=r-l+1，时间 O((√r+W)log log(r+3))，空间 O(√r+W)。倍数起点、递增变量均用 long long，避免 INT_MAX 端点溢出；0 和 1 不进入结果。

```
vector<int> segmented_primes(int l, int r)
{
    assert(0 <= l && l <= r);
    int limit = 0;
    while (1LL * (limit + 1) * (limit + 1) <= r) limit++;
    vector<char> composite(limit + 1);
    vector<char> removed(1LL * r - l + 1);
    for (int p = 2; p <= limit; p++)
        if (!composite[p])
        {
            for (long long j = 1LL * p * p; j <= limit; j += p) composite[j] = true;
            long long start = max(1LL * p * p, (l + 1LL * p - 1) / p * p);
            for (long long j = start; j <= r; j += p) removed[j - l] = true;
        }
    vector<int> primes;
    for (long long x = max(l, 2); x <= r; x++)
        if (!removed[x - l]) primes.push_back(x);
    return primes;
}
```

3.4.2 线性筛、欧拉函数与 Möbius 函数

功能：一次预处理 $0..n$，得到递增素数表 `prime`、最小质因子 `lp`、欧拉函数 `phi` 和 Möbius 函数 `mu`；反复除以 `lp` 可快速分解表内的数。

核心思路：欧拉筛：每个合数 $j = i \cdot p$ 只被它的最小质因子 p 筛一次。外层枚举 i，内层从小到大枚举已有素数 p 并令 `lp[j]`$= p$；一旦 $p \mid i$，更大的素数就不再是 $i \cdot p$ 的最小质因子，立即 `break`。积性函数同步转移：$p \mid i$ 时 $\varphi(ip) = \varphi(i)\,p$、$\mu(ip) = 0$；否则 $\varphi(ip) = \varphi(i)(p-1)$、$\mu(ip) = -\mu(i)$。

接口契约：构造参数 0≤n<INT_MAX，且数组须能分配内存。prime 为递增质数表；lp/phi/mu 为 0..n 数组。x≥2 时 lp[x] 是最小质因子，lp[x]==x 等价于 x 为素数；反复除以 lp[x] 可分解质因数。lp[0]=lp[1]=0，不能直接用 lp[x]==x 判断 0。n≥1 时 phi[1]=mu[1]=1；第 0 项均为占位 0。O(n) 时间和空间，分解单个表内正整数耗时 O(log x)。n=0 不可访问下标 1。

```
struct LinearSieve
{
    vector<int> prime, lp, phi, mu;

    LinearSieve(int n) : lp(n + 1), phi(n + 1), mu(n + 1)
    {
        if (n) phi[1] = mu[1] = 1;
        for (int i = 2; i <= n; i++)
        {
            if (!lp[i])
            {
                lp[i] = i;
                prime.push_back(i);
                phi[i] = i - 1;
                mu[i] = -1;
            }
            for (int p : prime)
            {
                if (p > n / i) break;
                int j = i * p;
                lp[j] = p;
                if (i % p == 0)
                {
                    phi[j] = phi[i] * p;
                    mu[j] = 0;
                    break;
                }
                phi[j] = phi[i] * (p - 1);
                mu[j] = -mu[i];
            }
        }
    }
};
```

3.4.3 线性筛：三张表的约定

`LinearSieve` 构造最小质因子、φ、μ，每个合数只由 lp 生成一次，$O(n)$。若 $p \mid i$，$\varphi(ip) = p\varphi(i)$、$\mu(ip) = 0$；否则为 $(p-1)\varphi(i)$ 与 $-\mu(i)$。判素用 $x \geq 2$ 且 $\mathrm{lp}(x) = x$；反复除 $\mathrm{lp}(x)$ 分解，0 不可分解。要求 $0 \leq n <$ `INT_MAX`，乘积前检查 $p \leq n/i$。相关恒等式：$\sum_{d|x} \varphi(d) = x$，$\sum_{d|x} \mu(d) = [x = 1]$，$\varphi(n) = \sum_{d|n} \mu(d)(n/d)$。

3.4.4 区间筛：只保存目标区间

实现见第 19 页。设 $0 \le L \le R \le$ INT_MAX，筛出不超过 $\lfloor\sqrt{R}\rfloor$ 的素数，对每个 p 从

$$\max\bigl(p^2, \lceil L/p\rceil p\bigr)$$

开始标记区间倍数，输出跳过 0 和 1。令 $W = R - L + 1$，空间 $O(\sqrt{R} + W)$，时间 $O((\sqrt{R} + W)\log\log(R + 3))$；每次调用重建底筛。$p^2$、取整加法与遍历变量用 long long，否则 $R =$ INT_MAX 时溢出。

3.5 分解质因数

3.5.1 Pollard–Rho 因数分解

问题描述：把 64 位正整数 n 分解质因数，返回升序且含重数的质因子列表。

核心思路：split 递归：n 为素数（Prime64）就记为一个因子，否则用 rho 找一个非平凡因子 d，分别递归 d 与 n/d，最后排序。rho 对偶数直接返回 2；否则随机取 c 和起点，迭代 $f(v) = v^2 + c \bmod n$，用 Floyd 判圈（x 走一步、y 走两步），每步计算 $\gcd(|x - y|, n)$。得到 $1 < d < n$ 即返回；得到 n 或轮数用尽，就换一组随机参数重试。

代码依赖：Mod64（第 7 页，2.1 节）、Prime64（第 16 页，3.2.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：factor(n) 返回升序质因子（含重数），1≤n≤ULLONG_MAX。n=1 返回空数组；n≥2 时 size()==1 等价于 n 为素数，否则 front()/back() 分别取最小/最大质因子。依赖 Mod64 与 Prime64。公开使用 factor；rho/split 为内部辅助，rho 只接受大于 1 的合数，不能直接传素数。Las Vegas 随机运行时间，固定默认种子方便复现，可由构造参数更换；无确定性最坏时间保证。每次 rho 尝试最多 200000 轮，失败后换随机参数重试，不将失败当作素数。递归仅沿真因子拆分，深度 O(log n)。

```
struct PollardRho
{
    using ull = unsigned long long;
    using u128 = __uint128_t;
    mt19937_64 rng;

    PollardRho(ull seed = 712367821) : rng(seed) {}

    ull rho(ull n)
    {
        if (n % 2 == 0) return 2;
        for (;;)
        {
            ull c = rng() % (n - 1) + 1, x = rng() % n, y = x, d = 1;
            auto f = [&](ull v)
            {
                return (Mod64::mul(v, v, n) + u128(c)) % n;
            };
            // Retry bounded attempts; Las Vegas: only exact divisors returned.
            for (int i = 0; i < 200000 && d == 1; i++)
            {
                x = f(x);
                y = f(f(y));
                d = gcd(x > y ? x - y : y - x, n);
            }
            if (1 < d && d < n) return d;
        }
    }
```

```
29
30      void split(ull n, vector<ull> &a)
31      {
32          if (n == 1) return;
33          if (Prime64::prime(n))
34          {
35              a.push_back(n);
36              return;
37          }
38          ull d = rho(n);
39          split(d, a);
40          split(n / d, a);
41      }
42
43      vector<ull> factor(ull n)
44      {
45          assert(n >= 1);
46          vector<ull> a;
47          split(n, a);
48          sort(a.begin(), a.end());
49          return a;
50      }
51  };
```

模板题使用：Luogu P4718

题意：逐个判断 n 是否为素数，是则输出 Prime，否则输出最大质因子。题目保证 n≥2，factor 返回升序含重数的质因子，因此可取 back；若用于 n=1，须先处理空结果。

所需模板：Mod64（第 7 页，2.1 节）、Prime64（第 16 页，3.2.1 节）、PollardRho（第 21 页，3.5.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4718

```
1   int main()
2   {
3       int t;
4       scanf("%d", &t);
5       PollardRho solver;
6       while (t--)
7       {
8           unsigned long long n;
9           scanf("%llu", &n);
10          auto factors = solver.factor(n);
11          if (factors.size() == 1)
12              puts("Prime");
13          else
14              printf("%llu\n", factors.back());
15      }
16      return 0;
17  }
```

3.6 裴蜀定理 & 一次不定方程

3.6.1 扩展欧几里得与 Bézout 系数

问题描述：给定非负整数 a, b，求 $g = \gcd(a, b)$ 以及一组整数 x, y 使 $ax + by = g$（Bézout 系数），据此可解一般的一次不定方程。

核心思路：按欧几里得算法递归：$b = 0$ 时取 $(x, y) = (1, 0)$。否则对 $(b, a \bmod b)$ 递归，并把 `y`、`x` 交换位置传入，得到 $bx' + (a \bmod b)y' = g$，此时 `x` 中已是 y'、`y` 中是 x'。由 $a \bmod b = a - \lfloor a/b \rfloor b$ 得 $x = y'$、$y = x' - \lfloor a/b \rfloor y'$，只需再执行 `y -= a / b * x`。

接口契约：extended_gcd(a,b,x,y) 约定 0≤a,b≤2^63，参数和输出系数为 int128，返回非负 gcd 并满足 ax+by=gcd(a,b)。a=b=0 返回 0、x=1、y=0；递归实现，中间乘加为 int128；O(log(min(a,b)+1)) 时间与递归空间。解 ax+by=c：令 g=gcd(a,b)，c 不是 g 的倍数时无解；否则 x0=x · c/g、y0=y · c/g，通解 x=x0+k · b/g、y=y0-k · a/g；最小非负 x 为 x0 对 b/g 取非负余数。乘 c/g 时注意 int128 范围。

```
__int128_t extended_gcd(__int128_t a, __int128_t b, __int128_t &x, __int128_t &y)
{
    using i128 = __int128_t;
    if (!b)
    {
        x = 1;
        y = 0;
        return a;
    }
    i128 d = extended_gcd(b, a % b, y, x);
    y -= a / b * x;
    return d;
}
```

模板题使用：AOJ NTL_1_E Extended Euclid Algorithm

题意：给出正整数 1≤a,b≤10^9，求 ax+by=gcd(a,b)，先最小化 |x|+|y|，仅在并列最小时取 x≤y。直接调用 extended_gcd；该正整数范围内递归所得解已满足最小性，a=b 时返回 (0,1)。系数用 ___int128_t 接收，输出转 long long 安全；不应对所有答案强行要求 x≤y。时间、递归空间均 O(log(min(a,b)+1))。

所需模板：extended_gcd（第 23 页，3.6.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.u-aizu.ac.jp/onlinejudge/description.jsp?id=NTL_1_E

```
int main()
{
    long long a, b;
    scanf("%lld%lld", &a, &b);
    __int128_t x, y;
    extended_gcd(a, b, x, y);
    printf("%lld %lld\n", (long long)x, (long long)y);
    return 0;
}
```

3.6.2 一次不定方程：特解与正整数解范围

底层接口见第 3.6.1 节（第 23 页）。a,b 不全为零时令 $g=\gcd(|a|,|b|)>0$；当且仅当 $g\mid c$ 有整数解。由 $|a|u+|b|v=g$ 乘 c/g 并恢复符号得特解 $ax_0+by_0=c$，全部解为

$$x=x_0+\frac{b}{g}t,\qquad y=y_0-\frac{a}{g}t,\qquad t\in\mathbb{Z}.$$

$a=b=0$ 时 $c=0$ 全体整数对为解，否则无解。`exgcd` 约定非负系数、上限 2^{63}，返回非负 g；取负、缩放后的特解都用 128 位保存。

正系数 a,b,c 求正整数解：$d_x=b/g, d_y=a/g$，把 x_0 调到 $x_{\min}\in[1,d_x]$，$y_{\max}=(c-ax_{\min})/b$。若 $y_{\max}>0$，

$$N=\left\lfloor\frac{y_{\max}-1}{d_y}\right\rfloor+1,\quad x_{\max}=x_{\min}+(N-1)d_x,\quad y_{\min}=y_{\max}-(N-1)d_y.$$

否则无正解，x,y 的最小正值分别按周期归一化（通常不是同一组解）；正值归一化用 `(v % d + d - 1) % d + 1`。

3.6.2.1 有符号矩形范围：把两维约束化成一个参数区间

求 $L_x\le x\le R_x$、$L_y\le y\le R_y$ 内的解数（a,b,c 可负可零）。空区间答案为 0；$a=b=0$ 时 $c\neq0$ 无解，$c=0$ 答案 $(R_x-L_x+1)(R_y-L_y+1)$。否则取特解与带符号步长 $d_x=b/g$、$d_y=-a/g$，对 $z=z_0+dt$：

$d>0$	$\lceil(L-z_0)/d\rceil\le t\le\lfloor(R-z_0)/d\rfloor$
$d<0,\ k=-d>0$	$\lceil(z_0-R)/k\rceil\le t\le\lfloor(z_0-L)/k\rfloor$
$d=0$	$L\le z_0\le R$ 时不限制t，否则无解

两区间取交 $[T_l,T_r]$，解数 $\max(0,T_r-T_l+1)$。floor/ceil 为数学取整：正分母 D、C++ 商余 $q=n/D, r=n\%D$ 时

$$\lfloor n/D\rfloor=q-[r<0],\qquad\lceil n/D\rceil=q+[r>0].$$

差值、区间长度可能超出 64 位（全零系数覆盖全部 64 位坐标时答案为 2^{128}）；不能证明范围时用任意精度整数。求特解后只需常数次运算。

3.7 费马小定理 & 欧拉定理

3.7.1 十进制大指数：扩展欧拉降幂

问题描述：给定底数 a、模数 m 及 $\varphi(m)$，以及以十进制字符串给出的超大指数 b，求 $a^b \bmod m$；a 与 m 不必互素。

核心思路：扩展欧拉定理：$b < \varphi(m)$ 时指数保持 b；$b \geq \varphi(m)$ 时 $a^b \equiv a^{(b \bmod \varphi(m))+\varphi(m)} \pmod m$。逐位扫描字符串，同时维护 `rem`$= b \bmod \varphi$ 和饱和值 `cap`$= \min(b, \varphi)$；扫描结束时 `cap` 等于 φ 就说明 $b \geq \varphi$，指数加上 φ。降幂后的指数小于 2φ，再用 `Mod64::mul` 做快速幂。

代码依赖：Mod64（第 7 页，2.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：euler_power(a,b,m,phi) 求 a^b mod m，a、m 为 unsigned long long，m>0；b 为非空十进制数字串，可为 0、可含前导零，约定 0^0=1。调用者必须传入正确的 phi=φ(m)，包括 $\varphi(1)=1$；函数只检查数值范围，不验证 phi 正确性。指数小于 phi 时保持原值，否则使用 b mod phi + phi，非互质底数也适用。相加和十进制更新都用 int128，支持完整 unsigned64 模数。O(|b|+log m) 时间、O(1) 额外空间，不含求 phi 的成本；依赖 Mod64::mul。

```
unsigned long long euler_power(unsigned long long a,
                               string_view b,
                               unsigned long long m,
                               unsigned long long phi)
{
    using U = unsigned long long;
    using I = __uint128_t;
    assert(m && phi && phi <= m && !b.empty());
    U rem = 0, cap = 0;
    for (char c : b)
    {
        assert('0' <= c && c <= '9');
        int d = c - '0';
        rem = (I(rem) * 10 + d) % phi;
        cap = min(I(phi), I(cap) * 10 + d);
    }
    I e = I(rem) + (cap == phi ? I(phi) : 0);
    U ans = 1 % m;
    a %= m;
    for (; e; e >>= 1, a = Mod64::mul(a, a, m))
        if (e & 1) ans = Mod64::mul(ans, a, m);
    return ans;
}
```

模板题使用：Luogu P5091

题意：输入顺序为底数 a、模数 m、指数 b；b 最多 20000001 位，以字符串读入。先用试除法计算 $\varphi(m)$，再做扩展欧拉降幂；题目 m≤10^8，不能把输入顺序写成 a,b,m。

所需模板：Mod64（第 7 页，2.1 节）、Prime64（第 16 页，3.2.1 节）、PollardRho（第 21 页，3.5.1 节）、euler_phi（第 18 页，3.3.1 节）、euler_power（第 25 页，3.7.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5091

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    unsigned long long a, m;
    string b;
    cin >> a >> m >> b;
    auto phi = euler_phi(m);
    cout << euler_power(a, b, m, phi) << '\n';
}
```

3.7.2 Carmichael：最小公共指数

$\lambda(n)$ 为所有可逆元阶的最小公倍数，$\lambda(1)=1$，$\lambda(n)\mid\varphi(n)$。

$$\lambda(n)=\operatorname{lcm}_i \lambda(p_i^{e_i}),\qquad \lambda(p^e)=\begin{cases}2^{e-2}, & p=2,\ e\ge 3,\\ p^{e-1}(p-1), & \text{其他情形.}\end{cases}$$

仅可逆底数可把指数对 $\lambda(n)$ 取模。合数 n 是 Carmichael 数当且仅当 $\lambda(n)\mid(n-1)$，等价于无平方因子且每个素因子 p 满足 $(p-1)\mid(n-1)$（须排除素数）；Fermat 检验不能代替素性判定。

3.8 模逆元

3.8.1 任意正模数的单个逆元

问题描述：给定整数 a 和正整数 m（可为合数），求 a 模 m 的乘法逆元；$\gcd(a,m)\neq 1$ 时逆元不存在。

核心思路：逆元 x 满足 $ax+my=1$。先把 a 规范到 $[0,m)$，调用 `extended_gcd` 求 Bézout 系数；gcd 不为 1 则不可逆，否则把系数 x 调整到 $[0,m)$ 返回。不使用费马小定理，因此不要求 m 为素数。

代码依赖：extended_gcd（第 23 页，3.6.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：mod_inverse(a,m) 支持完整 long long a 和正 long long m，将 a 规范化后用 extended_gcd 求逆。不可逆返回-1，可逆返回 0..m-1 内的代表；模数 1 返回 0。允许合数模数，不使用费马小定理。O(log m) 时间与递归空间。

```
long long mod_inverse(long long a, long long m)
{
    using i128 = __int128_t;
    assert(m > 0);
    a %= m;
    if (a < 0) a += m;
    i128 x, y;
    if (extended_gcd(a, m, x, y) != 1) return -1;
    return (x % m + m) % m;
}
```

3.8.2 任意数组的批量逆元

问题描述：给定 n 个整数和模数（可为合数），一次求出它们各自的模逆元；只要有一个不可逆就整体判定失败。

核心思路：前缀积技巧：`prefix[i]` 为前 i 个数之积。只对总积 `prefix[n]` 调用一次 `mod_inverse`（总积不可逆当且仅当某一项不可逆）。再从后往前扫：当前 `inverse` 是前 $i+1$ 项积的逆，乘 `prefix[i]` 得到 a_i^{-1}，乘 a_i 得到前 i 项积的逆。结果原地写回数组。

代码依赖：mod_inverse（第 26 页，3.8.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：batch_inverse(a,mod) 返回与 a 同序的逆元向量。要求 mod 为正 long long，每个输入可为完整 long long；只要某一项与 mod 不互素，整个结果返回 nullopt，不输出部分逆元。空输入返回有效的空向量；模数 1 约定返回全零，与单个逆元接口一致。模数可以是合数。函数按值接收数组并归一化；不再需要原数组时传 move(a) 可避免输入副本。依赖 mod_inverse 和 extended_gcd。仅求一次逆元，时间 O(n+log mod)，空间 O(n)；128 位中间乘法防止溢出。

```
optional<vector<long long>> batch_inverse(vector<long long> a, long long mod)
{
    assert(mod > 0);
    int n = a.size();
    vector<long long> prefix(n + 1, 1 % mod);
    for (int i = 0; i < n; i++)
    {
        a[i] %= mod;
        if (a[i] < 0) a[i] += mod;
        prefix[i + 1] = __int128_t(prefix[i]) * a[i] % mod;
    }
    long long inverse = mod_inverse(prefix[n], mod);
    if (inverse == -1) return nullopt;
    for (int i = n - 1; i >= 0; i--)
    {
        long long value = a[i];
        a[i] = __int128_t(inverse) * prefix[i] % mod;
        inverse = __int128_t(inverse) * value % mod;
    }
    return a;
}
```

模板题使用：Luogu P5431 【模板】模意义下的乘法逆元 2

题意：给定 1≤n≤5000000、2≤k<p≤10^9、1≤a_i<p，p 为素数，输出 ∑(i=1..n)k^i/a_i 模 p。先批量求全部逆元再累乘权重，注意首项权重是 k。完整抄写 read 与 main；read 仅适用于本题的非负十进制输入。move(a) 避免输入副本，此后不再读取 a。本题所有元素可逆；一般模数或零元素须检查 nullopt。乘积在 long long 范围内，批量核心内部用 int128 保护一般模数。

所需模板：extended_gcd（第 23 页，3.6.1 节）、mod_inverse（第 26 页，3.8.1 节）、batch_inverse（第 27 页，3.8.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5431

```
int read()
{
    int c = getchar_unlocked();
    while (c < '0' || c > '9')
    {
        if (c == EOF) return 0;
        c = getchar_unlocked();
    }
    int value = 0;
    while ('0' <= c && c <= '9')
    {
        value = value * 10 + c - '0';
        c = getchar_unlocked();
    }
    return value;
}

int main()
{
    int n = read(), p = read(), k = read();
    vector<long long> a(n);
    for (auto &x : a) x = read();
    auto inverse = batch_inverse(move(a), p);
    if (!inverse) return 1;
    long long answer = 0, weight = 1;
    for (auto x : *inverse)
    {
        weight = weight * k % p;
        answer = (answer + weight * x) % p;
    }
    printf("%lld\n", answer);
    return 0;
}
```

3.8.3 连续整数的线性逆元表

问题描述：给定素数 p 和 $n < p$，线性时间求出 $1, 2, \ldots, n$ 模 p 的逆元表。

核心思路：设 $p = qi + r$，$q = \lfloor p/i \rfloor$，$r = p \bmod i < i$。由 $qi + r \equiv 0 \pmod p$，两边乘 $i^{-1}r^{-1}$ 得 $i^{-1} \equiv -q \cdot r^{-1} \pmod p$。因为 $r < i$，r^{-1} 已经算出，按 i 递增一次递推即可；代码写成 $p - (q \cdot r^{-1} \bmod p)$，使结果落在 $[1, p)$。**接口契约：**inverse_table(n,p) 返回长度 n+1 的 int 向量，位置 i 保存 i 模 p 的逆元，1≤i≤n。位置 0 固定为 0，仅为占位，不表示零有逆元；n=0 返回单个占位项。要求 p 为素数、2≤p≤INT_MAX 且 0≤n<p，不检测素性。时间和空间均 O(n)，中间乘法用 long long；独立可抄，不依赖其他模板。不能将同一递推直接用于合数模数，即使当前 i 与模数互素；一般模数使用扩展欧几里得或任意数组批量逆元。

```
vector<int> inverse_table(int n, int p)
{
    assert(p >= 2 && 0 <= n && n < p);
    vector<int> inverse(n + 1);
    if (n) inverse[1] = 1;
    for (int i = 2; i <= n; i++) inverse[i] = p - 1LL * (p / i) * inverse[p % i] % p;
    return inverse;
}
```

模板题使用：Luogu P3811 【模板】模意义下的乘法逆元

题意：输入 1≤n≤3000000、n<p<20000528，p 保证为素数，输出 1..n 各自的逆元，每项一行。inverse_table 返回 n+1 项，0 是占位且不输出。该递推要求素数，不能照搬到合数；一般数组使用 batch_inverse。

所需模板：inverse_table（第 29 页，3.8.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3811

```
int main()
{
    int n, p;
    scanf("%d%d", &n, &p);
    auto inverse = inverse_table(n, p);
    for (int i = 1; i <= n; i++) printf("%d\n", inverse[i]);
    return 0;
}
```

3.8.4 任意数组的批量逆元

设 $s_0 = 1$，$s_i = \prod_{j<i} a_j \pmod m$，只对 s_n 求一次逆元，再从右向左：若 $t = s_{i+1}^{-1}$，则

$$a_i^{-1} = ts_i \pmod m, \qquad s_i^{-1} = ta_i \pmod m.$$

先保存原 a_i 再覆盖。总乘积与 m 互素当且仅当每项互素，模数无需为素数。实现见第 27 页：无解返回空的可选值，空输入返回空向量，模数 1 返回全零。

3.8.5 连续整数的线性逆元表

素数 p、$1 < i < p$ 时

$$i^{-1} \equiv -\lfloor p/i \rfloor \, (p \bmod i)^{-1} \pmod p,$$

从 $1^{-1} = 1$ 递推，$O(N)$。实现见第 29 页，第 0 项占位，允许 $N = 0$；要求 $N < p$。合数模数不能照搬（$p \bmod i$ 可能不可逆），单个逆元用扩展欧几里得（$\gcd(a, m) = 1$ 才有解）或素数下的 a^{p-2}。

3.9 线性同余方程

3.9.1 线性同余方程

问题描述：求解线性同余方程 $ax + b \equiv 0 \pmod m$：给出最小非负解与解的周期，或判断无解。

核心思路：方程等价于 $ax \equiv -b \pmod m$。规范化 a 后用 `extended_gcd` 求 $g = \gcd(a, m)$ 及满足 $ax_0 \equiv g \pmod m$ 的 x_0。$g \nmid b$ 时无解；否则 $x \equiv x_0 \cdot (-b/g) \pmod{m/g}$，周期为 m/g，取模后调成非负。

代码依赖：extended_gcd（第 23 页，3.6.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：linear_congruence(a,b,m) 求 a*x+b≡0 (mod m)，返回 {最小非负解, 最小正周期}，无解返回 {-1,-1}。m 为正 long long，a、b 支持完整 long long。若 g=gcd(a,m) 整除 b，周期为 m/g；包括 a=0 和 m=1，所有整数均满足时返回 {0,1}。先归一化 a，并在 int128 中对 b 取相反数和乘法。依赖 extended_gcd；递归时间、空间 O(log(m+1))。注意本接口的常数项是 +b，若题目写 a*x≡c，应传 -c；c 为 LLONG_MIN 时须避免在 64 位中取负。

```
pair<long long, long long> linear_congruence(long long a, long long b, long long m)
{
    assert(m > 0);
    using i128 = __int128_t;
    a %= m;
    if (a < 0) a += m;
    i128 x, y;
    i128 g = extended_gcd(a, m, x, y);
    if (i128(b) % g) return {-1, -1};
    i128 period = m / g;
    x = x * (-i128(b) / g) % period;
    if (x < 0) x += period;
    return {(long long)x, (long long)period};
}
```

3.9.2 线性同余：最小解与最小周期

实现见第 30 页，解 $ax + b \equiv 0 \pmod m$。a 归一化后令 $g = \gcd(a, m)$；有解当且仅当 $g \mid b$，若 $au + mv = g$，则 $x_0 = u(-b/g)$，全部解为 $x_0 + (m/g)t$，返回 $[0, m/g)$ 内最小解与最小正周期 m/g。$a = 0$ 且 $m \mid b$ 或 $m = 1$ 时返回 $(0, 1)$。b 可为 `LLONG_MIN`（内部用 `-i128(b)`）；题目为 $ax \equiv c$ 时先把 c 归一化再取负传入。exgcd 见第 24 页。追及问题 $(m - n)t + (x - y) \equiv 0 \pmod L$ 直接传速度差与起点差，不取绝对值。

3.10 中国剩余定理

3.10.1 广义 CRT：逐项合并同余

问题描述：求同时满足若干同余式 $x \equiv r_i \pmod{m_i}$ 的最小非负整数，模数不必两两互素，或判断无解。函数每次把一个新同余式合并进当前解 $x \equiv r \pmod{m}$。

核心思路：合并 $x \equiv r \pmod{m}$ 与 $x \equiv b \pmod{n}$：设 $x = r + mk$，需 $mk \equiv b - r \pmod{n}$。令 $g = \gcd(m, n)$，由 `extended_gcd` 得 $mx_0 + ny_0 = g$；$g \nmid (b - r)$ 时无解，否则 $k \equiv \frac{b-r}{g} x_0 \pmod{n/g}$，取最小非负值。新模数为 $\mathrm{lcm} = m \cdot n/g$，新余数 $r + mk$ 对它取模，原地写回 `r`、`m`。

代码依赖：extended_gcd（第 23 页，3.6.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：crt_merge(r,m,b,n) 合并 x≡r (mod m) 与 x≡b (mod n)，m,n>0，r、m 须为不同变量。从 r=0、m=1 开始逐项合并；成功后 r 为最小非负解、m 为最小正周期。余数支持完整 long long。无解返回 false；有解但新周期超过 LLONG_MAX 抛 overflow_error。两种失败都会先规范化 r，m 不变，失败后不能视为完整解。依赖 extended_gcd，中间量用 int128；O(log min(m,n)) 时间与递归空间。

```
bool crt_merge(long long &r, long long &m, long long b, long long n)
{
    using ll = long long;
    using i128 = __int128_t;
    assert(m > 0 && n > 0);
    r %= m;
    if (r < 0) r += m;
    b %= n;
    if (b < 0) b += n;
    i128 x, y;
    ll g = (ll)extended_gcd(m, n, x, y);
    i128 diff = i128(b) - r;
    if (diff % g) return false;
    i128 q = n / g, k = (diff / g * x % q + q) % q, mod = i128(m) * q;
    if (mod > LLONG_MAX) throw overflow_error("CRT modulus");
    r = (r + i128(m) * k) % mod;
    m = (ll)mod;
    return true;
}
```

模板题使用：Luogu P4777

题意：求同时满足多个同余式的最小非负整数，模数可以不互质。每行按模数、余数输入，逐式合并；合并后模数须在 long long 范围内。

所需模板：extended_gcd（第 23 页，3.6.1 节）、crt_merge（第 31 页，3.10.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4777

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int count;
    cin >> count;
    long long r = 0, modulus = 1;
    while (count--)
    {
        long long m, b;
        cin >> m >> b;
        if (!crt_merge(r, modulus, b, m))
        {
            cout << -1 << '\n';
            return 0;
        }
    }
    cout << r << '\n';
    return 0;
}
```

3.10.2 Garner：大周期 CRT 与混合进制

问题描述：给定两两互素的模数 $m_0, \ldots, m_{k-1}$ 与余数 b_i，求满足全部同余式的最小非负整数 x（可能远超 64 位）对任意目标模数的余数，或求出 x 的混合进制各位。常用于三模数 NTT 的结果合并。

核心思路：把 x 写成混合进制 $x = d_0 + d_1 m_0 + d_2 m_0 m_1 + \cdots$，$0 \le d_i < m_i$，按顺序求各位。`value[j]` 维护已确定各位之和模 m_j，`coefficient[j]` 维护 $m_0 \cdots m_{j-1} \bmod m_j$。第 i 位须满足 $\text{value}_i + \text{coef}_i \cdot d_i \equiv b_i \pmod{m_i}$，故 $d_i = (b_i - \text{value}_i) \cdot \text{coef}_i^{-1} \bmod m_i$，系数不可逆说明模数不互素；随后更新后面各 j 的两个数组。`garner` 再从高位到低位用秦九韶法在目标模数下求出 x。

代码依赖：mod_inverse（第 26 页，3.8.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：garner_digits(b,moduli) 返回按输入顺序排列的混合进制各位；garner(b,moduli,target) 返回最小非负 CRT 解对 target 的余数。两个输入向量等长，长度可为 0；模数为正且两两互素，允许模数 1，余数支持完整 long long 范围。target 为正 long long，与输入模数无需互素。输入不互素时抛 invalid_argument，即使方程有解也不适用；此时用扩展 CRT。空输入的规范解取 0。算法从不构造模数乘积，中间运算使用 int128。k 个模数耗时 $O(k^2+k \log M)$，空间 O(k)；M 为最大输入模数。依赖 mod_inverse 和 extended_gcd。

```cpp
// Pairwise coprime positive moduli. Digits use the input order.
vector<long long> garner_digits(const vector<long long> &b,
                                const vector<long long> &moduli)
{
    using ll = long long;
    using i128 = __int128_t;
    int n = b.size();
    assert(moduli.size() == b.size());
    vector<ll> value(n), coefficient(n, 1), digit(n);
    for (ll m : moduli) assert(m > 0);
    for (int i = 0; i < n; i++)
    {
        ll m = moduli[i];
        ll inverse = mod_inverse(coefficient[i], m);
        if (inverse == -1) throw invalid_argument("Garner moduli are not coprime");
        i128 delta = (i128(b[i]) - value[i]) % m;
        if (delta < 0) delta += m;
        digit[i] = delta * inverse % m;
        for (int j = i + 1; j < n; j++)
        {
            value[j] = (value[j] + i128(coefficient[j]) * digit[i]) % moduli[j];
            coefficient[j] = i128(coefficient[j]) * m % moduli[j];
        }
    }
    return digit;
}
```

规范解对目标数取模

```
// Return the least nonnegative CRT solution modulo target, not all solutions.
long long
garner(const vector<long long> &b, const vector<long long> &moduli, long long target)
{
    assert(target > 0);
    auto digit = garner_digits(b, moduli);
    long long answer = 0;
    for (int i = (int)digit.size() - 1; i >= 0; i--)
        answer = (__int128_t(answer) * moduli[i] + digit[i]) % target;
    return answer;
}
```

模板题使用：Library Checker Convolution (Mod 1,000,000,007)

题意：给出长度 n、m 的升幂系数，输出模 1000000007 的卷积，共 n+m−1 项。两数组长度均为 1 到 524288；输入系数非负且小于目标模数。分别在 167772161、469762049、1224736769 下做 NTT，再逐项调用既有 garner。真实非负系数不超过 524288 乘以 1000000006 的平方，严格小于三素数乘积。因此 CRT 最小非负代表就是原整数系数，最后对目标模数取余；仅有模数互素并不足够。输入通过 ModInt 构造器归一化。三模数公共变换容量为 2^24，本题至多需 2^20；无系数界时不能直接套用。

所需模板：extended_gcd（第 23 页，3.6.1 节）、mod_inverse（第 26 页，3.8.1 节）、garner（第 33 页，3.10.2 节）、ModInt（第 12 页，3.1.2 节）、NttConvolution（第 73 页，4.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/convolution_mod_1000000007

```
template <int p>
auto conv(const vector<int> &a, const vector<int> &b)
{
    using N = NttConvolution<p>;
    typename N::Poly x(a.begin(), a.end()), y(b.begin(), b.end());
    return N::multiply(move(x), move(y));
}

int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<int> a(n), b(m);
    for (auto &x : a) cin >> x;
    for (auto &x : b) cin >> x;
    auto x = conv<167772161>(a, b);
    auto y = conv<469762049>(a, b);
    auto z = conv<1224736769>(a, b);
    vector<long long> r(3), p = {167772161, 469762049, 1224736769};
    for (int i = 0; i < n + m - 1; i++)
    {
        r[0] = x[i].v;
        r[1] = y[i].v;
        r[2] = z[i].v;
        cout << garner(r, p, 1000000007) << (i + 1 == n + m - 1 ? '\n' : ' ');
    }
```

```
}
```

3.10.3 扩展 CRT：逐式合并

实现见第 31 页的 `crt_merge`：从 $r = 0, m = 1$ 起依次合并 $x \equiv b \pmod{n}$，一旦无解即停止。设 $g = \gcd(m, n)$，$mu + nv = g$；若 $g \nmid (b - r)$ 无解，否则

$$q = n/g, \qquad t \equiv \frac{b-r}{g} u \pmod{q}, \qquad 0 \le t < q,$$

新状态为 $((r + mt) \bmod mq,\ mq)$。模数不必互素。要求正模数且最终周期不超过有符号 64 位；中间量用 128 位。无解返回假，周期越界抛异常。时间 $O(k \log M)$，递归栈 $O(\log M)$。

3.10.4 Garner：总周期可以超过整数范围

模数 m_i 两两互素，$P = \prod m_i$，把解写成混合进制

$$x = d_0 + d_1 m_0 + d_2 m_0 m_1 + \cdots, \qquad 0 \le d_i < m_i.$$

对每个后续模数维护已定部分余数 v_j 与位权 $c_j = \prod_{t<i} m_t \bmod m_j$，$d_i = (b_i - v_i) c_i^{-1} \bmod m_i$，然后 $v_j \leftarrow v_j + c_j d_i$，$c_j \leftarrow c_j m_i$。所得 $x \in [0, P)$ 为最小非负解；模数 1 可直接参与。实现见第 33 页。求 $x \bmod T$ 时从高位 Horner，T 无需与 m_i 互素；结果只是规范解的余数，多模 NTT 还原整数时须保证真实值在 $[0, P)$ 内。

3.11 升幂引理

3.11.1 素因子估值与升幂：先约去公共因子

p 为素数，a,b,n 为正整数，$v_p(x)$ 为非零整数 x 中 p 的重数。$v_p(0)$ **无定义**：差式先判 $a=b$。

3.11.1.1 基本运算与公共因子

$v_p(xy)=v_p(x)+v_p(y)$，$v_p(x^n)=nv_p(x)$；$v_p(x+y)\ge\min(v_p(x),v_p(y))$，估值不同才取等。记 $\alpha=v_p(a)$、$\beta=v_p(b)$：若 $\alpha\ne\beta$，$v_p(a^n\pm b^n)=n\min(\alpha,\beta)$；若 $\alpha=\beta=t$，写 $a=p^tA,b=p^tB$，则 $v_p(a^n\pm b^n)=nt+v_p(A^n\pm B^n)$，再对 A,B 用 LTE。

3.11.1.2 奇素数：差式与和式的条件不同

p 为奇素数且 $p\nmid ab$：

$$\begin{aligned} p\mid a-b,\ a\ne b \quad&\Longrightarrow\quad v_p(a^n-b^n)=v_p(a-b)+v_p(n),\\ p\mid a+b,\ n\text{ 为奇数} \quad&\Longrightarrow\quad v_p(a^n+b^n)=v_p(a+b)+v_p(n).\end{aligned}$$

差式不要求 n 奇，和式必须 n 奇；不要求 $\gcd(a,b)=1$。

3.11.1.3 素数 2：先分指数奇偶

a,b 均为奇数，差式要求 $a\ne b$：

$$v_2(a^n-b^n)=\begin{cases}v_2(a-b), & n\text{ 奇},\\ v_2(a-b)+v_2(a+b)+v_2(n)-1, & n\text{ 偶},\end{cases}\qquad v_2(a^n+b^n)=\begin{cases}v_2(a+b), & n\text{ 奇},\\ 1, & n\text{ 偶}.\end{cases}$$

若 $4\mid a-b$，差式可写 $v_2(a-b)+v_2(n)$。奇偶不同则和差均奇；都偶先约去公共 2 因子。

3.11.1.4 把大整数整除转成指数条件

$M=\prod p_i^{e_i}\mid X$（$X\ne0$）等价于逐项 $v_{p_i}(X)\ge e_i$，无需构造大幂。LTE 只处理满足条件的局部因子，不能代替乘法阶或离散对数。$a+b$、$|a-b|$、nt 须防溢出。阶乘估值用 $v_p(N!)=\sum_{j\ge1}\lfloor N/p^j\rfloor$（ExLucas，第 39 页），不套 LTE；素数幂开根还须检查单位部分有根，估值条件不充分。

3.12 阶乘取模

3.12.1 Legendre 赋值与 Kummer 进位判据

p 为素数，$s_p(n)$ 为 p 进制数位和，$n \geq 0$：

$$v_p(n!) = \sum_{j \geq 1} \left\lfloor \frac{n}{p^j} \right\rfloor = \frac{n - s_p(n)}{p-1}.$$

反复 $n \leftarrow \lfloor n/p \rfloor$ 累加，$O(1+\log_p(n+1))$，不要计算 p^j（会溢出）。`ExLucas::valuation(n,p)` 见第 39 页：n 为 unsigned64，p 为正 int 素数，不验证素性。

3.12.1.1 组合数：逐位相加时进了几次位

$0 \leq k \leq n$，$h = n - k$：

$$v_p\binom{n}{k} = \frac{s_p(k) + s_p(h) - s_p(n)}{p-1}$$

等于 p 进制下 $k + h$ 的进位次数（含传播进位），即 Kummer 定理。

3.12.1.2 素数幂为零与 Lucas 的边界

$\binom{n}{k} \equiv 0 \pmod{p^a} \iff v_p\binom{n}{k} \geq a$；合数 m 须每个素数幂分量都满足，模 1 恒为 0。零进位 $\iff$ 每位 $k_i \leq n_i$，即 Lucas 非零（第 38 页）。赋值只判整除，不给出非零余数。$k > n$ 先返回 0。LTE 选型见第 3.11.1 节（第 36 页）。

3.12.2 阶乘表：组合数与排列数

素数 p，$0 \leq k \leq n < p$：

$$C(n,k) = n!\,(k!)^{-1}\,((n-k)!)^{-1}, \qquad P(n,k) = n!\,((n-k)!)^{-1} \pmod{p}.$$

扩表到 N：正向补阶乘，只对 $N!$ 求一次逆，再 $(i-1)!^{-1} = i\,(i!)^{-1}$ 反向补。查询前显式扩表，查询 $O(1)$；扩表后旧元素引用失效。见第 111 页。

3.13 卢卡斯定理

3.13.1 Lucas：小素数模的大下标组合数

问题描述：给定较小的素数 p 和很大的 n, k，求 $\binom{n}{k} \bmod p$。

核心思路：Lucas 定理：把 n, k 写成 p 进制，$\binom{n}{k} \equiv \prod_i \binom{n_i}{k_i} \pmod p$，某一位 $k_i > n_i$ 时整体为 0。构造时预处理 $0..p-1$ 的阶乘 `fac`，用费马小定理求 $(p-1)!$ 的逆，再倒推逆阶乘 `ifac`。`choose` 每轮取最低位 $a = n \bmod p$、$b = k \bmod p$，乘上 $\binom{a}{b}$，再把 n, k 除以 p。

接口契约：Lucas(p) 预处理素数模数 p，choose(n,k) 返回 C(n,k) 模 p；n、k 为完整 unsigned long long 范围，k>n 返回 0，C(0,0)=1。重新赋值 Lucas(p) 可更换模数并重新预处理；不检测素性，要求调用者保证 p 为素数且 p≥2。预处理时间 O(p+log p)，动态版空间 O(p)；单次查询 O(1+log_p(n+1))。适合小素数模数，通常 $p \le 10^6$；大素数不能因查询下标小就忽略这里整段 O(p) 的预处理。fac、ifac 下标为 0..p-1，模数和表只读；中间乘法用 long long。合数模数请使用 exLucas。

```
struct Lucas
{
    int mod;
    vector<int> fac, ifac;

    Lucas(int p)
    {
        assert(p >= 2);
        mod = p;
        fac.resize(p);
        ifac.resize(p);
        fac[0] = 1;
        for (int i = 1; i < p; i++) fac[i] = 1LL * fac[i - 1] * i % p;
        long long a = fac[p - 1], r = 1;
        for (int e = p - 2; e; e >>= 1)
        {
            if (e & 1) r = r * a % p;
            a = a * a % p;
        }
        ifac[p - 1] = r;
        for (int i = p - 1; i; i--) ifac[i - 1] = 1LL * ifac[i] * i % p;
    }
```

```
    int choose(unsigned long long n, unsigned long long k) const
    {
        if (k > n) return 0;
        long long answer = 1;
        while (n || k)
        {
            int a = n % mod, b = k % mod;
            if (b > a) return 0;
            answer = answer * fac[a] % mod;
            answer = answer * ifac[b] % mod;
            answer = answer * ifac[a - b] % mod;
            n /= mod;
            k /= mod;
        }
        return answer;
    }
};
```

模板题使用：Luogu P3807

题意：多组查询 C(n+m,n) 模 p，题目保证 p 为素数且不超过 10^5。每组构造 Lucas(p)，choose 的参数为 n+m、n；此题范围内 n+m 不会溢出。

所需模板：Lucas（第 38 页，3.13.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3807

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t;
    cin >> t;
    while (t--)
    {
        unsigned long long n, m;
        int p;
        cin >> n >> m >> p;
        Lucas solver(p);
        cout << solver.choose(n + m, n) << '\n';
    }
    return 0;
}
```

3.13.2 合数模数组合数（扩展 Lucas）

问题描述：给定任意正整数模数 M（可为合数，规模约 10^6）和很大的 n, k，求 $\binom{n}{k} \bmod M$。

核心思路：把 M 分解为互素的素数幂 $q = p^e$，分别求 $\binom{n}{k} \bmod q$ 后用 CRT 合并。模 q 时把 $n!$ 写成 $p^{v(n)}u(n)$，$u(n)$ 与 p 互素：v 用 Legendre 公式；$n!$ 中不被 p 整除的因子模 q 以 q 为周期，故 $u(n) = (\text{整段积})^{\lfloor n/q \rfloor} \cdot (\text{零头积}) \cdot u(\lfloor n/p \rfloor)$。于是 $\binom{n}{k} \equiv p^v \, u(n)\, u(k)^{-1} u(n-k)^{-1} \pmod q$，其中 $v = v(n) - v(k) - v(n-k)$，$v \geq e$ 时该分量为 0。

思路中的情况	代码中的处理	行
分解模数	试除得到每个素数幂 $q = p^e$	41
段内前缀积	`fac` 中该因子的一段：$0..q-1$ 内不被 p 整除的数之积，末尾多存一份整段积	50
CRT 系数	$c_q = (M/q) \cdot ((M/q)^{-1} \bmod q)$，模 q 为 1、模其他素数幂为 0	53
$n!$ 中 p 的指数	Legendre 公式 $\sum_{i\geq 1}\lfloor n/p^i \rfloor$	63
去掉 p 后的 $n!$	整段积的 $\lfloor n/q \rfloor$ 次幂乘零头，再对 $\lfloor n/p \rfloor$ 迭代	73
p 的指数不少于 e	该分量为 0，跳过	87
合并分量	分母单位部分用欧拉定理求逆，乘 p^v 后按 CRT 系数累加	92

接口契约： ExLucas(mod) 预处理固定模数，choose(n,k) 返回 C(n,k) 模 mod；n、k 为 unsigned long long，k>n 返回 0，mod=1 返回 0。适合 mod≤10^6 的合数或素数模数，不能用于 10^9 级素数的整段预处理。分解 mod 为互素素数幂 q=p^e，预处理每段内不含 p 因子的前缀积；unit(n,f) 去除 n! 中全部 p 因子，valuation 统计指数。分子分母指数之差不少于 e 时该项为零，否则只对互素单位部分求逆，最后 CRT 合并。预处理 O(√mod+∑q)，空间 O(∑(q+1))；每次查询 O(∑(log_p(n+1) · log(n+1)+log q))，求和遍历不同素因子。所有乘法用 long long；同一对象可重复查询，factors、fac 与辅助函数视为内部数据。

```
struct ExLucas
{
    using ull = unsigned long long;
    using ll = long long;

    struct Factor
    {
        int p, q, exponent, start, crt;
    };

    int mod;
    vector<int> fac;
    vector<Factor> factors;

    static ll power(ll a, ull b, int m)
    {
        ll result = 1;
        while (b)
        {
            if (b & 1) result = result * a % m;
            a = a * a % m;
            b >>= 1;
        }
        return result;
    }

    ExLucas(int m) : mod(m)
    {
        assert(m >= 1);
        int x = m;
        for (int p = 2; p <= x / p; p++)
        {
            if (x % p) continue;
            int q = 1, e = 0;
            do
            {
                x /= p;
                q *= p;
```

```
            e++;
        } while (x % p == 0);
        add_factor(p, q, e);
    }
    if (x > 1) add_factor(x, x, 1);
}

void add_factor(int p, int q, int e)
{
    int start = fac.size();
    fac.push_back(1);
    for (int i = 1; i < q; i++) fac.push_back(ll(fac.back()) * (i % p ? i : 1) % q);
    fac.push_back(fac.back());
    int phi = q - q / p;
    int crt = ll(mod / q) * power(mod / q % q, phi - 1, q) % mod;
    factors.push_back({p, q, e, start, crt});
}

static ull valuation(ull n, int p)
{
    ull result = 0;
    while (n)
    {
        n /= p;
        result += n;
    }
    return result;
}

ll unit(ull n, const Factor &f) const
{
    ll result = 1;
    while (n)
    {
        result = result * power(fac[f.start + f.q], n / f.q, f.q) % f.q;
        result = result * fac[f.start + n % f.q] % f.q;
        n /= f.p;
    }
    return result;
}
```

```
    int choose(ull n, ull k) const
    {
        if (k > n || mod == 1) return 0;
        ll answer = 0;
        for (const auto &f : factors)
        {
            ull e = valuation(n, f.p) - valuation(k, f.p) - valuation(n - k, f.p);
            if (e >= ull(f.exponent)) continue;
            ll denominator = unit(k, f) * unit(n - k, f) % f.q;
            ll residue =
                unit(n, f) * power(denominator, f.q - f.q / f.p - 1, f.q) % f.q;
            residue = residue * power(f.p, e, f.q) % f.q;
            answer = (answer + residue * f.crt) % mod;
        }
        return answer;
    }
};
```

模板题使用：Luogu P4720

题意：求 C(n,m) 模 p，p 可以为合数且不超过 10^6。构造 ExLucas(p) 后 choose(n,m)；预处理与模数规模有关，不能直接用于 10^9 级模数。

所需模板：ExLucas（第 39 页，3.13.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4720

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    unsigned long long n, m;
    int p;
    std::cin >> n >> m >> p;
    ExLucas comb(p);
    std::cout << comb.choose(n, m) << '\n';
}
```

3.13.3 Lucas：按素数进制逐位计算

p 为素数，n_i, k_i 为 p 进制位：$\binom{n}{k} \equiv \prod_i \binom{n_i}{k_i} \pmod p$，某位 $k_i > n_i$ 则为零。只预处理 $0!, \dots, (p-1)!$ 及逆元；每位常数次乘法。实现见第 38 页，参数为 n, k；自行求和时检查 64 位溢出。

3.13.4 合数模数组合数与插值

$q = p^e$，$v_p(n!) = \sum_{j \ge 1} \lfloor n/p^j \rfloor$，$U_p(n) = n!/p^{v_p(n!)}$，$t = v_p(n!) - v_p(k!) - v_p((n-k)!)$。$t \ge e$ 时为零，否则

$$\binom{n}{k} \equiv U_p(n)\, U_p(k)^{-1}\, U_p(n-k)^{-1}\, p^t \pmod q,$$

各素数幂结果 CRT 合并，见第 39 页。

素数域中 n 个两两不同的 x_i，$w_i = y_i / \prod_{j \ne i}(x_i - x_j)$：

$$f(k) = \sum_{i=0}^{n-1} w_i \prod_{j<i}(k - x_j) \prod_{j>i}(k - x_j),$$

查询点可等于样本点。$x_i = i$ 时分母为 $i!\,(n-1-i)!\,(-1)^{n-1-i}$，要求 $n \le p$。见第 165 页。$S_d(k) = \sum_{i \le k} i^d$ 在 $0 \le d \le p-2$ 时取 $0, \ldots, d+1$ 共 $d+2$ 个样本。

3.14 二次剩余

3.14.1 素数模平方根（Cipolla）

问题描述：给定素数 p 与整数 n，求方程 $x^2 \equiv n \pmod p$ 的全部解。

核心思路：$n \equiv 0$ 或 $p = 2$ 时唯一解就是 n。否则用 Euler 判别法，$n^{(p-1)/2} \neq 1$ 时无解。$p \equiv 3 \pmod 4$ 时 $n^{(p+1)/4}$ 直接是一个根。其余情况用 Cipolla：随机找 a 使 $w = a^2 - n$ 为非二次剩余，在扩域 $\mathbb{F}_p[\sqrt{w}]$ 中计算 $(a + \sqrt{w})^{(p+1)/2}$，元素用（实部，虚部）表示，乘法中 $(\sqrt{w})^2 = w$，结果的实部就是一个根。另一个根为 $p - x$，排序后一并返回。

思路中的情况	代码中的处理	行
$n \equiv 0$ 或 $p = 2$	只有一个根 n	6
Euler 判别	n 不是二次剩余则返回空	18
$p \equiv 3 \pmod 4$	根为 $n^{(p+1)/4}$	21
找非剩余 w	随机 a 直到 $w^{(p-1)/2} = p - 1$	30
扩域乘法	$(x_1 + y_1\sqrt{w})(x_2 + y_2\sqrt{w})$ 的实部与虚部	32
两个根	x 与 $p - x$ 按大小排序返回	48

接口契约：mod_sqrt(n,p) 返回 vector<long long>，按升序列出全部不同根；无解返回空向量，零或模数 2 返回单根。要求 p 为素数，2≤p≤INT_MAX，n 支持完整 long long 范围并在入口归一化。先用 Euler 判别；p≡3 (mod 4) 时直接快速幂，其余使用 Cipolla 二次扩域。函数内保存固定种子的伪随机生成器；随机性影响寻找非剩余的尝试次数，不影响结果，通常期望 O(log p) 时间、O(1) 辅助空间。乘法用 long long，扩域实部的三因子乘积中间先取模；指数中的 p+1 先转 long long，避免 INT_MAX 溢出。此版本不处理合数模数及 64 位大素数，也不用于浮点开方。

```
vector<long long> mod_sqrt(long long n, int p)
{
    using ll = long long;
    assert(p >= 2);
    n = (n % p + p) % p;
    if (n == 0 || p == 2) return {n};
    auto power = [p](ll a, ll b)
    {
        ll result = 1;
        while (b)
        {
            if (b & 1) result = result * a % p;
            a = a * a % p;
            b >>= 1;
        }
        return result;
    };
    if (power(n, (p - 1) / 2) != 1) return {};
    ll answer;
    if (p % 4 == 3)
        answer = power(n, (ll(p) + 1) / 4);
    else
    {
        static mt19937_64 rng(712367);
        ll a, w;
```

```
        do
        {
            a = rng() % p;
            w = (a * a % p - n + p) % p;
        } while (power(w, (p - 1) / 2) != p - 1);
        using P = pair<ll, ll>;
        auto mul = [p, w](P x, P y)
        {
            ll real = (x.first * y.first + x.second * y.second % p * w) % p;
            ll imag = (x.first * y.second + x.second * y.first) % p;
            return P{real, imag};
        };
        P result{1, 0}, base{a, 1};
        ll exponent = (ll(p) + 1) / 2;
        while (exponent)
        {
            if (exponent & 1) result = mul(result, base);
            base = mul(base, base);
            exponent >>= 1;
        }
        answer = result.first;
    }
    ll other = p - answer;
    if (answer > other) swap(answer, other);
    return {answer, other};
}
```

模板题使用：Luogu P5491

题意：多组求 x*x 与 N 模 p 同余的全部解，题目保证 p 为奇素数。mod_sqrt 返回升序且去重的根，空数组表示无解并输出 Hola!。

所需模板：mod_sqrt（第 43 页，3.14.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5491

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int tests;
    std::cin >> tests;
    while (tests--)
    {
        long long n;
        int p;
        std::cin >> n >> p;
        auto roots = mod_sqrt(n, p);
        if (roots.empty())
            std::cout << "Hola!\n";
        else
        {
            for (int i = 0; i < (int)roots.size(); i++)
                std::cout << roots[i] << (i + 1 == (int)roots.size() ? '\n' : ' ');
        }
    }
}
```

3.14.2 素数模平方根

奇素数 p、$n \not\equiv 0$：有平方根当且仅当 $n^{(p-1)/2} \equiv 1 \pmod p$。$p \equiv 3 \pmod 4$ 时 $r = n^{(p+1)/4}$，另一根 $p - r$。Cipolla：取 a 使 $w = a^2 - n$ 非平方，$\omega^2 = w$，根为 $(a+\omega)^{(p+1)/2}$。实现见第 43 页；零和模 2 单独处理；合数模不适用。

3.15 阶 & 原根

3.15.1 乘法阶与原根

问题描述：给定模数 n，求剩余类 a 的乘法阶，判断 a 是否为原根，求最小原根，或升序列出全部原根。

核心思路：构造时试除分解求 $\varphi(n)$ 及其不同素因子 `factors`，并按“n 为 $2, 4, p^k, 2p^k$（p 为奇素数）”判断原根是否存在。阶：从 $\varphi(n)$ 出发，对每个素因子 p，只要 $a^{\text{result}/p} \equiv 1$ 就把 `result` 除以 p。原根判定：a 与 n 互素，且对每个 p 都有 $a^{\varphi/p} \not\equiv 1$。最小原根从 1 起逐个试；全部原根是 g^k（$\gcd(k, \varphi) = 1$），按 k 累乘枚举后排序。

代码依赖：Mod64（第 7 页，2.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：构造 PrimitiveRoot(n)，要求 2≤n≤INT_MAX；重新赋值 PrimitiveRoot(n) 可重建。order(a) 返回 a 模 n 的乘法阶，a 不可逆时返回 0；is_root(a) 判断该剩余类是否为原根，a 可为完整 long long。minimum() 返回 1..n-1 内最小原根，无原根返回 -1；all() 返回升序的全部原根，无原根返回空向量。mod、phi、exists 只读；factors 为 phi 的不同素因子，辅助 factor 为试除分解。依赖 Mod64::power。初始化 O($\sqrt{n}$)，求阶 O($\log^2 n$)，原根判定 O((1+ω(phi))log n)。若最小原根为 G，顺序枚举最坏检查 G 个候选，不能假定 G 总是常数。枚举全部另需 O(phi log phi+c log c) 时间、O(c) 空间，c 为原根数；大模数只求最小原根或阶时，不应调用 all。

```
struct PrimitiveRoot
```

```
{
    int mod, phi;
    bool exists;
    vector<int> factors;

    static vector<int> factor(int n)
    {
        vector<int> result;
        for (int p = 2; p <= n / p; p++)
            if (n % p == 0)
            {
                result.push_back(p);
                while (n % p == 0) n /= p;
            }
        if (n > 1) result.push_back(n);
        return result;
    }
```

```
    PrimitiveRoot(int n)
    {
        assert(n >= 2);
        mod = n;
        phi = n;
        for (int p : factor(n)) phi -= phi / p;
        int odd = n;
        if (odd % 2 == 0) odd /= 2;
        exists = n == 2 || n == 4 || (odd % 2 && factor(odd).size() == 1);
        factors = factor(phi);
    }

    int order(long long a) const
    {
        a %= mod;
        if (a < 0) a += mod;
        if (gcd(a, (long long)mod) != 1) return 0;
        int result = phi;
        for (int p : factors)
            while (result % p == 0 && Mod64::power(a, result / p, mod) == 1)
                result /= p;
        return result;
    }
```

```
    bool is_root(long long a) const
    {
        if (!exists) return false;
        a %= mod;
        if (a < 0) a += mod;
        if (gcd(a, (long long)mod) != 1) return false;
        for (int p : factors)
            if (Mod64::power(a, phi / p, mod) == 1) return false;
        return true;
    }

    int minimum() const
    {
        if (!exists) return -1;
        for (int g = 1; g < mod; g++)
            if (is_root(g)) return g;
        return -1;
    }
```

```
    vector<int> all() const
    {
        int g = minimum();
        vector<int> roots;
        if (g == -1) return roots;
        long long value = 1;
        for (int k = 1; k <= phi; k++)
        {
            value = value * g % mod;
            if (gcd(k, phi) == 1) roots.push_back(value);
        }
        sort(roots.begin(), roots.end());
        return roots;
    }
};
```

模板题使用：Luogu P6091

题意：多组枚举模 n 的全部原根，先输出个数，再输出升序原根的第 d、2d、3d 等项。all 返回升序数组，下标从 d-1 开始；即使没有取到任何原根，第二行也必须输出空行。

所需模板：Mod64（第 7 页，2.1 节）、PrimitiveRoot（第 45 页，3.15.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P6091

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t;
    cin >> t;
    while (t--)
    {
        int n, d;
        cin >> n >> d;
        auto roots = PrimitiveRoot(n).all();
        cout << roots.size() << '\n';
        bool first = true;
        for (int i = d - 1; i < (int)roots.size(); i += d)
        {
            if (!first) cout << ' ';
            cout << roots[i];
            first = false;
        }
        cout << '\n';
    }
    return 0;
}
```

3.15.2 乘法阶与原根

$\gcd(a,n)=1$ 时阶 $d \mid \varphi(n)$：从 $d=\varphi(n)$ 起，对每个素因子 q，$a^{d/q}\equiv 1$ 就除去 q。不可逆 a 接口返回 0。a 为原根当且仅当 $\gcd(a,n)=1$ 且对 $\varphi(n)$ 每个素因子 q 有 $a^{\varphi(n)/q}\not\equiv 1$。$n\ge 2$ 时原根仅存在于 $2,4,p^k,2p^k$（p 奇素数）。$\operatorname{ord}(g^t)=\varphi(n)/\gcd(t,\varphi(n))$，全部原根为 $\gcd(t,\varphi(n))=1$ 的 g^t，共 $\varphi(\varphi(n))$ 个，按大小输出需排序。实现见第 45 页。$\operatorname{ord}(a^k)=d/\gcd(d,k)$；两阶互素时乘积之阶为两阶之积。

3.15.3 自动准备模数分解与原根

奇素数幂 $m=p^e$ 有原根，设 $\varphi=m/p\cdot(p-1)$，候选 g 为原根当且仅当

$$p\nmid g,\qquad g^{\varphi/q}\not\equiv 1 \pmod m \quad \text{对每个不同素因子} q\mid\varphi.$$

须检查模 p^e 的阶，不能只取模 p 的原根。准备成本含 M 与各 φ 的分解及逐个候选的模幂；同一模数多个方程应复用分解结果。

3.16 离散对数

3.16.1 扩展 BSGS

问题描述：给定 a, b, m（a 与 m 不必互质），求最小的非负整数 x 使 $a^x \equiv b \pmod m$，无解返回 -1。

核心思路：把方程看成 $k \cdot a^{x-\text{offset}} \equiv b$ 逐步化简：只要 $g = \gcd(a, m) > 1$，先检查当前指数 `offset` 是否已是解（即 $k \equiv b$），否则要求 $g \mid b$，把 b 和模数同除以 g，系数 k 乘上 a/g，偏移加一。a 与 m 互质后求 $a^y \equiv bk^{-1}$：baby step 表存 a^j（$0 \le j < s$，同值保留最小 j），giant step 从目标值出发每次乘 a^{-s}，第 i 步命中 a^j 即得 $y = is + j$。i 递增、j 取最小，首个命中就是最小解。

思路中的情况	代码中的处理	行
平凡情形	$m = 1$ 或 $b = 1$ 时 $x = 0$；底数为 1 而 $b \neq 1$ 时无解	11
$\gcd(a, m) > 1$	循环剥离公因子	14
较小指数恰为解	$k \equiv b$ 时返回当前 `offset`	16
化为互质问题	目标值乘 k^{-1}	23
baby step	记录 a^j 的最小指数	30
giant step 命中	返回 offset $+ is + j$	38

代码依赖：Mod64（第 7 页，2.1 节）、mod_inverse（第 26 页，3.8.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：依赖 number_theory.hpp 的求逆与快速幂。solve(a,b,m) 返回最小 x≥0 使 a^x≡b (mod m)，无解返回 -1；1≤m≤10^{12}，a、b 可为完整 long long 范围，入口归一化，约定 0^0=1。先剥离 gcd 并逐次检查较小指数，再在互质剩余问题上 BSGS。baby 表对同余幂保留最小指数 j，按递增大步 i 检查区间，因此首个命中即最小解。单次期望时间 O(√m+log m)，空间 O(√m)，哈希表先 reserve；基于 unordered_map 的界为期望界。每次调用独立建表；底数 1 及零指数解直接返回。

```
struct DiscreteLog
{
    using ll = long long;

    // 1<=m<=1e12; Expected O(sqrt(m)) time, O(sqrt(m)) storage. Smallest x>=0, or -1.
    static ll solve(ll a, ll b, ll m)
    {
        assert(m >= 1 && m <= 1000000000000LL);
        a = (a % m + m) % m;
        b = (b % m + m) % m;
        if (m == 1 || b == 1) return 0;
        if (a == 1) return -1;
        ll offset = 0, k = 1;
        for (ll g; (g = gcd(a, m)) > 1;)
        {
            if (b == k) return offset;
            if (b % g) return -1;
            b /= g;
            m /= g;
            k = (__int128)k * (a / g) % m;
            ++offset;
        }
        ll target = (__int128)b * mod_inverse(k, m) % m;
        ll step = sqrtl(m) + 1;
        unordered_map<ll, ll> baby;
        baby.reserve(step);
        ll cur = 1 % m;
        for (ll j = 0; j < step; j++)
```

```
        {
            if (!baby.count(cur)) baby[cur] = j;
            cur = (__int128)cur * a % m;
        }
        ll inv = mod_inverse(Mod64::power(a, step, m), m);
        cur = target;
        for (ll i = 0; i <= step; i++)
        {
            auto it = baby.find(cur);
            if (it != baby.end()) return offset + i * step + it->second;
            cur = (__int128)cur * inv % m;
        }
        return -1;
    }
};
```

模板题使用：Luogu P4195

题意：多组求 a^x 与 b 模 p 同余的最小非负整数 x，a 与 p 不必互质。输入顺序为 a、p、b，调用顺序为 solve(a,b,p)；返回 -1 时输出 No Solution，三个 0 结束输入。

所需模板：Mod64（第 7 页，2.1 节）、extended_gcd（第 23 页，3.6.1 节）、mod_inverse（第 26 页，3.8.1 节）、DiscreteLog（第 49 页，3.16.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4195

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    long long a, p, b;
    while (std::cin >> a >> p >> b)
    {
        if (a == 0 && p == 0 && b == 0) break;
        long long answer = DiscreteLog::solve(a, b, p);
        if (answer == -1)
            std::cout << "No Solution\n";
        else
            std::cout << answer << '\n';
    }
}
```

3.17 高次剩余 & 单位根

3.17.1 素数模高次剩余：全部根的紧凑表示

问题描述：给定素数 p 及其原根 g、正整数 k 与整数 a，求同余方程 $x^k \equiv a \pmod p$ 的全部解。解集以首项 `first`、公比 `ratio`、个数 `count` 的等比数列形式返回，不必展开即可得知解数。

核心思路：令 $x = g^y$，并用 `DiscreteLog` 求出 $a = g^L$ 中的 L，原方程化为指数上的线性同余 $ky \equiv L \pmod{p-1}$。`linear_congruence` 给出最小解 y_0 与周期 $T = (p-1)/\gcd(k, p-1)$，同余无解则原方程无解。于是全部根为 $g^{y_0}(g^T)^i$，$0 \le i < \gcd(k, p-1)$，即 `first` 取 g^{y_0}、`ratio` 取 g^T、`count` 取 $(p-1)/T$。$a \equiv 0$ 时唯一根为 0，单独返回。

代码依赖：DiscreteLog（第 49 页，3.16.1 节）、Mod64（第 7 页，2.1 节）、linear_congruence（第 30 页，3.9.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：solve(a,k,p,g) 求 x^k≡a (mod p)；p 必须是 2 到 1e12 的素数，g 是模 p 原根，k 为正 unsigned long long，a 支持完整 signed long long。无解返回 nullopt；否则得到 first、ratio、count，全部解为 first*ratio^i mod p，0≤i<count，无重复；first 不保证最小。a≡0 时仅有根 0，返回 {0,1,1}。枚举时每次乘 ratio，乘法先转 int128；无需展开全部根即可读取解数。给定原根后，期望 O(√p) 时间与空间，依赖 DiscreteLog、linear_congruence 及模快速幂。不验证 p 的素性或 g 的阶；合数模数不在此接口范围内。

```
struct KthResidue
{
    using ll = long long;
    ll first, ratio, count;

    // Prime p <= 1e12; g is a primitive root of p; k > 0.
    // All roots: first * ratio^i mod p, 0 <= i < count.
    static optional<KthResidue> solve(ll a, unsigned long long k, ll p, ll g)
    {
        assert(p >= 2 && p <= 1000000000000LL && g >= 1 && g < p && k > 0);
        a %= p;
        if (a < 0) a += p;
        if (a == 0) return KthResidue{0, 1, 1};
        ll logarithm = DiscreteLog::solve(g, a, p);
        if (logarithm < 0) return nullopt;
        auto [exponent, period] = linear_congruence(k % (p - 1), -logarithm, p - 1);
        if (exponent < 0) return nullopt;
        ll first = Mod64::power(g, exponent, p);
        ll ratio = Mod64::power(g, period, p);
        return KthResidue{first, ratio, (p - 1) / period};
    }
};
```

3.17.2 素数模高次剩余与单位根

实现见第 51 页；依赖离散对数、线性同余和快速幂。设 p 为素数，g 是原根，$k > 0$。$a = 0$ 时根只有零；否则 $a = g^b$、$x = g^y$，

$$ky \equiv b \pmod{p-1}.$$

令 $d = \gcd(k, p-1)$，$d \nmid b$ 时无解；否则 $y = y_0 + i(p-1)/d$，$0 \le i < d$，存为

$$\texttt{first} = g^{y_0}, \qquad \texttt{ratio} = g^{(p-1)/d}, \qquad \texttt{count} = d.$$

依次乘 ratio 枚举，不保证递增。$p \le 10^{12}$，调用者提供已验证的素数与原根；k 为 unsigned long long，a 先正规化。空 optional 表示无解，`{0,1,1}` 表示唯一零根。期望时间、空间 $O(\sqrt{p})$，枚举另需 $O(d)$；模乘须扩展到 128 位。

3.17.3 素数幂模高次剩余与根的提升

令 $M = p^e$。$a = 0$ 时根为 $p^{\lceil e/k \rceil}$ 的倍数。否则 $t = v_p(a) < e$，须 $k \mid t$，否则无解；取 $s = t/k$，$q = p^{e-t}$，$x = p^s y$，解

$$y^k \equiv a/p^t \pmod q,$$

每个单位根 y 对应 $x = p^s y + \ell p^s q$，$0 \le \ell < p^{t-s}$。奇素数用模 p^e 的原根转线性同余；$q = 2$ 时单位根只有 1；$q \ge 4$ 为二的幂时单位写成 $(-1)^z 5^y$，z 模 2、y 模 $q/4$（$q = 4$ 只有符号分支），分别解指数同余。二的高次幂没有原根。时间由 BSGS 决定，期望 $O(\sqrt{q})$。

3.17.4 一般合数模高次剩余：CRT 组合

设 $M = \prod_t m_t$，$m_t = p_t^{e_t}$ 两两互素。分别求局部根族，任一局部无解则整体无解；整体根数为 $\prod_t c_t$。预处理

$$E_t = \frac{M}{m_t}\left(\left(\frac{M}{m_t}\right)^{-1} \bmod m_t\right) \bmod M, \qquad x = \sum_t r_t E_t \bmod M.$$

按混合进制索引可逐个取根，不需建出全部根；CRT 乘积扩展到 128 位。$M = 1$ 时唯一代表为 0。

3.18 数论分块

3.18.1 整除分块：前缀和来源决定算法

$H(t) = \sum_{i \le t} h(i)$，$H(0) = 0$，$q_A = \lfloor A/l \rfloor, q_B = \lfloor B/l \rfloor$，$l \le \min(A, B)$：

$$r = \min(\lfloor A/q_A \rfloor, \lfloor B/q_B \rfloor), \quad \text{整块贡献}(H(r) - H(l-1))q_A q_B.$$

两边都要参与取 r。段数 $O(\sqrt{A} + \sqrt{B})$（且 $\le \min(A, B)$），不是 $O(\sqrt{\min(A, B)})$。

3.18.1.1 按数据与资源选前缀和

上界可预筛，询问多： 第 20 页 `LinearSieve s(L)` 的 `s.mu`、`s.phi` 建前缀和，$O(L)$ 时间空间，查表 $O(1)$；筛上限覆盖短边。μ 前缀可负，φ 前缀勿用 `int`。

只求少量巨大前缀： 第 54 页 `DuJiao s(B)`：`s.mertens(n)`（有符号 64 位）、`s.totient_sum(n)`（128 位），$0 \le n \le 10^{11}$，$1 \le B <$ `INT_MAX`，B 是预筛上限。单次时间 $O(B + N/\sqrt{B})$，$B \approx N^{2/3}$ 得 $O(N^{2/3})$（哈希期望）。

内存：`DuJiao` 两张 64 位前缀表约 $16(B + 1)$ 字节，构造时另有 `LinearSieve` 表；$N = 10^{11}$ 取 $B = N^{2/3}$ 约 345 MB，先看内存限制。不同查询增缓存；勿每次重建筛对象。正方形互质对直接 `2 * s.totient_sum(n) - 1`（$n \ge 1$）。一般矩形用 `mertens` 分块时，内层递归使总复杂度不止外层段数。

$$M(n) = 1 - \sum_{i=2}^{n} M(\lfloor n/i \rfloor), \qquad \Phi(n) = \frac{n(n+1)}{2} - \sum_{i=2}^{n} \Phi(\lfloor n/i \rfloor),$$

$M(n) = \sum_{i \le n} \mu(i)$，按等商段乘段长。先提升类型再乘；模下负和要归一化；最后除以 2 时先整数除再取模，合数模下不擅自求 2 的逆元。

3.19 狄利克雷卷积

3.19.1 幂的约数和：不除以 $p-1$

若 $a=\prod p_i^{e_i}$，则 $\sigma(a^b)=\prod_i(1+p_i+\cdots+p_i^{e_ib})$。合数模数或 $p\equiv 1$ 时分母不可逆，不能用几何级数除法公式。令 $G(a,n)=(a^n,\sum_{i=0}^{n-1}a^i)$，$G(a,0)=(1,0)$；由 $h=\lfloor n/2\rfloor$ 的 (P,S)，偶数项得 $(P^2,S(1+P))$，奇数项得 $(P^2a,S(1+P)+P^2)$。时间 $O(\log(n+1))$，任意正模数。参数是项数，约数和须传 e_ib+1；e_ib 可能超过 64 位，模乘在 128 位中做。

3.20 莫比乌斯反演

3.20.1 整除与反演：先辨认求和方向

数论函数定义在**正整数**上；下标 0 只是前缀和占位。

$$(f*g)(n)=\sum_{d\mid n}f(d)g(n/d),\qquad \varepsilon(n)=[n=1],\quad \mathbf{1}(n)=1,\quad \mathrm{id}(n)=n,\qquad \mu*\mathbf{1}=\varepsilon,\quad \varphi*\mathbf{1}=\mathrm{id}\,.$$

3.20.1.1 约数和与倍数和不能混用

约数方向。$F(n)=\sum_{d\mid n}f(d)\Rightarrow f(n)=\sum_{d\mid n}\mu(d)F(n/d)$，不要求 f 积性。
倍数方向。$E(k)$ 仅在 $1\le k\le V$ 非零，$G(d)=\sum_{j\ge 1,\,dj\le V}E(dj)$，则

$$E(k)=\sum_{t=1}^{\lfloor V/k\rfloor}\mu(t)G(kt),$$

或从大到小 $E(k)=G(k)-\sum_{j\ge 2,\,kj\le V}E(kj)$。须有限支持。互质下标对：$c_d=\#\{i:d\mid a_i\}$，答案 $\sum_{d=1}^{V}\mu(d)\binom{c_d}{2}$（重复值按下标区分）。枚举倍数求 c_d，时间 $O(V\log V)$、空间 $O(V)$；V 太大时须改为枚举实际约数。

3.20.1.2 GCD 权值先反演，再交换求和

令 $h=\mu*w$（$w=h*\mathbf{1}$），则

$$\sum_{x=1}^{A}\sum_{y=1}^{B}w(\gcd(x,y))=\sum_{d=1}^{\min(A,B)}h(d)\left\lfloor\frac{A}{d}\right\rfloor\left\lfloor\frac{B}{d}\right\rfloor.$$

互质计数 $h=\mu$，GCD 总和 $h=\varphi$；系数是卷积 $h(d)$，一般不是 $\mu(d)w(d)$。已知 h 的前缀和后按第 3.18.1 节分块。

3.20.2 GCD 计数：缩放、边界与对称性

记 $C(A,B)$ 为 $[1,A]\times[1,B]$ 中的**有序**互质数对数，任一边为 0 时为 0；不含坐标轴，不去掉 $x=y$。

3.20.2.1 恰好等于 k：先缩放整除对象，再做矩形容斥

$k\ge 1$，$a,c\ge 1$，$a\le x\le b$、$c\le y\le d$ 中 $\gcd=k$ 的对数为 $C(A,B)-C(A_0,B)-C(A,B_0)+C(A_0,B_0)$，其中

$$A=\lfloor b/k\rfloor,\quad A_0=\lfloor(a-1)/k\rfloor,\qquad B=\lfloor d/k\rfloor,\quad B_0=\lfloor(c-1)/k\rfloor.$$

先减 1 再除；$k>\min(b,d)$ 时自动为 0。多测预筛到所有查询的 $\max\min(b/k,d/k)$。

3.20.2.2 去掉顺序之前，先检查交换是否仍在定义域内

同一正整数区间 I，T 为 $I\times I$ 中 GCD 为 k 的有序对数，$\delta=[k\in I]$：$\#\{x<y\}=(T-\delta)/2$，$\#\{x\le y\}=(T+\delta)/2$。仅当定义域交换对称时可用；一般矩形不能直接除以 2。不同下标计数回到频率反演。正方形：$C(n,n)=2\Phi(n)-1$，$\Phi(n)=\sum_{i\le n}\varphi(i)$，$n\ge 1$；$n=0$ 为 0。含 0：$\gcd(0,y)=|y|$，轴上仅 $(0,k),(k,0)$ 另行判断；$(0,0)$ 按题面约定；负数处理绝对值。

3.20.3 矩形 GCD 计数：缩放后容斥

前缀矩形互质数对数

$$F(A,B)=\sum_{i=1}^{\min(A,B)}\mu(i)\left\lfloor\frac{A}{i}\right\rfloor\left\lfloor\frac{B}{i}\right\rfloor,$$

按两个商同时分块，右端点 $r=\min(A/\lfloor A/l\rfloor,B/\lfloor B/l\rfloor)$，用 μ 前缀和；只需筛到 $\min(A,B)$。$A=0$ 或 $B=0$ 时为 0。矩形 $[a,b]\times[c,d]$ 内 $\gcd(x,y)=k$ 的对数为

$$\begin{aligned}&F(\lfloor b/k\rfloor,\lfloor d/k\rfloor)-F(\lfloor(a-1)/k\rfloor,\lfloor d/k\rfloor)\\&-F(\lfloor b/k\rfloor,\lfloor(c-1)/k\rfloor)+F(\lfloor(a-1)/k\rfloor,\lfloor(c-1)/k\rfloor),\end{aligned}$$

必须先减 1 再除以 k。答案不超过面积，可存有符号 64 位；分块累加用 128 位。

3.21 狄利克雷双曲线法 & 杜教筛

3.21.1 杜教筛

问题描述：对很大的 n（约 10^{11}），求莫比乌斯函数前缀和 $M(n)=\sum_{i\le n}\mu(i)$ 与欧拉函数前缀和 $\Phi(n)=\sum_{i\le n}\varphi(i)$，支持多次查询。

核心思路：利用狄利克雷卷积 $\mu*1=\varepsilon$、$\varphi*1=\mathrm{id}$ 得 $\sum_{d=1}^{n}M(\lfloor n/d\rfloor)=1$、$\sum_{d=1}^{n}\Phi(\lfloor n/d\rfloor)=n(n+1)/2$。把 $d=1$ 项移到一边，其余 $d\ge 2$ 按 $\lfloor n/d\rfloor$ 相同的区间 $[l,r]$ 整除分块，递归求更小的值。不超过 `limit` 的值直接查线性筛预处理的前缀和 `pmu`、`pphi`，更大的值记忆化在哈希表 `mmu`、`mphi` 中。

代码依赖：LinearSieve（第 20 页，3.4.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：构造预筛上限 1≤B<INT_MAX，内存须可用；mertens(n) 返回 μ 的前缀和，totient_sum(n) 返回 φ 的前缀和（int128）。约定 0≤n≤10^{11}，n=0 返回 0。按整除分块递归并写入 mmu/mphi 缓存；重复查询复用结果，清空缓存可重算，不可修改 pmu/pphi/limit。单次最大参数 N 的常用均摊分析为 O(B+N/$\sqrt{}$B)（哈希按期望常数计算），B 约 N^(2/3) 时为 O(N^(2/3))；小 B 不构成大输入性能保证。多次不同 N 可能新增缓存，空间随实际缓存增长；递归深度 O(log N)。

```
struct DuJiao
{
    using ll = long long;
    using I = __int128_t;
    int limit;
    vector<ll> pmu, pphi;
    unordered_map<ll, ll> mmu;
    unordered_map<ll, I> mphi;

    DuJiao(int limit) : limit(limit), pmu(limit + 1), pphi(limit + 1)
    {
        assert(limit >= 1);
        LinearSieve s(limit);
        for (int i = 1; i <= limit; i++)
        {
            pmu[i] = pmu[i - 1] + s.mu[i];
            pphi[i] = pphi[i - 1] + s.phi[i];
        }
    }

    ll mertens(ll n)
    {
        if (n <= limit) return pmu[n];
        if (mmu.count(n)) return mmu[n];
        ll ans = 1;
```

```
        for (ll l = 2, r; l <= n; l = r + 1)
        {
            r = n / (n / l);
            ans -= (r - l + 1) * mertens(n / l);
        }
        return mmu[n] = ans;
    }

    I totient_sum(ll n)
    {
        if (n <= limit) return pphi[n];
        if (mphi.count(n)) return mphi[n];
        I ans = I(n) * (n + 1) / 2;
        for (ll l = 2, r; l <= n; l = r + 1)
        {
            r = n / (n / l);
            ans -= I(r - l + 1) * totient_sum(n / l);
        }
        return mphi[n] = ans;
    }
};
```

模板题使用：Luogu P4213

题意：多次查询 φ 与 μ 的前缀和，先输出 φ，再输出 μ。先读入全部 n，预筛到 min(最大 n,2000000)，同一结构复用缓存。题目 n<2^31，所以 φ 的总和可安全转为 long long；模板更大范围的 int128 返回值不能照搬此转换。

所需模板：LinearSieve（第 20 页，3.4.2 节）、DuJiao（第 54 页，3.21.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4213

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t;
    cin >> t;
    vector<long long> a(t);
    for (auto &n : a) cin >> n;
    int b = min(2000000LL, *max_element(a.begin(), a.end()));
    DuJiao d(b);
    for (long long n : a)
        cout << (long long)d.totient_sum(n) << ' ' << d.mertens(n) << '\n';
}
```

3.21.2 杜教筛：预筛与缓存都不可省略

整除反演建模（第 3.20.1 节，第 53 页）的两条卷积恒等式给出 DuJiao 的 mertens 与 totient_sum 递推：商相同的一段合并，系数为区间长度；超过预筛上限的结果必须写回缓存。最大查询 N、预筛上限 B 时单次 $O(B+N/\sqrt{B})$，取 $B\approx N^{2/3}$ 得 $O(N^{2/3})$；多个不同查询会继续增加缓存。范围 $0\le n\le 10^{11}$，欧拉函数前缀和用 128 位。取模时负值先求余，仅余数为负时加模数。

3.22 Min_25 筛

3.22.1 Min_25 积性函数求和

问题描述：给定积性函数 f：$f(p)$ 是素数 p 的至多二次多项式，$f(p^e)$ 由回调给出。求 $\sum_{i=1}^{n} f(i)$ 模素数，n 可达 10^{12}。

核心思路：所有 $\lfloor n/l \rfloor$ 只有 $O(\sqrt{n})$ 种，降序存于 `w`，经 `id` 用 `small`、`large` 两张表定位。第一阶段对每个 $x \in w$ 求素数幂和 $g(x,d)=\sum_{p\le x} p^d$ $(d=0,1,2)$：初值为 $\sum_{i=2}^{x} i^d$，依次对素数 $p \le \sqrt{n}$ 去掉最小质因子为 p 的合数。第二阶段 `dfs(x,k)` 求 $2..x$ 中最小质因子不小于第 k 个素数的数的 f 之和：素数部分由 g 的线性组合减去前 k 个素数的和，合数部分枚举最小质因子及其指数递归。

思路中的情况	代码中的处理	行
枚举所有整除商	$x=\lfloor n/l \rfloor$ 入 `w`，按 $x \le s$ 与否记入 `small[x]` 或 `large[n/x]`	21
g 的初值	把 $2..x$ 都当作素数：$\sum_{i=2}^{x} i^0, i^1, i^2$	32
判断 p 是否为素数	此时 $g(p,0)$ 已是精确的素数个数，与 $g(p-1,0)$ 相等说明 p 是合数	38
筛去最小质因子为 p 的合数	对 $x \ge p^2$：$g(x)$ $-=$ $p^d\,(g(x/p)-\sum_{q<p} q^d)$，`prefix` 是较小素数的幂和	45
`dfs` 的素数部分	$c_0g_0+c_1g_1+c_2g_2$ 减去前 k 个素数的 $f(p)$ 之和	65
`dfs` 的合数部分	枚举 p^e：加 $f(p^e)\cdot$`dfs`$(x/p^e, i+1)$ 与 $f(p^{e+1})$	70
加上 $f(1)$	`dfs` 只统计 ≥ 2 的数，最后补 1	75

代码依赖：ModInt（第 12 页，3.1.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：构造 Min25<mod>(n)，1≤n≤10^12，mod 为大于 3 的素数。sum(c, power) 求 ∑(1≤i≤n) f(i)，要求 f 为积性函数、f(1)=1，f(p)=c[0]+c[1]p+c[2]p²；power(p,e,pe) 返回 f(p^e)，其中 pe=p^e、e≥1，回调必须与一次幂系数一致。支持同一对象多次 sum；不改筛表。预处理 O(n^0.75 / log n)，回调 O(1) 时递归求和保守 O(n) 上界；O(√n) 空间、O(log n) 递归栈。w 降序存不同整除商，g[id(x)][d] 是模意义下 ∑(p≤x) p^d；id 仅允许查询这些整除商（包含 1..√n），各字段只读。依赖 ModInt；普通递归版，未承诺全部官方时限达标。

```cpp
template <int mod> struct Min25
{
    using ll = long long;
    using Z = ModInt<mod>;
    ll n;
    int s;
    vector<ll> w;
    vector<int> small, large, primes;
    vector<array<Z, 3>> g;

    Min25(ll n) : n(n)
    {
        static_assert(mod > 3);
        assert(1 <= n && n <= 1000000000000LL);
        s = sqrtl(n);
        while (1LL * (s + 1) * (s + 1) <= n) s++;
        while (1LL * s * s > n) s--;
        small.resize(s + 1);
        large.resize(s + 1);
        Z half = Z(2).inv(), sixth = Z(6).inv();
        for (ll l = 1, r; l <= n; l = r + 1)
        {
            ll x = n / l;
            r = n / x;
            int k = w.size();
            if (x <= s)
                small[x] = k;
            else
                large[n / x] = k;
            w.push_back(x);
            Z v = x;
            g.push_back(
                {v - 1, v * (v + 1) * half - 1, v * (v + 1) * (v * 2 + 1) * sixth - 1});
        }
```

```
        array<Z, 3> prefix{};
        for (int p = 2; p <= s; p++)
        {
            if (g[id(p)][0].v == g[id(p - 1)][0].v) continue;
            primes.push_back(p);
            array<Z, 3> weight{1, p, Z(p) * p};
            for (int j = 0; j < (int)w.size() && 1LL * p * p <= w[j]; j++)
            {
                int k = id(w[j] / p);
                for (int d = 0; d < 3; d++)
                    g[j][d] = g[j][d] - weight[d] * (g[k][d] - prefix[d]);
            }
            for (int d = 0; d < 3; d++) prefix[d] = prefix[d] + weight[d];
        }
    }

    int id(ll x) const { return x <= s ? small[x] : large[n / x]; }

    template <class F> Z sum(array<Z, 3> c, F power) const
    {
        vector<Z> prefix(primes.size() + 1);
        for (int i = 0; i < (int)primes.size(); i++)
        {
            Z p = primes[i];
            prefix[i + 1] = prefix[i] + c[0] + c[1] * p + c[2] * p * p;
        }
        auto dfs = [&](auto &&self, ll x, int k) -> Z
        {
            if (x < 2 || (k && x <= primes[k - 1])) return 0;
            auto a = g[id(x)];
            Z ans = c[0] * a[0] + c[1] * a[1] + c[2] * a[2] - prefix[k];
            for (int i = k; i < (int)primes.size() && primes[i] <= x / primes[i]; i++)
            {
                ll p = primes[i], pe = p;
                for (int e = 1; pe <= x / p; e++, pe *= p)
                    ans = ans + power(p, e, pe) * self(self, x / pe, i + 1) +
                          power(p, e + 1, pe * p);
            }
            return ans;
        };
        return dfs(dfs, n, 0) + 1;
    }
};
```

模板题使用：Luogu P5325

题意：求积性函数前缀和，f(p^e)=p^e(p^e-1)，模 1000000007。一次幂为 p^2-p，故系数传 {0,-1,1}；回调的 pe 已是整数 p^e，无需再快速幂。

所需模板：ModInt（第 12 页，3.1.2 节）、Min25（第 56 页，3.22.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5325

```
int main()
{
    using Z = ModInt<1000000007>;
    long long n;
    cin >> n;
    Min25<1000000007> sieve(n);
    auto power = [](long long p, int e, long long pe)
    {
        Z x = pe;
        return x * (x - 1);
    };
    cout << sieve.sum({0, -1, 1}, power).v << '\n';
}
```

3.23 类欧几里德算法

3.23.1 有符号整除和：类欧几里德

问题描述：给定 n, m, a, b（a, b 可以为负），求 $\sum_{i=0}^{n-1} \lfloor (ai+b)/m \rfloor$。

核心思路：先把 a, b 按数学向下取整规范到 $[0, m)$，分离出的商 q_a, q_b 贡献 $q_a \cdot n(n-1)/2 + q_b \cdot n$。之后循环（类欧几里得）：每轮先把 a, b 中不小于 m 的整商部分直接累加并取余；若 $y = an + b < m$，剩下各项全为 0，结束。否则把和看成直线 $(ax+b)/m$ 下方的格点数并交换坐标轴，问题化为项数 $\lfloor y/m \rfloor$、常数 $y \bmod m$、并交换 a 与 m 的同形和式，(a, m) 像欧几里得算法一样迅速缩小。

接口契约：floor_sum(n,m,a,b) 返回 i=0..n-1 的 floor((ai+b)/m) 之和，结果为 int128。要求 0≤n≤10^9、1≤m≤10^9，a、b 支持完整 long long；n=0 返回 0。先按数学向下取整规范化负系数，再迭代交换分母和余数；不能把 C++ 负数截零除法当作 floor。O(log m) 时间、O(1) 空间，不依赖完整数论模块。

```
__int128_t floor_sum(long long n, long long m, long long a, long long b)
{
    using ll = long long;
    using i128 = __int128_t;
    assert(n >= 0 && m > 0);
    i128 ans = 0;
    auto norm = [&](ll &v)
    {
        ll q = v / m;
        if (v % m < 0) --q;
        v = (ll)(i128(v) - i128(q) * m);
        return q;
    };
    ans += i128(norm(a)) * n * (n - 1) / 2;
    ans += i128(norm(b)) * n;
    while (true)
    {
        ans += i128(n) * (n - 1) / 2 * (a / m);
        a %= m;
        ans += i128(n) * (b / m);
        b %= m;
        i128 y = i128(a) * n + b;
        if (y < m) break;
        n = (ll)(y / m);
        b = (ll)(y % m);
        swap(a, m);
    }
    return ans;
}
```

模板题使用：Library Checker sum_of_floor_of_linear

题意：求 i=0..n-1 的 floor((a*i+b)/m) 之和。此题有 0≤a,b<m，因此结果可转为 long long 输出；更一般的有符号输入不能照搬该转换。

所需模板：floor_sum（第 60 页，3.23.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/sum_of_floor_of_linear

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t;
    cin >> t;
    while (t--)
    {
        long long n, m, a, b;
        cin >> n >> m >> a >> b;
        // This task has 0 <= a,b < m, so the sum is at most n*(n-1)/2.
        cout << (long long)floor_sum(n, m, a, b) << '\n';
    }
}
```

3.23.2 整除和、带权和与平方和

问题描述： 对 $i=0..n-1$ 令 $y_i=\lfloor(ai+b)/m\rfloor$，同时求 $\sum y_i$、$\sum i\,y_i$、$\sum y_i^2$（模 998244353）。

核心思路： 先把 a,b 规范到 $[0,m)$，商为 x,y，则 $y_i=xi+y+z_i$，展开后只需 z_i 的三种和 f,g,h 及 $\sum i$、$\sum i^2$（s1、s2）。求 f,g,h 时令 t 为最大值 height，交换求和次序：$z_i\ge j$ 当且仅当 $i\ge c_j$，而 $c_{j+1}=\lfloor(mj+m+a-b-1)/a\rfloor$ 仍是同形的整除式，于是以参数 $(t,a,m,m+a-b-1)$ 递归（a 与 m 互换，类欧几里得）得到 $u=\sum c$、$v=\sum j\,c$、$w=\sum c^2$，再由 $f=nt-u$、$g=t\,s_1-(w-u)/2$、$h=nt^2-2v-u$ 还原。

代码依赖：ModInt（第 12 页，3.1.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约： floor_moments(n,m,a,b) 对 i=0..n-1，令 y_i=floor((a*i+b)/m)，返回 {∑y_i, ∑i*y_i, ∑y_i²}，元素为 ModInt<998244353>，取整数用.v。0≤n≤1000000001，1≤m≤10^9；a、b 支持完整 long long。n=0 返回全零。时间与递归空间 O(log(m+1))；模数固定，499122177 与 166374059 分别是 2、6 的逆元。P5170 输入上限为闭区间，驱动传 n+1，输出索引顺序 0、2、1。

```
array<ModInt<998244353>, 3>
floor_moments(long long n, long long m, long long a, long long b)
{
    using Z = ModInt<998244353>;
    assert(n >= 0 && n <= 1000000001 && m >= 1 && m <= 1000000000);
    if (!n) return {0, 0, 0};
    auto normalize = [&](long long &v)
    {
        long long q = v / m;
        if (v % m < 0) q--;
        v = (__int128_t(v) - __int128_t(q) * m);
        return Z(q);
    };
    Z x = normalize(a);
    Z y = normalize(b);
    Z count = n;
    Z s1 = count * Z(n - 1) * Z(499122177);
    Z s2 = count * Z(n - 1) * Z(2 * n - 1) * Z(166374059);
    Z f = 0, g = 0, h = 0;
    if (a)
    {
        long long height = (a * (n - 1) + b) / m;
        auto [u, v, w] = floor_moments(height, a, m, m + a - b - 1);
        Z t = height;
        f = count * t - u;
        g = t * s1 - (w - u) * Z(499122177);
        h = count * t * t - Z(2) * v - u;
    }
    return {f + x * s1 + y * count,
            g + x * s2 + y * s1,
            h + x * x * s2 + y * y * count + Z(2) * x * y * s1 + Z(2) * x * g +
                Z(2) * y * f};
}
```

模板题使用：Luogu P5170

题意：求 i=0..n 的整除和、整除值平方和、i 乘整除值之和，模 998244353。t 至 10^5，n/a/b/c 至 10^9 且 c>0。核心参数顺序 (n+1,c,a,b)，返回顺序是和/带权和/平方和；本题输出索引 0、2、1，不能只核对第一项。

所需模板：ModInt（第 12 页，3.1.2 节）、floor_moments（第 62 页，3.23.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5170

```
int main()
{
    int t;
    scanf("%d", &t);
    while (t--)
    {
        long long n, a, b, c;
        scanf("%lld%lld%lld%lld", &n, &a, &b, &c);
        auto result = floor_moments(n + 1, c, a, b);
        printf("%d %d %d\n", result[0].v, result[2].v, result[1].v);
    }
    return 0;
}
```

3.23.3 类欧几里德：有符号整除求和

接口见第 3.23.1 节（第 60 页）。`floor_sum(n,m,a,b)` 返回

$$F(n,m,a,b)=\sum_{i=0}^{n-1}\left\lfloor\frac{ai+b}{m}\right\rfloor.$$

要求 $0\le n\le 10^9$、$1\le m\le 10^9$，a,b 为完整有符号 64 位，返回 128 位。n 是项数，下标 $0\dots N$ 时传 $N+1$。令 $a=q_am+r_a$、$b=q_bm+r_b$（余数在 $[0,m)$，负数商须向下取整），先加 $q_an(n-1)/2+q_bn$。对 $0\le a,b<m$，设 $y=an+b$；$y<m$ 时余下为 0，否则

$$F(n,m,a,b)=F(\lfloor y/m\rfloor,a,m,y \bmod m).$$

时间 $O(\log(m+1))$，额外空间 $O(1)$。

3.23.4 类欧几里德的三个求和量

实现见第 3.23.2 节（第 62 页）。令 $y_i=\lfloor(ai+b)/m\rfloor$，$0\le i<n$，求 $f=\sum y_i$、$g=\sum iy_i$、$h=\sum y_i^2$。范围 $0\le n\le 1000000001$、$1\le m\le 10^9$，a,b 为完整有符号 64 位；$n=0$ 返回 $(0,0,0)$。结果固定模 998244353，换模数须同时换 2、6 的逆元。

归一化 $a=xm+a'$、$b=ym+b'$（向下取整），规范化问题返回 (f_0,g_0,h_0)，$S_1=\sum i$、$S_2=\sum i^2$：

$$\begin{aligned}f&=f_0+xS_1+yn,\\ g&=g_0+xS_2+yS_1,\\ h&=h_0+x^2S_2+y^2n+2xyS_1+2xg_0+2yf_0.\end{aligned}$$

$a'=0$ 时剩余量为 0。$0<a'<m$ 时令 $t=\lfloor(a'(n-1)+b')/m\rfloor$，递归 $(t,a',m,m+a'-b'-1)$ 得 (u,v,w)：

$$f_0=nt-u,\qquad g_0=tS_1-(w-u)/2,\qquad h_0=nt^2-2v-u.$$

递归 $O(\log(m+1))$ 层。P5170 上界含 n，传 $n+1$，输出顺序为 f,h,g。

第四章　多项式与生成函数

4.1　快速傅里叶变换

4.1.1　复数 FFT（可复用单位根）

功能：对长度为 n（二次幂）的复数数组做原地离散傅里叶变换及逆变换；单位根表在构造时一次算好，可供多次变换复用，是浮点多项式乘法、拆系数任意模卷积等的底层工具。

核心思路：构造时直接按角度用 $\cos,\sin$ 算出 $\omega^i = e^{2\pi \mathrm{i}\, i/n}$ $(i < n/2)$，不靠连乘，避免误差累积。`transform` 先做位逆序重排（`j` 是 `i` 的二进制反转，增量维护），再自底向上合并：两段长度为 `half` 的子结果用单位根 `roots[j*step]` 做蝴蝶 $u \pm \omega v$。逆变换把单位根取共轭，最后整体除以 n。

接口契约：ComplexFFT(n) 预计算单位根，n 为 1..2^22 的二次幂。transform(a,inverse=false) 原地正变换，正向指数为正；inverse=true 用共轭根并除以 n。a 长度须等于 n；C=complex<long double>。根直接用角度计算，不沿整层连乘。单次变换 O(n log(n+1))，根表 O(n)，可复用。浮点环境须二进制至少 53 位、最近舍入、禁用 fast-math；整数卷积还要求单位根绝对误差 $\le 8\times 2^{-53}$，本机全根已用百位参考核验。变换本身是近似值。

```
struct ComplexFFT
{
    using Real = long double;
    using C = complex<Real>;
    static constexpr int max_size = 1 << 22;
    int n;
    vector<C> roots;

    explicit ComplexFFT(int size) : n(size)
    {
        static_assert(numeric_limits<Real>::radix == 2);
        static_assert(numeric_limits<Real>::digits >= 53);
        assert(n > 0 && n <= max_size && (n & (n - 1)) == 0);
        roots.resize(n / 2);
        Real pi = acosl(-1);
        for (int i = 0; i < n / 2; i++)
        {
            Real angle = pi * (2.0L * i / n);
            roots[i] = C(cosl(angle), sinl(angle));
        }
    }
```

正逆变换（接上页同一结构体）：

```
    void transform(vector<C> &a, bool inverse = false) const
    {
        assert(a.size() == (unsigned)n);
        for (int i = 1, j = 0; i < n; i++)
        {
            int bit = n >> 1;
            for (; j & bit; bit >>= 1) j ^= bit;
            j ^= bit;
            if (i < j) swap(a[i], a[j]);
        }
        for (int half = 1; half < n; half *= 2)
        {
            int step = n / (2 * half);
            for (int i = 0; i < n; i += 2 * half)
            {
                for (int j = 0; j < half; j++)
                {
                    C w = roots[j * step];
                    if (inverse) w = conj(w);
                    C u = a[i + j], v = a[i + j + half] * w;
                    a[i + j] = u + v;
                    a[i + j + half] = u - v;
                }
            }
        }
        if (inverse)
            for (auto &x : a) x /= n;
    }
};
```

模板题使用：Luogu P3803 【模板】多项式乘法（FFT）—直接变换接口

题意：给出次数 0≤n,m≤1000000、系数均为 0..9 整数的两个多项式，按升幂读入并输出乘积的 n+m+1 项。补零到不小于 n+m+1 的最小二次幂 L（至多 2^21），显式构造 ComplexFFT，分别正变换、逐点相乘、逆变换后 llroundl 取整。保留本节的二进制至少 53 位、最近舍入、单位根误差界及禁用 fast-math 前提；本题范数乘积至多 81000081。时间 O(L log L)，空间 O(L)。这是底层变换接口的完整用法，卷积包装器和精确 CRT 用法另列。

所需模板：ComplexFFT（第 66 页，4.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3803

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    int size = 1;
    while (size < n + m + 1) size *= 2;
    auto fft = ComplexFFT(size);
    vector<ComplexFFT::C> a(size), b(size);
    for (int i = 0, x; i <= n; i++) cin >> x, a[i] = x;
    for (int i = 0, x; i <= m; i++) cin >> x, b[i] = x;
    fft.transform(a);
    fft.transform(b);
    for (int i = 0; i < size; i++) a[i] *= b[i];
    fft.transform(a, true);
    for (int i = 0; i <= n + m; i++) cout << llroundl(a[i].real()) << ' ';
    cout << '\n';
}
```

4.1.2 拆系数 FFT 任意模卷积

功能：计算两个整数多项式乘积的系数模任意给定整数 M，即 $c_k = \sum_{i+j=k} a_i b_j \bmod M$；$M$ 可以是合数或不支持 NTT 的素数。

核心思路：每个系数先归一化到 $[0, M)$，再拆成三个 11 位块 $v = v_0 + v_1 B + v_2 B^2$（$B = 2^{11}$），这样任意两块卷积的真实系数都足够小，浮点 FFT 后四舍五入即可精确还原。对 a, b 的三个块各做一次正变换（共六次），再对每对块 (i, j) 逐点相乘、逆变换、`llroundl` 取整，乘上 $B^{i+j} \bmod M$（预存于 `weight`）累加进结果，共九次逆变换。

代码依赖：ComplexFFT（第 66 页，4.1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：convolution_mod_fft(a,b,mod) 接受 vector<int>，返回 [0,mod) 系数；1≤mod≤INT_MAX，允许合数与负输入，mod=1 返回零。空输入返回空，非空结果长度 ≤2^19。依赖 ComplexFFT 及其浮点前提；归一化后拆成三个 11 位块，六次正变换和九次逆变换，分别舍入后模合并。每路误差界小于 0.079；不依赖更宽的 long double。时间 O(N log N)，空间 O(N)，常数高于打包版本以保持数值界与代码清楚；覆盖 P4245。

```
// Three 11-bit limbs; same floating-point assumptions as convolution_fft.
vector<int> convolution_mod_fft(const vector<int> &a, const vector<int> &b, int mod)
{
    assert(mod >= 1);
    if (a.empty() || b.empty()) return {};
    assert(a.size() + b.size() - 1 <= (1 << 19));
    int size = a.size() + b.size() - 1, n = 1;
    while (n < size) n *= 2;
    vector<int> result(size);
    if (mod == 1) return result;
```

```
    ComplexFFT fft(n);
    using C = ComplexFFT::C;
    const int base = 1 << 11;
    vector<vector<C>> x(3, vector<C>(n)), y = x;
    for (int i = 0; i < (int)a.size(); i++)
    {
        unsigned v = (a[i] % mod + (long long)mod) % mod;
        for (int j = 0; j < 3; j++) x[j][i] = (v >> (11 * j)) & (base - 1);
    }
    for (int i = 0; i < (int)b.size(); i++)
    {
        unsigned v = (b[i] % mod + (long long)mod) % mod;
        for (int j = 0; j < 3; j++) y[j][i] = (v >> (11 * j)) & (base - 1);
    }
    for (int j = 0; j < 3; j++)
    {
        fft.transform(x[j]);
        fft.transform(y[j]);
    }
    long long weight[5] = {1};
    for (int i = 1; i < 5; i++) weight[i] = weight[i - 1] * base % mod;
    vector<C> z(n);
    for (int i = 0; i < 3; i++)
    {
        for (int j = 0; j < 3; j++)
        {
            for (int k = 0; k < n; k++) z[k] = x[i][k] * y[j][k];
            fft.transform(z, true);
            for (int k = 0; k < size; k++)
            {
                long long v = llroundl(z[k].real()) % mod;
                result[k] = (result[k] + v * weight[i + j]) % mod;
            }
        }
    }
    return result;
}
```

模板题使用：Luogu P4245

题意：输入两个多项式的次数、任意模数与升幂系数，输出模乘积。题目次数不超过十万、模数最多 1000000009；函数会先归一化再拆系数，不要求模数支持 NTT。这里使用三块浮点 FFT，并遵守其单位根与舍入前提。

所需模板：ComplexFFT（第 66 页，4.1.1 节）、convolution_mod_fft（第 68 页，4.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4245

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, mod;
    cin >> n >> m >> mod;
    vector<int> a(n + 1), b(m + 1);
    for (int &x : a) cin >> x;
    for (int &x : b) cin >> x;
    auto c = convolution_mod_fft(a, b, mod);
    for (int x : c) cout << x << ' ';
    cout << '\n';
}
```

相关应用：CDQ 分治配合 NTT 的卷积递推，附件标题为分治 FFT；算法不做浮点变换。

4.1.3 CDQ 分治卷积递推

问题描述：求满足半在线卷积递推 $f_i = \text{init}_i + \sum_{j<i} f_j\, g_{i-j}$ 的前 n 项（核满足 $g_0 = 0$，模 NTT 素数）。f_i 依赖自身更早的项，不能直接做一次卷积。

核心思路：在区间 $[l, r)$ 上分治：先递归解出左半 $[l, m)$，此时左半的 f 已全部确定；把它与核 $g_0, \ldots, g_{r-l-1}$ 做一次 NTT 乘法，乘积第 $i - l$ 项就是左半对右半位置 $i \in [m, r)$ 的贡献，累加进 `initial[i]`；再递归右半。区间长度为 1 时该位置的所有贡献都已加完（$g_0 = 0$，无自身项），即为最终值。数组 `initial` 原地存放结果。

代码依赖：ModInt（第 12 页，3.1.2 节）、NttConvolution（第 73 页，4.2.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：cdq_convolution<mod=998244353,primitive=3>(kernel,initial) 求 f[i]=initial[i]+∑(j<i)f[j]kernel[i-j]，返回前 n 项。两数组为等长 vector<ModInt<mod»，kernel[0] 必须为 0；空输入返回空。模数须为 int 范围素数，primitive 是原根；依赖 NttConvolution。非空时要求 n+floor(n/2)-1≤NTT::max_size，保证每层完整卷积可变换。初始项可任意，并非只有 f[0]=1；左半区解定后，将其到右半区的卷积贡献加上，再递归右半区。O(n $\log^2$(n+1)) 时间、O(n) 额外空间，递归深度 O(log n)。本函数是静态线性递推，不提供流式追加或非线性叶子回调。P4721 使用 initial[0]=1、其他项 0。

```
// f[i] = initial[i] + sum_{j<i} f[j] * kernel[i-j]; kernel[0] must be zero.
template <int mod = 998244353, int primitive = 3>
vector<ModInt<mod>> cdq_convolution(const vector<ModInt<mod>> &kernel,
                                    vector<ModInt<mod>> initial)
{
    using Ntt = NttConvolution<mod, primitive>;
    using Poly = typename Ntt::Poly;
    assert(kernel.size() == initial.size());
    if (initial.empty()) return {};
    assert(kernel[0].v == 0);
    assert(initial.size() + initial.size() / 2 - 1 <= Ntt::max_size);
    int n = initial.size();
    auto solve = [&](auto &&self, int l, int r) -> void
    {
        if (r - l == 1) return;
        int m = (l + r) / 2;
        self(self, l, m);
        Poly left(initial.begin() + l, initial.begin() + m);
        Poly right(kernel.begin(), kernel.begin() + r - l);
        Poly product = Ntt::multiply(move(left), move(right));
        for (int i = m; i < r; i++) initial[i] += product[i - l];
        self(self, m, r);
    };
    solve(solve, 0, n);
    return initial;
}
```

模板题使用：Luogu P4721 分治 FFT

题意：输入 g[1..n-1]，计算 f[0]=1、f[i]=∑(j=1..i)f[i-j]g[j]，模 998244353。kernel[0] 留零，initial 仅第零项为 1；调用的是真正 CDQ 分治卷积，依赖 NTT 与模整数。

所需模板：ModInt（第 12 页，3.1.2 节）、NttConvolution（第 73 页，4.2.1 节）、cdq_convolution（第 71 页，4.1.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4721

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    using Z = ModInt<998244353>;
    vector<Z> g(n), b(n);
    for (int i = 1; i < n; i++)
    {
        int x;
        cin >> x;
        g[i] = x;
    }
    b[0] = 1;
    auto f = cdq_convolution(g, b);
    for (int i = 0; i < n; i++) cout << f[i].v << " \n"[i + 1 == n];
}
```

4.1.4 浮点 FFT：舍入前先约束最坏误差

前提：53 位有效精度、最近舍入、禁用 fast-math，单位根绝对误差不超过 $8u$（$u = 2^{-53}$）。长度 $N = 2^{\ell}$、两输入欧氏范数积 S 时卷积误差界

$$S\left((1+u)^{3\ell}(1+\sqrt{5}u)^{3\ell+1}(1+8u)^{3\ell}-1\right).$$

整数入口要求 $N \le 2^{22}$、$\max|a_i|\max|b_i|\sqrt{|a||b|} \le 2\cdot 10^{12}$，误差 < 0.166，用 llroundl 舍入；超限不保证正确。任意模卷积拆三个 11 位块，结果长度 $\le 2^{19}$，误差 < 0.079；模数可为 1 或合数，空输入返回空。

4.2 快速数论变换

4.2.1 可指定素数与原根的 NTT 卷积

功能：给定两个模素数 p 的多项式（系数数组）a, b，求乘积 $c_k = \sum_{i+j=k} a_i b_j \bmod p$；同时提供原地数论变换 `ntt` 供其他多项式算法复用。

核心思路：取 $p-1$ 的二次幂因子中的长度 n 与 n 次单位根 $\omega = g^{(p-1)/n}$，把系数表示变成 n 个点值：位逆序重排后自底向上做蝴蝶合并。两个点值逐项相乘后做逆变换（同一正变换后翻转下标 $1..n-1$ 并乘 n^{-1}）还原系数。单位根表按层缓存，各长度复用。

代码依赖：ModInt（第 12 页，3.1.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：声明 NttConvolution<mod, primitive>，primitive 默认 3。mod 必须为 int 范围内的素数，primitive 是模 mod 原根；不在接口内验证素性或阶。max_size 是 mod-1 的最大二次幂因子。ntt(a,invert) 原地变换，长度须为正二次幂且不超过 max_size；invert=true 为逆变换。multiply(a,b) 返回普通卷积，任一为空则为空；非空时长度和减一不超过 max_size。元素类型 Z，数组类型 Poly。O(n log n) 时间；每种 mod/primitive 实例缓存分层单位根，空间 O(历次最大变换长度)，同一实例调用时不并发扩容。输入输出均为自然顺序，逆变换复用正变换后翻转下标 1..n-1 并乘 n 的逆元。998244353 和 167772161 均可用原根 3。

```
template <int mod, int primitive = 3> struct NttConvolution
{
    static_assert(mod >= 2 && primitive >= 1 && primitive < mod);
    static constexpr int max_size = (mod - 1) & -(mod - 1);
    using Z = ModInt<mod>;
    using Poly = vector<Z>;

    // mod is prime, primitive is a primitive root modulo mod.
    static void ntt(Poly &a, bool invert = false)
    {
        int n = a.size();
        assert(n > 0 && (n & (n - 1)) == 0 && n <= max_size);
        static Poly roots{0, 1};
        for (int half = roots.size(); half < n; half *= 2)
        {
            Z step = Z(primitive).pow((mod - 1) / (2 * half));
            roots.resize(2 * half);
            for (int i = half / 2; i < half; i++)
            {
                roots[2 * i] = roots[i];
                roots[2 * i + 1] = roots[i] * step;
            }
        }
        for (int i = 1, j = 0; i < n; i++)
        {
            int bit = n >> 1;
            for (; j & bit; bit >>= 1)
                j ^= bit;
            j ^= bit;
            if (i < j)
                swap(a[i], a[j]);
        }
```

蝶形变换与卷积（接上页同一结构体与函数）：

```
        for (int half = 1; half < n; half *= 2)
        {
            for (int i = 0; i < n; i += 2 * half)
            {
                for (int j = 0; j < half; j++)
                {
                    Z u = a[i + j];
                    Z v = a[i + j + half] * roots[half + j];
                    a[i + j] = u + v;
                    a[i + j + half] = u - v;
                }
            }
        }
        if (invert)
        {
            reverse(a.begin() + 1, a.end());
            Z inverse = Z(n).inv();
            for (auto &x : a)
                x *= inverse;
        }
    }

    static Poly multiply(Poly a, Poly b)
    {
        if (a.empty() || b.empty())
            return {};
        assert(a.size() + b.size() - 1 <= max_size);
        int size = a.size() + b.size() - 1, n = 1;
        while (n < size)
            n *= 2;
        a.resize(n);
        b.resize(n);
        ntt(a);
        ntt(b);
        for (int i = 0; i < n; i++)
            a[i] *= b[i];
        ntt(a, true);
        a.resize(size);
        return a;
    }
};
```

模板题使用：Library Checker convolution_mod

题意：求两个多项式的卷积，模 998244353。输入为各自系数个数和升幂系数；输出 n+m-1 个系数。题目保证输入系数已经取模。

所需模板：ModInt（第 12 页，3.1.2 节）、NttConvolution（第 73 页，4.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/convolution_mod

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    using C = NttConvolution<998244353>;
    int n, m;
    cin >> n >> m;
    C::Poly a(n), b(m);
    for (auto &x : a) cin >> x.v;
    for (auto &x : b) cin >> x.v;
    auto c = C::multiply(move(a), move(b));
    for (int i = 0; i < (int)c.size(); i++)
        cout << c[i].v << (i + 1 == (int)c.size() ? '\n' : ' ');
}
```

4.2.2 精确有符号整数卷积

功能：计算两个 `long long` 整数系数（可为负）多项式乘积的精确系数，要求结果系数本身落在有符号 64 位范围内。

核心思路：用三个 NTT 友好素数 p, q, r（乘积约 2^{86}）分别做模卷积（`convolution_residue`），再用 Garner 算法合并：先由模 p、q 的余数求出 $v \equiv c \pmod{pq}$，再求 u 使 $v + pq\,u \equiv c \pmod{r}$，得到 $[0, pqr)$ 中的唯一代表。它超过 $pqr/2$ 时减去 pqr 取中心代表，从而还原负系数。Garner 所需的两个逆元只计算一次。

代码依赖：ModInt（第 12 页，3.1.2 节）、NttConvolution（第 73 页，4.2.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：convolution_i64(a,b) 返回 signed64 整数卷积，空输入返回空。非空结果长度 ≤2^24，所有真实系数必须在 signed64 范围内。依赖 NttConvolution，三个素数模卷积后用 int128 CRT 重构中心代表。断言不能检测任意越界结果的模数别名。O(L log L) 时间、O(L) 空间，L 为补齐的二次幂长度。负数与极值已通过 cpp_int 参考验证；P3803 最大次数驱动本地通过，线上性能待验证。

```
template <int p>
vector<int> convolution_residue(const vector<long long> &a, const vector<long long> &b)
{
    using N = NttConvolution<p>;
    typename N::Poly x(a.begin(), a.end()), y(b.begin(), b.end());
    auto c = N::multiply(move(x), move(y));
    vector<int> r(c.size());
    for (int i = 0; i < (int)c.size(); i++) r[i] = c[i].v;
    return r;
}

vector<long long> convolution_i64(const vector<long long> &a,
                                  const vector<long long> &b)
{
    if (a.empty() || b.empty()) return {};
    assert(a.size() <= (1 << 24) && b.size() <= (1 << 24));
    assert(a.size() + b.size() - 1 <= (1 << 24));
    const long long p = 167772161, q = 469762049, r = 1224736769;
    const long long pq = p * q;
    const __int128 mod = (__int128)pq * r;
    static const long long ip = ModInt<q>(p).inv().v;
    static const long long ipq = ModInt<r>(pq).inv().v;
    auto x = convolution_residue<p>(a, b);
    auto y = convolution_residue<q>(a, b);
    auto z = convolution_residue<r>(a, b);
    vector<long long> c(x.size());
    for (int i = 0; i < (int)c.size(); i++)
    {
        long long t = (y[i] - (long long)x[i] + q) % q * ip % q;
        long long v = x[i] + p * t;
        long long u = (z[i] - v % r + r) % r * ipq % r;
        __int128 value = v + (__int128)pq * u;
        if (value > mod / 2) value -= mod;
        assert(LLONG_MIN <= value && value <= LLONG_MAX);
        c[i] = (long long)value;
    }
    return c;
}
```

模板题使用：Luogu P3803

题意：求两个整数多项式的乘积，系数按升幂输入。首行 n、m 是次数，系数个数分别为 n+1、m+1；输出精确整数，不能取模。本题系数在 0..9，满足 signed64 结果范围。

所需模板：ModInt（第 12 页，3.1.2 节）、NttConvolution（第 73 页，4.2.1 节）、convolution_i64（第 76 页，4.2.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3803

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<long long> a(n + 1), b(m + 1);
    for (auto &x : a) cin >> x;
    for (auto &x : b) cin >> x;
    auto c = convolution_i64(a, b);
    for (int i = 0; i < (int)c.size(); i++)
        cout << c[i] << (i + 1 == (int)c.size() ? '\n' : ' ');
}
```

4.2.3 参数化 NTT 与模数适配

实现见第 73 页。传入素数 p 与原根 g；长度 n 为二次幂且整除 $p-1$，上限 $L = 2^{v_2(p-1)}$。普通卷积补齐到不小于长度和减一的二次幂。998244353 的 $L = 2^{23}$，167772161 的 $L = 2^{25}$，原根均为 3。接口不检查素性或原根的阶；模数限 int 范围内的素数，乘积用 64 位。逆变换用逆单位根并乘 n^{-1}。

4.3 快速沃尔什变换

4.3.1 按位 OR、AND、XOR 卷积

功能：给定两个长度为 2^m 的数组，求按下标位运算合并的卷积 $c_s = \sum_{i \circ j = s} a_i b_j$，其中 $\circ$ 为按位或、与、异或之一（模 `mod`）；也单独提供原地变换 `transform`。

核心思路：三种运算都能逐位独立处理：每层把下标在该位上为 0、1 的一对元素 u, v 做变换。OR 做子集和 $v \leftarrow v + u$，AND 做超集和 $u \leftarrow u + v$，逆变换改为减法；XOR 做 Walsh 变换 $(u, v) \leftarrow (u + v, u - v)$，逆变换每层再乘 $1/2$（取 $(\mathrm{mod} + 1)/2$，故只要求模数为奇数）。`multiply` 对两数组正变换、逐点相乘，再逆变换。

代码依赖：ModInt（第 12 页，3.1.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：SetConvolution<mod=998244353>；Z 为 ModInt<mod>，Poly 为 vector<Z>。数组长度 N 必须是正的 2 次幂且 ≤INT_MAX，表示固定二进制位数的全集，长度 1 合法。transform(a,op,invert=false) 原地变换，op 只能为 |、&、^；| 为子集和，& 为超集和，^ 为 Walsh 变换。multiply(a,b,op) 返回 $\sum$ a[i]b[j]（满足 i op j=s），a、b 长度必须相同。mod 可为 ≥2 的 int 范围合数；XOR 要求 mod 为奇数，逆 2 用 (mod+1)/2，不使用费马逆元。O(N log N) 时间，原地变换 O(1) 额外空间，卷积 O(N) 空间。

```
template <int mod = 998244353> struct SetConvolution
{
    static_assert(mod >= 2);
    using Z = ModInt<mod>;
    using Poly = vector<Z>;

    static void transform(Poly &a, char op, bool invert = false)
    {
        assert(!a.empty() && a.size() <= INT_MAX);
        int n = a.size();
        assert((n & (n - 1)) == 0);
        assert(op == '|' || op == '&' || op == '^');
        assert(op != '^' || mod % 2 == 1);
        Z half = (mod + 1LL) / 2;
        for (int len = 1; len < n; len *= 2)
            for (int i = 0; i < n; i += len * 2)
                for (int j = 0; j < len; j++)
                {
                    Z &u = a[i + j], &v = a[i + j + len];
                    if (op == '|')
                        v = invert ? v - u : v + u;
                    else if (op == '&')
                        u = invert ? u - v : u + v;
                    else
                    {
                        Z x = u, y = v;
                        u = x + y;
                        v = x - y;
                        if (invert)
                        {
                            u = u * half;
                            v = v * half;
                        }
                    }
                }
    }
```

点乘并逆变换（接上页同一结构体）：

```
    static Poly multiply(Poly a, Poly b, char op)
    {
        assert(a.size() == b.size());
        transform(a, op);
        transform(b, op);
        for (int i = 0; i < int(a.size()); i++) a[i] = a[i] * b[i];
        transform(a, op, true);
        return a;
    }
};
```

模板题使用：Luogu P4717

题意：给两个长度为 2^m 的数组，分别求按下标 OR、AND、XOR 合并的卷积，模 998244353，按此顺序输出三行。multiply 的最后参数选择运算。

所需模板：ModInt（第 12 页，3.1.2 节）、SetConvolution（第 78 页，4.3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4717

```
int main()
{
    int m;
    scanf("%d", &m);
    int n = 1 << m;
    using F = SetConvolution<998244353>;
    F::Poly a(n), b(n);
    for (auto &x : a)
    {
        int v;
        scanf("%d", &v);
        x = v;
    }
    for (auto &x : b)
    {
        int v;
        scanf("%d", &v);
        x = v;
    }
    for (char op : string("|&^"))
    {
        auto c = F::multiply(a, b, op);
        for (int s = 0; s < n; s++) printf("%d%c", c[s].v, s + 1 == n ? '\n' : ' ');
    }
}
```

4.3.2 不相交子集卷积

功能：把下标视为集合，求不相交子集卷积 $c_S = \sum_{T \subseteq S} a_T\, b_{S \setminus T}$，即只统计 $i \cup j = S$ 且 $i \cap j = \varnothing$ 的乘积对（模 mod）。

核心思路：按集合大小（popcount，存于 rank）分层：f[k]、g[k] 只保留大小为 k 的下标，各自做 OR 变换（子集和）。$i \cup j = S$ 且 $|i| + |j| = |S|$ 恰等价于 i, j 不相交，所以对每个 k 在变换域逐点累加 $\sum_t f_t \cdot g_{k-t}$，逆 OR 变换后只取 $|S| = k$ 的位置写入答案。

代码依赖：ModInt（第 12 页，3.1.2 节）、SetConvolution（第 78 页，4.3.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：subset_convolution<mod=998244353>(a,b) 返回 c[s]=∑ a[t]b[s xor t]，求和枚举 t⊆s，等价于 i|j=s 且 i&j=0。依赖 SetConvolution，只使用 OR 变换；mod 为 ≥2 的 int 范围整数，允许偶数与合数。两个数组长度相同、非空，N=2^m≤INT_MAX；长度 1 的结果为 a[0]b[0]。按 popcount 分层，层间普通卷积再逆 OR 变换，保留总层数等于目标集合大小的项。O(m²2^m) 时间、O(m2^m) 空间；两组分层数组约占 8(m+1)2^m 字节，另有线性辅助数组。m=20 时仅分层数组约 168 MiB，使用前核对内存。

```
// c[s] = sum_{t subset s} a[t] * b[s xor t].
template <int mod = 998244353>
vector<ModInt<mod>> subset_convolution(const vector<ModInt<mod>> &a,
                                       const vector<ModInt<mod>> &b)
{
    using F = SetConvolution<mod>;
    using Poly = typename F::Poly;
    assert(!a.empty() && a.size() == b.size() && a.size() <= INT_MAX);
    int n = a.size(), m = 0;
    assert((n & (n - 1)) == 0);
    while ((1 << m) < n) m++;
    vector<Poly> f(m + 1, Poly(n)), g(m + 1, Poly(n));
    vector<int> rank(n);
    for (int s = 0; s < n; s++)
    {
        rank[s] = __builtin_popcount(unsigned(s));
        f[rank[s]][s] = a[s];
        g[rank[s]][s] = b[s];
    }
    for (int k = 0; k <= m; k++)
    {
        F::transform(f[k], '|');
        F::transform(g[k], '|');
    }
    Poly answer(n), h(n);
    for (int k = 0; k <= m; k++)
    {
        fill(h.begin(), h.end(), 0);
        for (int i = 0; i <= k; i++)
            for (int s = 0; s < n; s++) h[s] = h[s] + f[i][s] * g[k - i][s];
        F::transform(h, '|', true);
        for (int s = 0; s < n; s++)
            if (rank[s] == k) answer[s] = h[s];
    }
    return answer;
}
```

模板题使用：Luogu P6097

题意：把下标看作集合，求 c[s]=∑ a[t]*b[s⁀]，t 遍历 s 的子集，两部分不相交。输入首项为集合大小 m，数组长度 2^m；本题模数为 1000000009。

所需模板：ModInt（第 12 页，3.1.2 节）、SetConvolution（第 78 页，4.3.1 节）、subset_convolution（第 80 页，4.3.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P6097

```
int main()
{
    int m;
    scanf("%d", &m);
    int n = 1 << m;
    using F = SetConvolution<1000000009>;
    F::Poly a(n), b(n);
    for (auto &x : a)
    {
        int v;
        scanf("%d", &v);
        x = v;
    }
    for (auto &x : b)
    {
        int v;
        scanf("%d", &v);
        x = v;
    }
    auto c = subset_convolution<1000000009>(a, b);
    for (int s = 0; s < n; s++) printf("%d%c", c[s].v, s + 1 == n ? '\n' : ' ');
}
```

4.3.3 集合变换与不相交子集卷积

代码见第 78 页与第 80 页。长度 $N = 2^m$，第 s 项对应集合 s。OR 变换 $\widehat{a}_s = \sum_{t \subseteq s} a_t$，AND 变换求超集和，逆变换改加为减。XOR 变换系数 $(-1)^{\mathrm{popcount}(s\&t)}$，蝶形 $(u, v) \to (u + v, u - v)$，逆变换每层除以 2。OR、AND 可用任意模数；XOR 需 2 可逆，本实现限奇数模数并用 $(\mathrm{mod} + 1)/2$。三类卷积均为变换、逐项相乘、逆变换。

$$c_s = \sum_{t \subseteq s} a_t b_{s \setminus t} \qquad \text{（两集合不相交）}$$

按大小分层 $A_{i,s} = [|s| = i]a_s$，各层 OR 变换后 $H_{k,s} = \sum_{i=0}^{k} \widehat{A}_{i,s} \widehat{B}_{k-i,s}$，逆变换后取 $k = |s|$。时间 $O(m^2 2^m)$，空间 $O(m2^m)$；$m = 20$ 时两组分层数组占 168 MiB。

4.3.4 集合变换：逆变换对模数的要求

OR（子集和）单层 $(u,v)\mapsto(u,u+v)$，AND（超集和）单层 $(u+v,v)$，只需加减，任意模数。XOR 单层 $(u+v,u-v)$，两次等于乘 2，要求奇数模数，逆 2 取 $(p+1)/2$。长度为正的二次幂；不同于子集卷积。

4.4 Chirp Z 变换

4.4.1 Chirp Z：等比点求值

功能：给定多项式 f 与 a,r，一次求出 f 在等比点 $a,ar,ar^2,\dots,ar^{m-1}$ 处的 m 个值（模 NTT 素数），比逐点秦九韶求值快。

核心思路：关键恒等式 $ij=\binom{i+j}{2}-\binom{i}{2}-\binom{j}{2}$，于是 $f(ar^i)=r^{-\binom{i}{2}}\sum_j\left(f_ja^jr^{-\binom{j}{2}}\right)r^{\binom{i+j}{2}}$，成为一次差卷积。`inverse[t]` 存 $r^{-\binom{t}{2}}$，三角数指数由连乘递推，不计算平方级指数。把 $f_ja^jr^{-\binom{j}{2}}$ 逆序放入 `x`，$r^{\binom{t}{2}}$ 放入 `y`，循环卷积后第 $n-1+i$ 项乘 $r^{-\binom{i}{2}}$ 即为答案；循环回绕只落在下标小于 $n-1$ 处，不影响结果。$a=0$、$r=0$ 或 $r=1$ 时点值退化为 $f(a)$ 与 $f(0)$，用秦九韶直接填充。

代码依赖：ModInt（第 12 页，3.1.2 节）、NttConvolution（第 73 页，4.2.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：chirp_z(f,a,r,m) 返回 f(a*r^i)，i=0..m-1；f 为升幂系数数组，a、r 为 ModInt<mod>。m≥0、f.size()≤INT_MAX；空 f 表示零多项式，m=0 返回空。约定 r^0=1，因此 r=0 时首项为 f(a)、其余为 f(0)；a=0 或 r=1 也单独处理。普通分支依赖 NTT 素数及原根，非空长度 n 要求 n+m-1≤NTT::max_size。利用三角数指数与循环卷积，避免平方指数的整数溢出；O((n+m) log(n+m)) 时间、O(n+m) 空间，特殊分支 O(n+m)。默认模板参数 mod=998244353、primitive=3。输出顺序允许重复点。

```
template <int mod = 998244353, int primitive = 3>
vector<ModInt<mod>>
chirp_z(const vector<ModInt<mod>> &f, ModInt<mod> a, ModInt<mod> r, int m)
{
    using N = NttConvolution<mod, primitive>;
    using Z = typename N::Z;
    using Poly = typename N::Poly;
    assert(m >= 0 && f.size() <= INT_MAX);
    if (m == 0 || f.empty()) return Poly(m);
    if (a.v == 0 || r.v == 0 || r.v == 1)
    {
        Poly answer(m, f[0]);
        Z value = 0;
        for (int i = int(f.size()) - 1; i >= 0; i--) value = value * a + f[i];
        answer[0] = value;
        if (r.v == 1) fill(answer.begin(), answer.end(), value);
        return answer;
    }
    assert(f.size() + m - 1 <= N::max_size);
    int n = f.size(), size = 1;
    while (size < n + m - 1) size *= 2;
    Poly x(size), y(size), inverse(max(n, m));
    Z power = 1, triangle = 1, ri = r.inv();
    for (auto &v : inverse)
    {
        v = triangle;
        triangle = triangle * power;
        power = power * ri;
    }
```

构造循环卷积并取点值（接上页同一函数）：

```
    power = 1;
    for (int j = 0; j < n; j++)
    {
        x[n - 1 - j] = f[j] * power * inverse[j];
        power = power * a;
    }
    power = 1;
    triangle = 1;
    for (int j = 0; j < n + m - 1; j++)
    {
        y[j] = triangle;
        triangle = triangle * power;
        power = power * r;
    }
    N::ntt(x);
    N::ntt(y);
    for (int j = 0; j < size; j++) x[j] = x[j] * y[j];
    N::ntt(x, true);
    Poly answer(m);
    for (int i = 0; i < m; i++) answer[i] = x[n - 1 + i] * inverse[i];
    return answer;
}
```

模板题使用：Library Checker multipoint_evaluation_on_geometric_sequence

题意：给多项式 f 的升幂系数及 a、r，依次输出 f(a*r^i)，i=0..m-1，模 998244353。r=0 时首项仍为 f(a)，后续各项为 f(0)。

所需模板：ModInt（第 12 页，3.1.2 节）、NttConvolution（第 73 页，4.2.1 节）、chirp_z（第 82 页，4.4.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/multipoint_evaluation_on_geometric_sequence

```
int main()
{
    int n, m, a, r;
    scanf("%d%d%d%d", &n, &m, &a, &r);
    using Z = ModInt<998244353>;
    vector<Z> f(n);
    for (auto &x : f)
    {
        int v;
        scanf("%d", &v);
        x = v;
    }
    auto answer = chirp_z(f, Z(a), Z(r), m);
    for (int i = 0; i < m; i++) printf("%d%c", answer[i].v, i + 1 == m ? '\n' : ' ');
}
```

4.5 多项式牛顿迭代

4.5.1 形式幂级数求逆

功能：给定常数项非零的形式幂级数 $a(x)$，求 $b(x)$ 使 $a(x)b(x) \equiv 1 \pmod{x^n}$，返回前 n 项（模 998244353）。

核心思路：牛顿迭代：已知 $b \bmod x^k$ 时，$b \leftarrow b - b(ab-1)$ 把精度翻倍到 $2k$，初值为 a_0^{-1}。实现只计算新增的高半段：取 a 的前 $2k$ 项 `f` 与当前 b 做长度 $2k$ 的循环卷积，真实乘积低 k 项为 $1, 0, \ldots, 0$，而回绕只污染低 k 项，于是清零低半段后剩下的就是 $ab-1$ 的有效部分；再乘一次 b（复用其频域值 `g`），第 k 到 $m-1$ 项取负即为 b 新增的系数。

代码依赖：NttConvolution（第 73 页，4.2.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：FpsInverse::inverse(a,n) 返回 a 的逆级数前 n 项，模数固定 998244353。要求 $1 \le n \le 2^{23}$、a 非空且 a[0]≠0；a.size() 在 int 范围内，不足项视为零，多余项只取需要前缀。继承 NttConvolution<998244353> 的 Poly、Z、multiply。Newton 公式 b←b(2-ab) 只更新新增高半段：长度 2k 循环卷积的回绕只污染低 k 项，清除低半段后再乘 b，高半段即修正项。每轮五次长度 2k 变换，复用 b 的频域值。O(n log n) 时间、O(n) 工作空间；单位根缓存另按 NTT 约定。

```
struct FpsInverse : NttConvolution<998244353>
{
    static constexpr int mod = 998244353;

    static Poly inverse(const Poly &a, int n)
    {
        assert(n > 0 && n <= max_size && !a.empty() && a[0].v);
        Poly b{a[0].inv()};
        while ((int)b.size() < n)
        {
            int k = b.size(), len = 2 * k;
            int m = min(n, len);
            Poly f(a.begin(), a.begin() + min(m, (int)a.size()));
            Poly g = b;
            f.resize(len);
            g.resize(len);
            ntt(f);
            ntt(g);
            for (int i = 0; i < len; i++)
                f[i] *= g[i];
            ntt(f, true);
            // Only the high half of a*b is free of cyclic wraparound.
            fill(f.begin(), f.begin() + k, Z(0));
            ntt(f);
            for (int i = 0; i < len; i++)
                f[i] *= g[i];
            ntt(f, true);
            b.resize(m);
            for (int i = k; i < m; i++)
                b[i] = Z(0) - f[i];
        }
        return b;
    }
};
```

模板题使用：Library Checker inv_of_formal_power_series

题意：给出常数项非零的形式幂级数，求其乘法逆的前 n 项，模 998244353。系数按升幂输入，题目保证已经取模。

所需模板：ModInt（第 12 页，3.1.2 节）、NttConvolution（第 73 页，4.2.1 节）、FpsInverse（第 84 页，4.5.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/inv_of_formal_power_series

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    using F = FpsInverse;
    int n;
    cin >> n;
    F::Poly a(n);
    for (auto &x : a) cin >> x.v;
    auto b = F::inverse(a, n);
    for (int i = 0; i < n; i++) cout << b[i].v << (i + 1 == n ? '\n' : ' ');
}
```

4.5.2 多项式带余除法

功能：给定多项式 a, b（$b \neq 0$），求商 q 与余式 r，使 $a = qb + r$ 且 $\deg r < \deg b$（模 998244353）；另附去掉高次零系数的 `trim`。

核心思路：设 $\deg a = n$、$\deg b = m$，则商的次数为 $n - m$。记系数反转 $a^R(x) = x^n a(1/x)$，有 $a^R = q^R b^R + x^{n-m+1}(\cdots)$，故 $q^R \equiv a^R (b^R)^{-1} \pmod{x^{n-m+1}}$。代码取 a 反转后的前 $k = n - m + 1$ 项，与 b^R 的逆级数（`inverse`）相乘、截断、再反转得到 q；最后 $r = a - qb$ 只保留低 m 项并去高次零。

代码依赖：FpsInverse（第 84 页，4.5.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：PolynomialDivision::divide(a,b) 返回 pair<Poly,Poly>{商, 余式}，模数固定 998244353，系数从低次到高次。输入按值传入；先删除高次端零系数，零多项式统一返回空 vector；b 不可为零。支持零被除式、常数除数及 deg(a)<deg(b)。返回 a=q*b+r，deg(r)<deg(b)，商和余式都去掉高次零。依赖 FpsInverse 与 NttConvolution；反转后求逆得到商，再计算余式。输入各长度 $\leq 2^{22}$；令 N=max(a.size(),b.size())，时间 O(N log(N+1))，额外空间 O(N)。trim(a) 是可直接使用的去高次零工具。P4512 的固定长度输出需在使用处 resize 补零。

```
struct PolynomialDivision : FpsInverse
{
    static void trim(Poly &a)
    {
        while (!a.empty() && a.back().v == 0) a.pop_back();
    }

    static pair<Poly, Poly> divide(Poly a, Poly b)
    {
        assert(a.size() <= max_size / 2 && b.size() <= max_size / 2);
        trim(a);
        trim(b);
        assert(!b.empty());
        if (a.size() < b.size()) return {{}, a};
        int k = a.size() - b.size() + 1;
        Poly ar(a.rbegin(), a.rbegin() + k);
        Poly br(b.rbegin(), b.rend());
        br.resize(min(k, (int)br.size()));
        auto q = multiply(ar, inverse(br, k));
        q.resize(k);
        reverse(q.begin(), q.end());
        auto product = multiply(q, b);
        for (int i = 0; i < (int)a.size(); i++) a[i] -= product[i];
        a.resize(b.size() - 1);
        trim(a);
        return {q, a};
    }
};
```

模板题使用：Luogu P4512 多项式除法

题意：输入两个多项式的次数与系数，输出商的 n-m+1 项和余式的 m 项；模板返回的商、余式均已去高次零，打印前按题目要求补零。

所需模板：ModInt（第 12 页，3.1.2 节）、NttConvolution（第 73 页，4.2.1 节）、FpsInverse（第 84 页，4.5.1 节）、PolynomialDivision（第 86 页，4.5.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4512

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    using P = PolynomialDivision;
    P::Poly a(n + 1), b(m + 1);
    for (auto &x : a)
    {
        int value;
        cin >> value;
        x = value;
    }
    for (auto &x : b)
    {
        int value;
        cin >> value;
        x = value;
    }
    auto [q, r] = P::divide(a, b);
    q.resize(n - m + 1);
    r.resize(m);
    for (auto x : q) cout << x.v << ' ';
    cout << '\n';
    for (auto x : r) cout << x.v << ' ';
    cout << '\n';
}
```

4.5.3 多项式带余除法：先反转再求逆

`PolynomialDivision::divide(a,b)` 返回规范化的商、余式，零多项式为空向量；除数去高次零后须非零。设 $A = \deg a$、$B = \deg b$、$k = A - B + 1$：

$$\operatorname{rev}(q) = \operatorname{rev}(a)\ \operatorname{rev}(b)^{-1} \pmod{x^k}, \qquad r = a - qb.$$

定长输出需自行补零。各长度 $\le 2^{22}$，$O(N \log(N+1))$ 时间、$O(N)$ 空间。

4.6 多项式多点求值 | 快速插值

4.6.1 任意点多项式多点求值

问题描述：给定模 998244353 的多项式 f 和 m 个任意点 $x_0, \ldots, x_{m-1}$（可以重复），求全部 $f(x_i)$。点集固定后，同一棵树可对多个不同的 f 反复求值。

核心思路：利用 $f(x_i) = f \bmod (X - x_i)$。构造时在线段树形的乘积树上自底向上建出 p[u]$= \prod_{l \le i < r}(X - x_i)$：叶子为 $X - x_l$，内部结点用 multiply 合并左右儿子。eval 自顶向下传递余式：进入结点时，若当前多项式次数不低于 p[u] 就对它取模，再分别递归左右儿子。余式已为零多项式时直接返回（答案数组初值为 0）；只剩常数时把它填进整段点的答案，不再下探。

代码依赖：PolynomialDivision（第 86 页，4.5.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：固定模数 998244353，Poly 为升幂系数 vector<Z>。MultipointEvaluation(x) 建立点集乘积树，evaluate(f) 按输入点顺序返回 f(x[i])；允许重复点、零点、空点集、空或尾零多项式，返回长度始终等于点数。构造复制点值，查询按值接收 f，不改变树或调用者的左值，可多次复用；不再使用 f 时可 move 传入。系数个数 n≤2^22，点数 m<2^22，且内存可用；本地最大规模证据按报告列出，不表示已实测此绝对上限。每结点存首一多项式，查询将余式下传，叶值就是答案；仅求值，不包含快速插值。建树 O(m $\log^2$(m+1))，查询 O(n log(n+1)+m $\log^2$(m+1))，常驻空间 O(m log(m+1))、含临时数组 O(n+m log(m+1))，递归深度 O(log(m+1))。依赖 PolynomialDivision/FpsInverse/NTT 及模整数。

```
struct MultipointEvaluation : PolynomialDivision
{
    int n;
    vector<Poly> p;

    MultipointEvaluation(const Poly &x)
    {
        assert(x.size() < max_size / 2);
        n = x.size();
        p.resize(4 * max(1, n));
        if (n) build(1, 0, n, x);
    }

    void build(int u, int l, int r, const Poly &x)
    {
        if (r - l == 1)
        {
            p[u] = {Z(0) - x[l], 1};
            return;
        }
        int m = (l + r) / 2;
        build(2 * u, l, m, x);
        build(2 * u + 1, m, r, x);
        p[u] = multiply(p[2 * u], p[2 * u + 1]);
    }
```

余式下传与求值入口（接上页同一结构体）：

```
    void eval(int u, int l, int r, Poly f, Poly &a) const
    {
        if (f.size() >= p[u].size()) f = divide(move(f), p[u]).second;
        if (f.empty()) return;
        if (f.size() == 1)
        {
            fill(a.begin() + l, a.begin() + r, f[0]);
            return;
        }
        int m = (l + r) / 2;
        eval(2 * u, l, m, f, a);
        eval(2 * u + 1, m, r, move(f), a);
    }

    Poly evaluate(Poly f) const
    {
        assert(f.size() <= max_size / 2);
        trim(f);
        Poly a(n);
        if (n) eval(1, 0, n, move(f), a);
        return a;
    }
};
```

模板题使用：Library Checker Multipoint Evaluation

题意：给出 n 项升幂系数和 m 个任意横坐标，按输入顺序输出全部 f(x[i])，模 998244353。1≤n,m≤131072，系数和点均已取模，最高次系数非零；横坐标可以重复。构造一次乘积树后调用 evaluate；同一树可以继续求其他多项式的点值。

所需模板：ModInt（第 12 页，3.1.2 节）、NttConvolution（第 73 页，4.2.1 节）、FpsInverse（第 84 页，4.5.1 节）、PolynomialDivision（第 86 页，4.5.2 节）、MultipointEvaluation（第 88 页，4.6.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/multipoint_evaluation

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    using P = MultipointEvaluation;
    P::Poly f(n), x(m);
    for (auto &a : f)
    {
        int v;
        cin >> v;
        a = v;
    }
    for (auto &a : x)
    {
        int v;
        cin >> v;
        a = v;
    }
    auto a = P(x).evaluate(f);
    for (int i = 0; i < m; i++) cout << a[i].v << (i + 1 == m ? '\n' : ' ');
}
```

4.6.2 任意点多项式快速插值

问题描述：给定 n 个横坐标互异的点 (x_i, y_i)，在模 998244353 下求次数小于 n 的唯一多项式 f，使 $f(x_i) = y_i$，返回其系数。

核心思路：Lagrange 插值写成

$$f(X) = \sum_i \frac{y_i}{P'(x_i)} \cdot \frac{P(X)}{X - x_i},$$

其中 $P(X) = \prod_i (X - x_i)$。先用 `MultipointEvaluation` 建乘积树，根 `tree.p[1]` 就是 P；逐项求导得 P'，多点求值得到各 $P'(x_i)$（出现 0 说明横坐标重复），权重 $w_i = y_i / P'(x_i)$。然后在同一棵树上自底向上合并：叶子返回常数 w_l，内部结点返回 “左半结果乘右儿子乘积，加右半结果乘左儿子乘积”，根处即为 f。

代码依赖：MultipointEvaluation（第 88 页，4.6.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：polynomial_interpolation(x,y) 返回模 998244353 下次数 <m 的唯一插值多项式，升幂 vector 恰好 m 项并保留高次零。optional 为空表示长度不符或归一化横坐标重复；两个空数组返回有值的空 vector，不改输入。复用 MultipointEvaluation 乘积树及导数求值，按 y/P'(x) 递归合并；重复点只在独立求值接口合法。m≤$2^{22}-1$ 是既有除法/NTT 的统一充分界，官方最大实测 131072。时间 O(m $\log^2$(m+1)+m log p)，p=998244353；单点逆元沿用模幂，不省略 m log p 项。总空间 O(m log(m+1)+1)，递归 O(log(m+1))。

```
optional<MultipointEvaluation::Poly>
polynomial_interpolation(const MultipointEvaluation::Poly &x,
                         const MultipointEvaluation::Poly &y)
{
    using P = MultipointEvaluation;
    using Poly = P::Poly;
    if (x.size() != y.size()) return nullopt;
    assert(x.size() < P::max_size / 2);
    int n = x.size();
    if (!n) return Poly{};
    const P tree(x);
    Poly d(n);
    for (int i = 1; i <= n; i++) d[i - 1] = tree.p[1][i] * i;
    auto w = tree.evaluate(move(d));
    for (auto v : w)
        if (!v.v) return nullopt;
    for (int i = 0; i < n; i++) w[i] = y[i] * w[i].inv();
    auto combine = [&](auto &&self, int u, int l, int r) -> Poly
    {
        if (r - l == 1) return {w[l]};
        int m = (l + r) / 2;
        auto a = self(self, 2 * u, l, m);
        auto b = self(self, 2 * u + 1, m, r);
        a = P::multiply(move(a), tree.p[2 * u + 1]);
        b = P::multiply(move(b), tree.p[2 * u]);
        for (int i = 0; i < r - l; i++) a[i] += b[i];
        return a;
    };
    return combine(combine, 1, 0, n);
}
```

模板题使用：Library Checker Polynomial Interpolation

题意：给出 n 个互异横坐标及对应点值，输出唯一的次数小于 n 的多项式的 n 项升幂系数，模 998244353。1≤n≤131072，坐标与点值均已取模；先以乘积根多项式的导数求权重，再递归合并。接口返回 optional，长度不符或模意义重复横坐标返回空 optional，空输入返回有值的空向量；本题保证输入合法。

所需模板：ModInt（第 12 页，3.1.2 节）、NttConvolution（第 73 页，4.2.1 节）、FpsInverse（第 84 页，4.5.1 节）、PolynomialDivision（第 86 页，4.5.2 节）、MultipointEvaluation（第 88 页，4.6.1 节）、polynomial_interpolation（第 91 页，4.6.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/polynomial_interpolation

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    using Poly = MultipointEvaluation::Poly;
    Poly x(n), y(n);
    for (auto &a : x)
    {
        int v;
        cin >> v;
        a = v;
    }
    for (auto &a : y)
    {
        int v;
        cin >> v;
        a = v;
    }
    auto f = polynomial_interpolation(x, y);
    assert(f);
    for (int i = 0; i < n; i++) cout << (*f)[i].v << (i + 1 == n ? '\n' : ' ');
}
```

4.7 多项式初等函数

4.7.1 形式幂级数开根（含前导零与无解）

功能： 给定形式幂级数 $a(x)$，求 $b(x)$ 使 $b^2 \equiv a \pmod{x^n}$ 的前 n 项（模 998244353），无解时报告；`inverse_sqrt` 求 $a^{-1/2}$ 的前 n 项。

核心思路： 先求逆平方根：常数项用 `mod_sqrt` 取一个模平方根再求逆，之后按牛顿迭代 $b \leftarrow (3b - ab^3)/2$ 倍增。已有 k 项时，在长度 $4k$ 下一次算出 ab^3，回绕只落到低 k 项；新 b 的低 k 项不变，高半段等于 $-\frac{1}{2}(ab^3)$ 的对应项。`sqrt` 先找首个非零项次数 k：k 为奇数或该系数不是二次剩余则无解；否则除掉 x^k 得 `f`，用 $\sqrt{f} = f \cdot f^{-1/2}$ 求根，最后在前面补 $k/2$ 个零。

代码依赖：FpsInverse（第 84 页，4.5.1 节）、mod_sqrt（第 43 页，3.14.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约： FpsSqrt::sqrt(a,n) 返回 optional<Poly>，模 998244353；无解为 nullopt，有解恰好 n 项。$0 \le n \le 2^{22}$、a.size() 在 int 范围内，仅看前 n 项，不足补零；n=0 返回有值空 vector。全零返回全零，首个非零项次数 k 为奇数或系数非二次剩余时无解。否则提出 x^k，首项选较小模平方根，末尾未确定的 k/2 项取零。inverse_sqrt(a,n) 直接求所选根的逆：n>0 时要求常数项非零且为二次剩余，否则 nullopt；n=0 仍返回空。按 $b \leftarrow (3b - ab^3)/2$ 倍增，长度 4k 的循环卷积只回绕到低 k 项，保留旧低半段、更新高半段；sqrt 再乘原级数并移位。不枚举全部根。依赖 mod_sqrt、FpsInverse、NTT；O(n log(n+1)) 时间加一次标量开根的期望 O(log mod)，O(n) 工作空间，单位根缓存另按 NTT 约定。P5205 的 a[0]=1 得到首项 1。

```
struct FpsSqrt : FpsInverse
{
    static optional<Poly> inverse_sqrt(const Poly &a, int n)
    {
        assert(n >= 0 && n <= max_size / 2);
        if (n == 0)
            return Poly{};
        if (a.empty() || a[0].v == 0)
            return nullopt;
        auto roots = mod_sqrt(a[0].v, mod);
        if (roots.empty())
            return nullopt;
        Poly b{Z(roots[0]).inv()};
        Z half = Z(2).inv();
        while ((int)b.size() < n)
        {
            int k = b.size(), len = 4 * k;
            int m = min(n, 2 * k);
            Poly f(a.begin(), a.begin() + min(m, (int)a.size()));
            Poly g = b;
            f.resize(len);
            g.resize(len);
            ntt(f);
            ntt(g);
            for (int i = 0; i < len; i++)
                f[i] *= g[i] * g[i] * g[i];
            ntt(f, true);
            // b <- (3*b - a*b^3)/2; cyclic wrap only reaches below k.
            b.resize(m);
            for (int i = k; i < m; i++)
                b[i] = Z(0) - f[i] * half;
        }
        return b;
    }
```

平方根与前导零处理（接上页同一结构体）：

```
    static optional<Poly> sqrt(const Poly &a, int n)
    {
        assert(n >= 0 && n <= max_size / 2);
        int k = 0;
        while (k < n && (k >= (int)a.size() || a[k].v == 0))
            k++;
        if (k == n)
            return Poly(n);
        if (k & 1)
            return nullopt;
        int target = n - k;
        Poly f(a.begin() + k, a.begin() + min(n, (int)a.size()));
        auto inverse = inverse_sqrt(f, target);
        if (!inverse)
            return nullopt;
        Poly b = multiply(f, *inverse);
        b.resize(target);
        b.insert(b.begin(), k / 2, Z(0));
        b.resize(n);
        return b;
    }
};
```

模板题使用：Luogu P5205 多项式开根

题意：输入 n 项、保证 a[0]=1，输出平方根的前 n 项，首项取较小的 1。通用接口也可用于无解输入，须先检查 optional。

所需模板：ModInt（第 12 页，3.1.2 节）、NttConvolution（第 73 页，4.2.1 节）、FpsInverse（第 84 页，4.5.1 节）、mod_sqrt（第 43 页，3.14.1 节）、FpsSqrt（第 93 页，4.7.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5205

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    FpsSqrt::Poly a(n);
    for (auto &x : a)
    {
        int value;
        cin >> value;
        x = value;
    }
    auto b = FpsSqrt::sqrt(a, n);
    if (!b)
    {
        cout << -1 << '\n';
        return 0;
    }
    for (auto x : *b) cout << x.v << ' ';
    cout << '\n';
}
```

4.7.2 形式幂级数微积分、对数与指数

功能： 形式幂级数的基本运算：逐项求导 `derivative`、常数项为零的积分 `integral`、常数项为 1 时的对数 $\ln a$，以及常数项为 0 时的指数 $\exp a$，后两者返回前 n 项（模 998244353）。

核心思路： 积分所需的 $1/i$ 用线性递推 $i^{-1} = -\lfloor p/i \rfloor \cdot (p \bmod i)^{-1}$ 求出。$\ln a = \int a'/a$：截取前 n 项，导数乘逆级数（`inverse`）后积分。exp 用牛顿迭代 $b \leftarrow b(1 - \ln b + a)$，从 $b = 1$ 开始每轮精度翻倍：求 $\ln b$ 的前 m 项，构造 $a - \ln b + 1$，再与 b 相乘并截断到 m 项。

代码依赖：FpsInverse（第 84 页，4.5.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约： FpsFunctions 继承 FpsInverse，复用同一 NTT 和求逆。derivative(a) 形式求导；integral(a) 返回常数项为零的积分，要求 a.size()<998244353。log(a,n) 要求 n>0、a 非空、a[0]=1；exp(a,n) 要求 n>0、a 为空或 a[0]=0。返回前 n 项，不足输入项视为零，超过 n 的尾项不参与计算。内部每次卷积结果长度须 $\le 2^{23}$；n$\le 2^{22}$ 是统一充分界，输入数组长度须在 int 范围内。微积分 O(a.size())；log/exp 为 O(n log(n+1)) 时间和 O(n) 辅助空间。

```
struct FpsFunctions : FpsInverse
{
    static Poly derivative(const Poly &a)
    {
        Poly b;
        for (int i = 1; i < (int)a.size(); i++) b.push_back(a[i] * i);
        return b;
    }

    static Poly integral(const Poly &a)
    {
        assert(a.size() < mod);
        Poly b(a.size() + 1);
        vector<Z> inv(a.size() + 1);
        if (!a.empty()) inv[1] = 1;
        for (int i = 2; i <= (int)a.size(); i++)
            inv[i] = Z(mod - mod / i) * inv[mod % i];
        for (int i = 0; i < (int)a.size(); i++) b[i + 1] = a[i] * inv[i + 1];
        return b;
    }

    static Poly log(const Poly &a, int n)
    {
        assert(n > 0 && !a.empty() && a[0].v == 1);
        Poly f(a.begin(), a.begin() + min(n, (int)a.size()));
        auto b = multiply(derivative(f), inverse(f, n));
        b.resize(n - 1);
        return integral(b);
    }
```

```
    static Poly exp(const Poly &a, int n)
    {
        assert(n > 0 && (a.empty() || a[0].v == 0));
        Poly b{1};
        while ((int)b.size() < n)
        {
            int m = min(n, 2 * (int)b.size());
            auto t = log(b, m);
            for (int i = 0; i < m; i++) t[i] = (i < (int)a.size() ? a[i] : Z(0)) - t[i];
            t[0] = t[0] + 1;
            b = multiply(b, t);
            b.resize(m);
        }
        return b;
    }
};
```

模板题使用：Library Checker log_of_formal_power_series

题意：给定常数项为 1 的形式幂级数，求其形式对数的前 n 项，模 998244353。系数按升幂输入且已取模，结果常数项为 0。

所需模板：ModInt（第 12 页，3.1.2 节）、NttConvolution（第 73 页，4.2.1 节）、FpsInverse（第 84 页，4.5.1 节）、FpsFunctions（第 96 页，4.7.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/log_of_formal_power_series

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    using F = FpsFunctions;
    int n;
    cin >> n;
    F::Poly a(n);
    for (auto &x : a) cin >> x.v;
    auto b = F::log(a, n);
    for (int i = 0; i < n; i++) cout << b[i].v << (i + 1 == n ? '\n' : ' ');
}
```

4.7.3 形式幂级数快速幂（十进制大指数）

功能：给定形式幂级数 $a(x)$ 与以十进制字符串给出的非负整数 k（可能极大），求 $a(x)^k \bmod x^n$ 的前 n 项（模 998244353）。

核心思路：设首个非零项为 $c\,x^s$，则 $a = c\,x^s\,b$ 且 b 的常数项为 1，$a^k = c^k x^{sk} \exp(k \ln b)$。读指数时同时保留 $k \bmod p$（乘进 $\ln b$）、$k \bmod (p-1)$（`phi`，由费马小定理用于 c^k）以及截到 n 的真实值 `cap`（判断位移 sk 是否越界）。若 $sk \geq n$ 结果全零；否则只需 b 的前 $n - sk$ 项，做 log、乘 k、exp、乘 c^k 后平移 sk 位。$k = 0$ 时直接返回 1。

思路中的情况	代码中的处理	行
读入大指数	逐位维护 $k \bmod p$、$k \bmod (p-1)$ 及饱和于 n 的 `cap`	14
指数为 0	返回常数 1	18
位移 $sk \geq n$ 或级数全零	直接返回全零	25
提出 $c\,x^s$	截取 b 并整体乘 c^{-1}，使常数项为 1	29
计算 $\exp(k \ln b)$	对数乘 $k \bmod p$ 后取指数	32
乘回 $c^k x^{sk}$	乘 c^{phi} 并写到第 `shift` 项之后	34

代码依赖：FpsFunctions（第 96 页，4.7.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：FpsPower::power(a,exponent,n) 返回 a 的非负整数次幂模 x^n 的前 n 项，固定模数 998244353。exponent 为非空十进制数字 string_view，允许前导零；整数指数可传 to_string(k)。$0 \leq n \leq 2^{22}$，a.size() 须在 int 范围内；短输入补零、长输入只看前 n 项。n=0 返回空，零次幂（含零多项式的零次幂）返回 1。对首个非零项次数 s，若真实 s*k≥n 则返回全零；否则提取常数 c，计算 x^(s*k)*c^k*exp(k*log(a/(c*x^s)))。指数分别保留模 p、模 p-1 和截到 n 的值；不可仅用 k mod p 判零或算位移。依赖 FpsFunctions、FpsInverse、NTT。指数位数 L，时间 O(L+n log(n+1))、额外空间 O(n)，不支持负指数或小数指数。

```
struct FpsPower : FpsFunctions
{
    static Poly power(const Poly &a, string_view exponent, int n)
    {
        assert(n >= 0 && n <= max_size / 2 && !exponent.empty());
        long long k = 0, phi = 0;
        int cap = 0;
        for (char c : exponent)
        {
            assert('0' <= c && c <= '9');
            int digit = c - '0';
            k = (k * 10 + digit) % mod;
            phi = (phi * 10 + digit) % (mod - 1);
            cap = min(n, cap * 10 + digit);
        }
        Poly result(n);
        if (n == 0) return result;
        if (cap == 0)
        {
            result[0] = 1;
            return result;
        }
        int first = 0;
        while (first < n && (first >= (int)a.size() || a[first].v == 0)) first++;
        if (first == n || (first && cap > (n - 1) / first)) return result;
        int shift = first * cap, m = n - shift;
        Poly b(a.begin() + first, a.begin() + min(first + m, (int)a.size()));
        Z leading = a[first], inverse_leading = leading.inv();
        for (auto &x : b) x *= inverse_leading;
        auto f = log(b, m);
        for (auto &x : f) x *= Z(k);
        auto t = exp(f, m);
        Z scale = leading.pow(phi);
        for (int i = 0; i < m; i++) result[i + shift] = t[i] * scale;
        return result;
    }
};
```

模板题使用：Luogu P5273 多项式幂函数（加强版）

题意：输入 n 和十进制大指数 k，输出 a^k 的前 n 项；k 必须用 string 读入，不能只读 k mod p。允许前导零和 k=0。本程序同样适用于 P5245（a[0]=1）及 Library Checker Pow of Formal Power Series，三个入口均已本地验证。

所需模板：ModInt（第 12 页，3.1.2 节）、NttConvolution（第 73 页，4.2.1 节）、FpsInverse（第 84 页，4.5.1 节）、FpsFunctions（第 96 页，4.7.2 节）、FpsPower（第 98 页，4.7.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5273

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    string k;
    cin >> n >> k;
    FpsPower::Poly a(n);
    for (auto &x : a)
    {
        int value;
        cin >> value;
        x = value;
    }
    auto b = FpsPower::power(a, k, n);
    for (auto x : b) cout << x.v << ' ';
    cout << '\n';
}
```

4.7.4 级数开根：前导零和自由尾项

`FpsSqrt::sqrt(a,n)` 只用前 n 项。全零返回全零；首个非零项次数 k 为奇数或其系数非二次剩余则无解。否则 $a = x^k h$，迭代 $b \leftarrow (b + h/b)/2$ 求 h 的根前 $n-k$ 项后左移 $k/2$；末尾 $k/2$ 个自由系数取零，故解一般不止两个。返回 optional，$n = 0$ 返回空向量。$n \le 2^{22}$，$O(n \log(n+1))$ 加一次模开根，空间 $O(n)$。

4.7.5 ln 与 exp：只处理所需前缀

log 要求 $a_0 = 1$，$(\log a)' = a'/a$；exp 要求 $a_0 = 0$，$b \leftarrow b(1 + a - \log b)$ 倍增。对数输入先截到 n 项。积分要求 $|a| < 998244353$。$n \le 2^{22}$，$O(n \log(n+1))$ 时间、$O(n)$ 空间。

4.7.6 幂函数的大指数：保留三份指数信息

`FpsPower::power(a,exponent,n)` 的指数为十进制字符串，求模 x^n 的非负整数次幂；$n = 0$ 返回空，零次幂返回 1（含 0^0）。$k > 0$ 时设 $a = x^s ch$、$h(0) = 1$、$c \neq 0$；全零或 $sk \ge n$ 时全零，否则

$$a^k = x^{sk} c^k \exp(k \log h) \pmod{x^n}.$$

$k \bmod p$ 用于对数系数，$k \bmod (p-1)$ 用于 c^k，$\min(k, n)$ 用于零次幂与位移（要求 $n \le 2^{22} < p$）。判 $sk \ge n$ 用 $k > (n-1)/s$ 防溢出。指数位数 L，$O(L + n \log(n+1))$ 时间、$O(n)$ 空间。

4.8 常系数齐次线性递推

4.8.1 特征多项式求线性递推第 k 项

问题描述：已知 k 阶常系数齐次线性递推 $a_i = \sum_{j=1}^{k} c_{j-1} a_{i-j}$ 及初值 $a_0, \ldots, a_{k-1}$，求第 n 项（n 很大），模数可为合数。

核心思路：设特征多项式 $P(x) = x^k - \sum_{j=1}^{k} c_{j-1} x^{k-j}$。把 a_i 看作单项式 x^i 的线性函数，递推说明 x^i 可以按 P 降次而值不变，故若 $x^n \equiv \sum_{i<k} r_i x^i \pmod P$，则 $a_n = \sum_i r_i a_i$。`multiply` 做朴素多项式乘法后，从最高次往下用 $x^i = \sum_j c_{j-1} x^{i-j}$ 消去次数不小于 k 的项；以此快速幂求 $x^n \bmod P$（$k = 1$ 时 x 已降为常数 c_0），最后与初值点乘。

接口契约：recurrence_nth(init,c,n,p) 使用 0-based 下标，n 为 unsigned long long。init 与 c 长度均为阶数 k；a[i]=∑ c[j-1]a[i-j]。2≤p≤INT_MAX，数值均在 [0,p)，本函数允许合数模数。空系数约定全零序列。计算 x^n 对特征多项式取余再与初值点乘，O(k² log(n+1)) 时间、O(k) 空间；不依赖 BM 或其他模板。较大阶数可使用本库 BostanMori::recurrence，须满足其 NTT 素数模数与容量条件；本函数允许合数模数。

```cpp
// init has exactly k entries; c[j-1] multiplies a[n-j]. Modulus need not be prime.
int recurrence_nth(const vector<int> &init,
                   const vector<int> &c,
                   unsigned long long n,
                   int p = 998244353)
{
    assert(p >= 2 && init.size() == c.size());
    int k = c.size();
    if (!k) return 0;
    for (int x : init) assert(0 <= x && x < p);
    for (int x : c) assert(0 <= x && x < p);
    if (n < (unsigned)k) return init[n];
    auto multiply = [&](const vector<int> &a, const vector<int> &b)
    {
        vector<int> t(2 * k - 1);
        for (int i = 0; i < k; i++)
        {
            for (int j = 0; j < k; j++) t[i + j] = (t[i + j] + 1LL * a[i] * b[j]) % p;
        }
        for (int i = 2 * k - 2; i >= k; i--)
        {
            for (int j = 1; j <= k; j++)
                t[i - j] = (t[i - j] + 1LL * t[i] * c[j - 1]) % p;
        }
        t.resize(k);
        return t;
    };
    vector<int> answer(k), x(k);
    answer[0] = 1;
    if (k == 1)
        x[0] = c[0];
    else
        x[1] = 1;
    while (n)
```

```
    {
        if (n & 1) answer = multiply(answer, x);
        x = multiply(x, x);
        n >>= 1;
    }
    long long value = 0;
    for (int i = 0; i < k; i++) value = (value + 1LL * answer[i] * init[i]) % p;
    return value;
}
```

模板题使用：Luogu P5487

题意：给出 n 个初值，恢复唯一的最短递推并求第 m 项，n≤10000、n<m≤10^9、阶数 ≤5000，模 998244353。BM 返回 c[j-1] 乘 a[i-j]，首行仅输出系数、不带阶数；裁取恰好 c.size() 个初值后调用 recurrence_nth。全零序列空递推仍输出空首行和第二行 0。复杂度 O(nk+k^2log m)，O(n+k) 空间。

所需模板：berlekamp_massey（第 181 页，11.1 节）、recurrence_nth（第 101 页，4.8.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5487

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    unsigned long long m;
    cin >> n >> m;
    vector<int> s(n);
    for (auto &x : s)
    {
        cin >> x;
        x %= 998244353;
        if (x < 0) x += 998244353;
    }
    auto c = berlekamp_massey(s);
    for (int i = 0; i < (int)c.size(); i++)
    {
        if (i) cout << ' ';
        cout << c[i];
    }
    cout << '\n';
    vector<int> init(s.begin(), s.begin() + c.size());
    cout << recurrence_nth(init, c, m) << '\n';
}
```

4.8.2 快速递推与有理生成函数单项

问题描述：求有理生成函数的单项系数 $[x^n]\frac{p(x)}{q(x)}$；以及给定 k 阶常系数齐次线性递推与初值，求第 n 项（模 NTT 素数）。

核心思路：分子分母同乘 $q(-x)$：$\frac{p(x)}{q(x)}=\frac{p(x)q(-x)}{q(x)q(-x)}$，新分母只含偶次项，记为 $V(x^2)$，所以只需分子中次数与 n 同奇偶的项，记为 $x^{n \bmod 2}U(x^2)$，于是 $[x^n]\frac{p}{q}=[x^{\lfloor n/2\rfloor}]\frac{U}{V}$。`coefficient` 每轮两次 NTT 乘法后抽取奇偶项、$n$ 减半，直到 $n=0$ 时答案为 p_0/q_0。`recurrence` 取 $q=1-\sum_i c_i x^{i+1}$、$p=(\text{init}\cdot q) \bmod x^k$，转化为上述问题。

代码依赖：NttConvolution（第 73 页，4.2.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约： BostanMori<mod=998244353,primitive=3>，依赖参数化 NTT；素数模数与原根条件同 NttConvolution。Poly 为升幂系数 vector<Z>。coefficient(p,q,n) 返回 [x^n]p(x)/q(x)，q 非空且 q[0]≠0，p 可空或次数不小于 q；令 d=max(p.size(),q.size())，要求 2d-1≤NTT::max_size。recurrence(init,c,n) 返回 a[n]，从下标 k 起 a[i]=c[0]a[i-1]+⋯+c[k-1]a[i-k]，init、c 长度均为 k；k=0 约定全零。非初值查询要求 2k+1≤NTT::max_size。n 支持完整 unsigned long long。每轮两次卷积后抽取奇偶项，O(d log d log(n+1)) 时间、O(d) 空间；递推接口 d=k+1。

```
template <int mod = 998244353, int primitive = 3> struct BostanMori
{
    using N = NttConvolution<mod, primitive>;
    using Z = typename N::Z;
    using Poly = typename N::Poly;
    using U = unsigned long long;

    // Coefficient of x^n in p(x) / q(x), with q[0] != 0.
    static Z coefficient(Poly p, Poly q, U n)
    {
        assert(!q.empty() && q[0].v != 0);
        assert(max(p.size(), q.size()) <= (N::max_size + 1ULL) / 2);
        while (n && !p.empty())
        {
            Poly r = q;
            for (int i = 1; i < int(r.size()); i += 2) r[i] = Z(0) - r[i];
            Poly a = N::multiply(p, r);
            Poly b = N::multiply(q, r);
            p.clear();
            q.clear();
            for (int i = n & 1; i < int(a.size()); i += 2) p.push_back(a[i]);
            for (int i = 0; i < int(b.size()); i += 2) q.push_back(b[i]);
            n >>= 1;
        }
        return p.empty() ? Z(0) : p[0] / q[0];
    }
```

从初值构造递推生成函数（接上页同一结构体）：

```
    // a[i] = c[0]*a[i-1] + ... + c[k-1]*a[i-k], for i >= k.
    static Z recurrence(const Poly &init, const Poly &c, U n)
    {
        assert(init.size() == c.size());
        int k = c.size();
        if (k == 0) return 0;
        if (n < init.size()) return init[n];
        assert(k + 1ULL <= (N::max_size + 1ULL) / 2);
        Poly q(k + 1);
        q[0] = 1;
        for (int i = 0; i < k; i++) q[i + 1] = Z(0) - c[i];
        Poly p = N::multiply(init, q);
        p.resize(k);
        return coefficient(p, q, n);
    }
};
```

模板题使用：Luogu P4723

题意：给 k 个递推系数和 a[0..k-1]，求 a[n] 模 998244353。c[0] 乘前一项，c[k-1] 乘前 k 项；输入先系数后初值，recurrence 参数先初值后系数。通过 ModInt 赋值归一化负数。

所需模板：ModInt（第 12 页，3.1.2 节）、NttConvolution（第 73 页，4.2.1 节）、BostanMori（第 102 页，4.8.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4723

```
int main()
{
    unsigned long long n;
    int k;
    scanf("%llu%d", &n, &k);
    using B = BostanMori<>;
    B::Poly c(k), init(k);
    for (auto &x : c)
    {
        int v;
        scanf("%d", &v);
        x = v;
    }
    for (auto &x : init)
    {
        int v;
        scanf("%d", &v);
        x = v;
    }
    printf("%d\n", B::recurrence(init, c, n).v);
}
```

4.8.3 线性递推：系数方向与外推条件

约定 $a_i = \sum_{j=1}^{k} c_{j-1} a_{i-j}$，初值 $a_0, \ldots, a_{k-1}$，特征多项式

$$F(x) = x^k - c_0 x^{k-1} - c_1 x^{k-2} - \cdots - c_{k-1}.$$

若 $x^n \bmod F(x) = \sum_{i=0}^{k-1} r_i x^i$，则 $a_n = \sum_{i=0}^{k-1} r_i a_i$。BM 要求素数模数；已知阶数至多 d 时至少给出前 $2d$ 项。系数末尾的零不能删除，否则改变递推开始适用的位置。BM 加特征多项式求余求第 m 项：时间 $O(nk + k^2 \log m)$，空间 $O(n + k)$。

4.8.4 Bostan–Mori 与快速递推单项

代码见第 102 页。对 $a_i = \sum_{j=1}^{k} c_{j-1} a_{i-j}$，令 $Q(x) = 1 - \sum_{j=1}^{k} c_{j-1} x^j$，$P(x) = (Q(x) \sum_{i=0}^{k-1} a_i x^i) \bmod x^k$，则 $a_n = [x^n]P/Q$。系数按前一项、前两项顺序给出，初值从 a_0 开始。

$$\frac{P(x)}{Q(x)} = \frac{P(x)Q(-x)}{Q(x)Q(-x)} = \frac{U_0(x^2) + xU_1(x^2)}{V(x^2)},$$

保留与 n 同奇偶的分子项、分母偶数项，n 右移一位；$n = 0$ 时答案为 $P(0)/Q(0)$。分母常数项须非零；不要求真分式，空分子返回零。初始最大长度 d 须满足 $2d - 1$ 不超过 NTT 容量。时间 $O(d \log d \log(n + 1))$，空间 $O(d)$；下标可达完整无符号 64 位。

4.9 多项式平移 | 连续点值平移

4.9.1 多项式系数平移

功能： 给定多项式 $f(x)$ 的升幂系数和常数 c，求 $f(x+c)$ 的升幂系数（模 NTT 素数）。

核心思路： 二项式展开得 $[x^j]f(x+c) = \frac{1}{j!}\sum_{i\geq j} f_i\, i! \cdot \frac{c^{i-j}}{(i-j)!}$，右边是关于差 $i-j$ 的卷积。把 $f_i\, i!$ 逆序放进 a（存在 a[n-1-i]），把 $c^t/t!$ 放进 b，一次 NTT 乘法后第 $n-1-j$ 项正是上式的求和部分，再乘 $1/j!$ 即得答案。阶乘与逆阶乘先线性预处理。

代码依赖：ModInt（第 12 页，3.1.2 节）、NttConvolution（第 73 页，4.2.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约： polynomial_shift<mod=998244353,primitive=3>(f,c) 返回 f(x+c) 的升幂系数，结果长度始终与 f 相同，保留末尾零；空输入返回空。参数 c 使用 ModInt<mod>。依赖参数化 NTT，mod 为素数且 primitive 为原根；非空长度 n 要求 2n-1≤NTT::max_size，因此阶乘均可逆。逆序的 f[i]i! 与 c^i/i! 卷积后提取对应项，O(n log n) 时间、O(n) 空间。c=0 合法。

```
template <int mod = 998244353, int primitive = 3>
vector<ModInt<mod>> polynomial_shift(const vector<ModInt<mod>> &f, ModInt<mod> c)
{
    using N = NttConvolution<mod, primitive>;
    using Z = typename N::Z;
    using Poly = typename N::Poly;
    if (f.empty()) return {};
    assert(f.size() <= (N::max_size + 1ULL) / 2);
    int n = f.size();
    Poly fact(n), inv(n), a(n), b(n);
    fact[0] = 1;
    for (int i = 1; i < n; i++) fact[i] = fact[i - 1] * i;
    inv[n - 1] = fact[n - 1].inv();
    for (int i = n - 1; i > 0; i--) inv[i - 1] = inv[i] * i;
    Z power = 1;
    for (int i = 0; i < n; i++)
    {
        a[n - 1 - i] = f[i] * fact[i];
        b[i] = power * inv[i];
        power = power * c;
    }
    auto product = N::multiply(a, b);
    for (int i = 0; i < n; i++) a[i] = product[n - 1 - i] * inv[i];
    return a;
}
```

模板题使用：Library Checker polynomial_taylor_shift

题意：给多项式 f 的 n 个升幂系数和常数 c，输出 f(x+c) 的 n 个升幂系数，模 998244353。平移方向为加 c，c=0 时原样返回。

所需模板：ModInt（第 12 页，3.1.2 节）、NttConvolution（第 73 页，4.2.1 节）、polynomial_shift（第 106 页，4.9.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/polynomial_taylor_shift

```
int main()
{
    int n, c;
    scanf("%d%d", &n, &c);
    using Z = ModInt<998244353>;
    vector<Z> f(n);
    for (auto &x : f)
    {
        int v;
        scanf("%d", &v);
        x = v;
    }
    auto answer = polynomial_shift(f, Z(c));
    for (int i = 0; i < n; i++) printf("%d%c", answer[i].v, i + 1 == n ? '\n' : ' ');
}
```

4.9.2 Taylor 平移与等比点求值

代码见第 106 页和第 82 页。升幂系数，长度 n 表示次数小于 n。

$$[x^i]f(x+c)=\frac{1}{i!}\sum_{j=i}^{n-1}f_j j!\frac{c^{j-i}}{(j-i)!}.$$

倒序 $f_j j!$ 与 $c^j/j!$ 卷积，取第 $n-1-i$ 项乘 $1/i!$。要求 NTT 素数且 $2n-1$ 不超过变换容量；空数组返回空数组。时间 $O(n\log n)$、空间 $O(n)$。这不是点值平移。

Chirp Z 返回 $f(ar^i)$，$0\le i<m$。设 $T_j=j(j-1)/2$，

$$f(ar^i)=r^{-T_i}\sum_{j=0}^{n-1}f_j a^j r^{-T_j}r^{T_{i+j}}.$$

倒序第一项数组与 r^{T_j} 卷积，取第 $n-1+i$ 项；循环卷积长度取不小于 $n+m-1$ 的二次幂即可。要求 $n+m-1$ 不超过 NTT 容量，时间 $O((n+m)\log(n+m))$、空间 $O(n+m)$。$r=0$（约定 $r^0=1$）、$a=0$、$r=1$ 单独处理；$m=0$ 返回空，空输入返回 m 个零。

4.10 符号化方法

4.10.1 生成函数建模到模板：三个可核验例子

记 N 为所求最大次数、$L=N+1$、$M(L)=O(L\log L)$ 为 NTT 乘法复杂度。第 73 页 `NttConvolution<p,g>` 可指定素数与原根；`FpsInverse`（第 84 页）、`FpsFunctions`（第 96 页）、`FpsPower`（第 98 页）固定 $p=998244353$。FPS 长度参数是项数，求 x^N 须传 $N+1$。NTT 检查完整乘积长度 $a.size()+b.size()-1$，固定模数最大 2^{23}，FPS 取 $L\le 2^{22}$。

4.10.1.1 例一：不同盒子、相同球、每盒有上下界（OGF 乘积）

$l_i\le a_i\le u_i$、$\sum a_i=N$。令 $c_i=u_i-l_i$、$R=N-\sum l_i$；若 $u_i<l_i$ 或 $R<0$ 答案为 0，否则

$$[x^R]\prod_{i=1}^{k}(1+x+\cdots+x^{c_i}).$$

只保留 $0,\dots,R$ 次：c_i 截至 R，乘后超过 $R+1$ 才截短，不要把中间多项式补零。小规模用滑动窗口

$$d_i(s)=d_i(s-1)+d_{i-1}(s)-[s>c_i]d_{i-1}(s-c_i-1),$$

$d_0(0)=1$、负下标为 0，$O(k(R+1))$ 时间、$O(R+1)$ 空间，任意模数可用。分治 NTT 为 $O(k+M(D+1)\log(k+1))$，$D=\sum\min(c_i,R)$。所有 $c_i=c$ 时可用 `FpsPower::power(P,to_string(k),R+1)`，$P=1+\cdots+x^{\min(c,R)}$，$O(M(R+1)+\log(k+1))$；或在 $k\ge 1$ 时容斥

$$\sum_{j=0}^{\min(k,\lfloor R/(c+1)\rfloor)}(-1)^j\binom{k}{j}\binom{R-j(c+1)+k-1}{k-1},$$

组合数接口见第 113 页。$k=0$ 单独返回 $[N=0]$。

4.10.1.2 例二：步长 1、2 的有序序列（OGF 求逆）

单步 OGF $B(x)=\sum b_sx^s$（$s\ge 1$），有序序列 $F=(1-B)^{-1}\bmod x^{N+1}$，$O(M(N+1))$ 时间、$O(N+1)$ 空间。步长 1、2 时调用：

```
FpsInverse::inverse({1,-1,-1},N+1)
```

要求 $B(0)=0$；只区分各长度使用次数时应改为 $\prod_s(1-x^s)^{-1}$。N 巨大且步长有界时改用远项算法。

4.10.1.3 例三：不同元素分成单点与无序对（EGF 指数）

块的 EGF 为

$$B(x)=x+\frac{x^2}{2!},\qquad F(x)=\exp B(x),\qquad a_n=n![x^n]F(x).$$

在 $p=998244353$、$N<p$ 下用 `FpsFunctions::exp(b,N+1)`，再乘 `Binomial` 的 `fac[N]`；$O(M(N+1))$。也有 $a_0=a_1=1$、$a_n=a_{n-1}+(n-1)a_{n-2}$。恰好 k 块为 $n![x^n]B(x)^k/k!$：用 `FpsPower::power(b,to_string(k),n+1)` 取第 n 项再乘 `fac[n]`、`ifac[k]`，表预处理至 $\max(n,k)<p$；先判 $k\le n\le 2k$，$(n,k)=(0,0)$ 为 1。盒子带标号时不除以 $k!$；允许空盒时单盒因子添 1。

4.11 Lagrange 反演

4.11.1 隐式生成函数与 Lagrange 反演

用于 $T = x\Phi(T)$ 只问 T 或 $H(T)$ 某一项的情形；这里是**复合逆**，不同于第 165 页的点值插值。

4.11.1.1 适用条件与取系数公式

特征零域上形式幂级数，$\Phi(0) \neq 0$（一般 $\mathbb{Q}$-代数中要求 $\Phi(0)$ 可逆），$T(x) = x\Phi(T(x))$、$T(0) = 0$ 唯一确定 T。对 $n \geq 1$：

$$\boxed{[x^n]H(T(x)) = \frac{1}{n}[t^{n-1}]H'(t)\Phi(t)^n,} \qquad \boxed{[x^n]T(x)^k = \frac{k}{n}[t^{n-k}]\Phi(t)^n \ (n \geq k \geq 1).}$$

边界：$[x^0]H(T) = H(0)$；$[x^n]T^0 = [n = 0]$；$k \geq 1$、$n < k$ 时为 0。不要求解析收敛。

4.11.1.2 模型一按内部结点计数的满有序多叉树

每个结点为叶子或恰有 $d \geq 2$ 个有序孩子，按内部结点计数：$A = 1 + xA^d$，$a_0 = 1$，$n \geq 1$ 时

$$a_n = \frac{1}{n}\binom{dn}{n-1} = \frac{1}{(d-1)n+1}\binom{dn}{n}.$$

孩子无序时不能用 A^d。用 `Binomial<p> c(d*n)` 须在宽类型中算 dn 并保证 $dn < p$，否则按第 113 页选组合数算法。

4.11.1.3 模型二带标号有根树的 EGF

根的孩子无序：$T = x\exp T$，$r_n = n![x^n]T = n^{n-1}$（$n \geq 1$）；指定根时为 n^{n-2}（$n \geq 2$）。EGF 系数须乘回 $n!$。

4.11.1.4 落到现有接口及模数边界

单个幂系数。第 98 页的 `FpsPower::power` 固定 $p = 998244353$；在 $1 \leq k \leq n < p$、$L = n - k + 1 \leq 2^{22}$ 下：

```
using F = FpsPower;
using Z = F::Z;
int l = n - k + 1;
auto q = F::power(phi, to_string(n), l);
Z answer = Z(k) * Z(n).inv() * q[n - k];
```

长度是项数（传 $n-k+1$），指数传完整十进制串。单次 $O(L\log(L+1)+\log(n+1))$、空间 $O(L)$；对所有 $n \leq N$ 逐个求为 $O(N^2\log(N+1))$。一般 H：先算 $q = \Phi^n \bmod t^n$，答案为 $n^{-1}\sum_{i=1}^{n} iH_i q_{n-i}$。第 96 页的 `log`（首项 1）、`exp`（首项 0）可构造已知表达式，不能解任意隐式方程；`Binomial<p> c(N)` 要求 $N < p$。第 84 页的 `inverse` 是乘法逆，要求常数项非零，不能求 $f = t/\Phi(t)$ 的复合逆。

模数。在 $\mathbb{F}_p$ 中使用 $1/n$ 公式要求 $p \nmid n$，EGF 的 $1/j!$ 不能跨过 $j = p$；否则改用无除法递推或提升精度后再整除。

4.12 普通生成函数

4.12.1 生成函数与递推

OGF 乘法对应普通卷积；EGF $\sum a_n x^n/n!$ 乘法对应 $c_n = \sum_k \binom{n}{k} a_k b_{n-k}$。求逆要求常数项可逆；$\log A$ 要求 $A(0) = 1$，$\exp A$ 要求 $A(0) = 0$；有限域积分要求 i 不被特征整除。

$$B_{\text{new}} = B(2 - AB) \pmod{x^{2m}}, \qquad (\log A)' = A'/A.$$

BM 外推未来项前须另知序列有不超过相应阶数的线性递推（如由矩阵转移、有限状态得到阶数上界）。